2026 AI 先锋未来人才大赛 · 安克创新命题 · 华南赛区

Anker AI 原生产品定义系统

让 AI 做你的**超级智囊 · 用户替身 · 行业专家**

从真实用户数据，到一份可溯源、可验证、可量化的产品提案

一套真实可运行的多智能体系统：把安克 JML / BEES / AMI 三大产品方法论平台做成 AI 原生版，端到端从真实评论跑出产品提案，并用受控实验回答——**AI 驱动相较”经验拍脑袋”的本质不同**。

戴尚好

中国科学技术大学

2026 年 6 月

参赛信息与项目链接

参赛者	戴尚好
学校	中国科学技术大学
赛道 / 命题	AI + 研发创新；安克创新——用 AI 重做一次产品设计与定义
赛区	华南
邮箱	bcefghj@163.com
个人主页	https://bcefghj.github.io
GitHub 源码	https://github.com/bcefghj/anker-ai-product-studio
在线 Demo	http://47.119.112.225/anker/app/
项目宣传页	http://47.119.112.225/anker/
项目导航 Hub	http://47.119.112.225/

一句话摘要。 本作品不是 PPT，而是一套真实可运行、数据可溯源、可评测的多智能体系统。它把安克内部真实的产品方法论平台 JML/BEES/AMI 做成 AI 原生版，让 AI 扮演超级智囊、用户替身、行业专家，端到端从真实用户评论跑出一份产品提案（Working Backwards PR/FAQ），并用一个受控对比实验量化回答命题：在同一 brief 下，AI 原生工作流相较”经验拍脑袋”，命中真实痛点率由 33% 提升到 100%、可溯源证据引用由 0 增至 56 条、预测 NPS 提升约 30 分。系统遵循对标大厂的 AIGC 工程规范（四层架构、AI Rule、Pre-PR、跨厂商模型对抗 CR），并已部署上线（与既有项目按目录/端口/路由隔离，互不影响）。

目录

第一部分　命题理解与背景洞察

1 大赛与命题

2026 AI 先锋未来人才大赛由飞书与数智教育发展国际大学联盟主办、教育部指导。安克创新（深交所 300866）给出的命题是：

> 假设你要为安克设计一款创新产品，让 *AI* 做你的超级智囊、用户替身、行业专家——
> 用这种新方法从头做一次产品设计与定义，比比看和靠经验拍脑袋有什么本质不同？

交付物有两层：(1) 选一个安克品类，设计并演示一套”AI 原生” 的产品设计与定义工作流，产出一份产品提案（用户洞察 + 产品概念 + 可行性分析）；(2) 说明这个 AI 原生方法论，并与传统方法对比（AI 驱动 vs 经验驱动，差异是什么）。

2 安克是谁：用数据说话

指标（2025 年报）	数据
营收	305.14 亿元（首破 300 亿，同比 +23.49%）
归母净利润	25.45 亿元（同比 +20.37%）
研发投入	28.93 亿元（同比 +37.20%，占营收 9.48%）
研发人员	3,549 人（占总员工 56.30%）
全球用户	超 2 亿，覆盖 180+ 国家/地区
三大产品线	充电储能（Anker）/ 智能创新（eufy）/ 智能影音（soundcore）

表 1: 安克创新基本面（研发增速显著高于营收增速，主动以利润率换技术领先）

3 为什么是这道题：企业意图分析

安克自 2023 年起全面 **All in AI**，构建了企业级 AI 能力底座 **AIME 平台**（沉淀 300+ 活跃 Agent，其中已包含**客户之声 VOC 洞察**、**需求生成**、**产品文档**等研发 Agent），研发代码采纳率突破 50%。在产品方法论层面，安克长期以三大平台驱动：

- **JML**（Joint Maker Lab）：用户调研平台，把用户反馈转化为产品需求；
- **BEES**（Best Experience Enhancement System）：用户体验评估，转化为设计优化；
- **AMI**（Anker Market Insights）：市场洞察（趋势、竞品、渠道、全球数据）；

并以 **NPS**（净推荐值）作为产品竞争力的核心指标，方法论本质是” 用户洞察（VOC）+ 第一性原理” 的双轮驱动。

CIO 龚银的两个判断尤其关键：其一，”在定义一款产品时，从用户调研、市场洞察到用户洞察等各个环节，都可以深度融入 AI 能力”；其二，”当前 AI 的创新很大程度是通过**技术原生驱动**创造全新 C 端体验，而非完全基于传统用户需求洞察”，且企业场景对**确定性**要求极高，”不能有幻觉”。

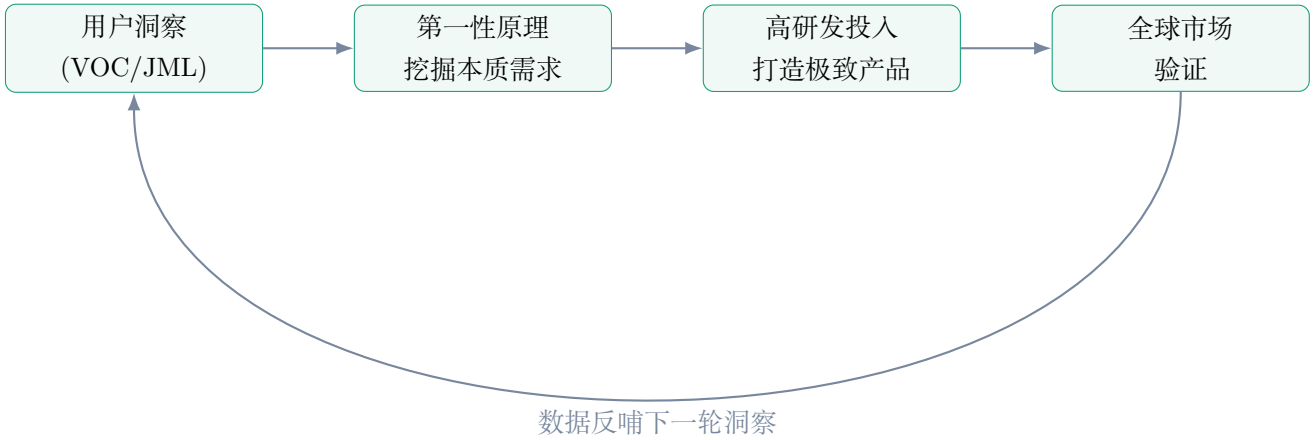

图 1: 安克”双轮驱动”产品方法论——本系统将其每个环节 AI 原生化

我们的判断：命题表面是”设计一款产品”，本质是要验证候选人能否用 AI 把”产品经理拍脑袋”重构为”**数据可溯源**、**可验证**、**可量化**、**可迭代**”的工作流，并能把方法论沉淀为可复用的 SOP——这恰好对接安克 AIME 平台”沉淀产品方法论超级大脑”的方向。因此本作品的主体是**方法论 + 工作流系统 + 一份提案 + AI/经验量化对比**，产品本身只是方法论的演示载体。

4 行业洞察：竞品的 AI 化与赛道趋势

竞品	代表性 AI 音频策略	启示
Bose	QuietComfort Ultra 的 Custom-Tune AI 音频校准	个性化音频是竞争焦点
Sony	WF-1000XM 系列 Precise Voice Pickup	端侧人声分离技术路线
Samsung	Galaxy Buds 的 AI Live Translate	实时翻译赋能场景
Apple	AirPods Pro 的 Hearing Aid 助听功能	AI 赋予耳机健康/医疗属性

表 2: 主要竞品 AI 音频策略——本系统在 AMI 模块对其评论做真实拆解

结合安克自身的技术原生能力（**Thus™ 芯片**：全球首款 NOR Flash 存算一体 CIM AI 芯片，算力较上代 +150 倍，首发 soundcore Liberty 5 Pro；以及端侧 AI 趋势、个性化听力健康趋势、多设备无缝连接趋势），我们选择 **智能影音 / soundcore 耳机**作为演示品类：它数据最丰富、竞

品 AI 策略最清晰、技术原生故事最强。系统本身品类无关，切换到充电或 eufy 仅需更换数据配置与 aspect 词典。

第二部分　AI 原生产品定义方法论

5 核心理念：把”判断”建立在可验证的证据上

产品经理的天花板就是产品判断的天花板。AI 原生方法的本质，是把”判断”从**个人经验**升级为**系统能力**——让每一条洞察都可追溯到真实证据、每一个假设都可被合成用户验证、每一项机会都可被量化排序。

为回应安克”大模型不确定性 vs 企业高确定性”的矛盾，本系统采用**确定性核心 + LLM 增强**的设计：能用确定性算法（词典 / 统计 / 规则）得到的结构化数值，绝不交给 LLM 猜；LLM 只负责**叙述**、**归纳**、**角色扮演**、**批判**。并以 **逐字引用校验**把幻觉关进可审计的笼子。

6 标准作业流程（SOP）

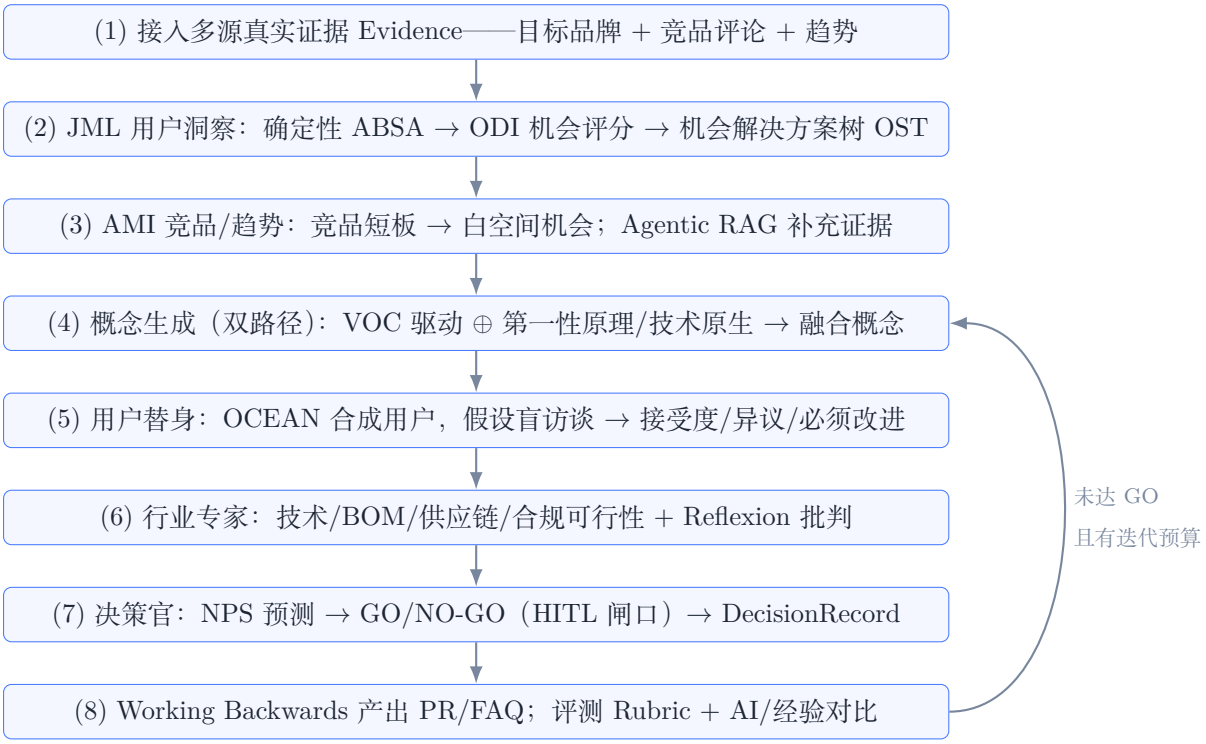

图 2: AI 原生产品定义 SOP（八步闭环，决策未通过则回到概念环节迭代）

7 映射安克三大平台

8 方法论锚点（均为业界成熟方法，非自造）

安克平台	传统形态	本系统的 AI 原生版
JML	人工访谈、问卷、Shulex VOC	真实评论 ABSA + ODI 机会评分 + 机会解决方案树
AMI	分析师做竞品/趋势报告	竞品 ABSA → 短板/白空间 + 趋势检索（Agentic RAG）
BEES	用户测试、NPS 调研	合成用户假设盲访谈 + 体验演化 + NPS 预测
第一性原理	资深判断	技术原生路径（端侧 AI / Thus 芯片能力正推）

表 3: 把安克现有方法论平台逐一 AI 原生化

- **Working Backwards**（Amazon）：先写” 上市新闻稿 + 内外部 FAQ”，写不动即说明概念不成立。
- **Opportunity Solution Tree**（Teresa Torres）：结果 → 机会 → 方案 → 实验，保证从洞察到方案可追溯。
- **ODI**（Anthony Ulwick）：机会分 = 重要性 + max(重要性 − 满意度, 0)，量化” 未被满足”。
- **VOC Impact**：Impact = Reach × (Severity + Value + Strategic)。
- **合成用户**：OCEAN 人格 + 真实评论 grounding + **假设盲**反谄媚（参考 SCOPE / Grounded Simulation / Synthetic Users）。

9 双路径：VOC 驱动 + 技术原生

工作流同时支持两条概念生成路径，并在产品经理节点融合，对应安克” 双轮驱动”：

- **路径一（VOC 驱动）**：从机会分最高的真实痛点反推方案——确定性、低风险、可解释。
- **路径二（第一性原理 / 技术原生）**：从能力跃迁（端侧 AI、Thus 芯片）正推过去做不到的新体验——前瞻、差异化。

这正是对命题中”AI 创新很大程度是技术原生驱动” 这一论断的工程化回应：好的产品定义既要” 听见用户”，也要” 看见技术拐点”。

第三部分 系统架构

10 四层架构

后端遵循 **Starter / Application / Infrastructure / Common** 四层，依赖方向单向向下，由 `scripts/pre_pr_check.py` 在提交前强制校验。

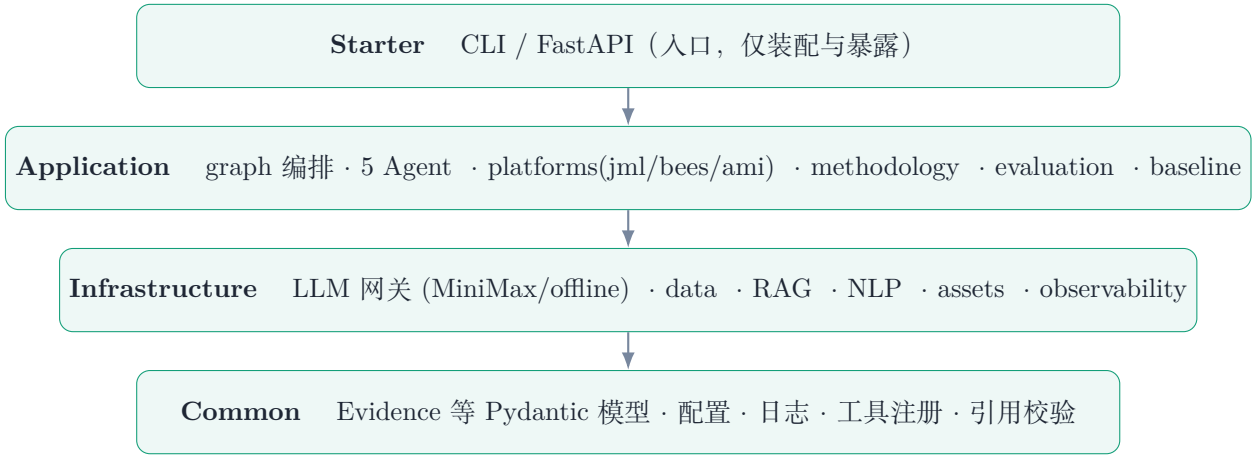

图 3: 四层架构：依赖只能 starter → application → infrastructure → common

11 StateGraph 编排

工作流以一个自研的、语义对齐 LangGraph 的最小状态图引擎编排（节点 / 条件边 / checkpoint / interrupt_before），既保证依赖极轻、离线可复现，又借鉴 Harness 工程的”错误即信息”（节点异常被记录后按策略路由而非崩溃）。

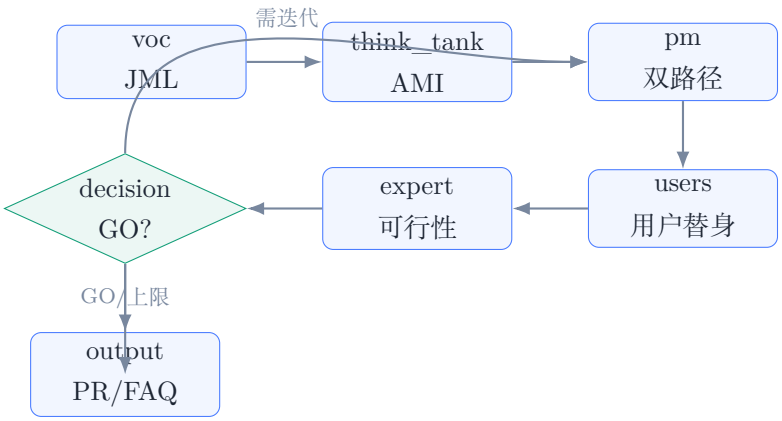

图 4: 编排状态图：决策闸前设 HITL interrupt_before；未达 GO 回到 pm 迭代

12 模块对照

13 全程治理

- **可溯源**：每条洞察/结论携带 `evidence_ids`；生成后由确定性脚本逐字校验引用命中，失败则重生成或标注为”未验证假设”。
- **防幻觉**：Agentic RAG 检索-打分-重写环（上限 3 次）；数值由确定性核心产出。

能力	模块	说明
确定性 ABSA	`infrastructure/nlp/`	aspect 词典 + 情感词典 + 否定翻转，子句级统计
机会量化	`methodology/odi.py`	ODI 机会分 + Impact 评分
混合检索	`infrastructure/rag/`	BM25 + TF-IDF，RRF 融合，检索-重写迭代
LLM 网关	`infrastructure/llm/`	MiniMax M3 / offline 双 provider，自动降级
引用校验	`common/citations.py`	逐字命中 + 跨语言 aspect 桥接
编排引擎	`application/graph/`	StateGraph + checkpoint + interrupt
评测对比	`application/evaluation/`	Rubric + AI/经验量化对比

表 4: 核心模块对照（完整源码见附录）

- **HITL 决策闸**：`interrupt_before` 在 GO/NO-GO 前暂停，支持人工确认后 `checkpoint` 恢复。
- **审计**：每次决策写入 `DecisionRecord`（含理由、置信度、条件、引用、时间戳）。
- **可观测**：每个节点的耗时/结果/引用命中写入 `runs/*.jsonl`，前端实时可视化。

第四部分　五个 AI 角色详解

14 超级智囊（AI 原生 AMI）

职责：行业研究、竞品分析、趋势洞察。**输入**：竞品评论集 + 趋势。**输出**：`MarketIntel`（竞品优势/短板、白空间机会、趋势信号）。

对每个竞品（Bose/Sony/Samsung/Apple）做确定性 ABSA：负面率高且提及广的 aspect 记为**短板**（即差异化白空间），满意度高的记为**优势**；多个竞品共同的短板被聚合为强度更高的”白空间机会”。随后用 **Agentic RAG** 以英文检索词（由中文 aspect 经词典映射）为每个白空间机会补充真实评论证据，并累计检索迭代数供评测统计。

15 产品经理（编排，双路径）

职责：需求拆解、概念生成、迭代收敛。首轮生成两条路径的候选概念（VOC 驱动 / 技术原生）并融合：融合概念既覆盖机会分最高的真实痛点，又叠加端侧 AI 技术使能与竞品白空间。后续轮次吸收用户替身的 `must_fixes` 与行业专家的风险，对概念做修订。每个差异点都携带 `evidence_ids`。

16 用户替身（合成用户面板）

职责：用户访谈、痛点验证。把真实评论按”主诉 aspect”聚成细分人群，构建 **OCEAN** 大五人格画像的合成用户（人格由确定性哈希生成、可复现）。访谈采用**假设盲**原则：只向 persona 暴露其自身画像与概念要点，**不暴露研究目标**，从架构上杜绝谄媚式确认（参考 Grounded Simulation 的反谄媚控制）。输出每个细分人群的判定（would_buy / maybe / would_not_buy）、接受度、异议与必须改进项，并回链其来源评论。

17 行业专家（可行性 + Reflexion）

职责：技术 / 供应链 / BOM / 合规可行性评估，并以 **Reflexion** 方式批判概念、产出风险项与缓解方案。例如：端侧 AI 芯片 + 多麦克风抬高 BOM（建议首发只上最高价值能力、其余 OTA）；实时翻译重负载难全端侧（端云协同）；听力个性化在欧美受医疗法规约束（分区合规）。总体可行性以红/黄/绿三档输出，作为决策官输入。

18 决策官（NPS + GO/NO-GO）

职责：NPS 预测、GO/NO-GO 决策、可审计记录。NPS 预测以可解释方式合成：目标品牌当前满意度基线 + 命中高机会痛点的覆盖加成 + 合成用户接受度修正。决策规则综合 NPS、可行性、平均接受度三者，输出 `GO` / `CONDITIONAL_GO` / `NO_GO` 与条件清单，并写入 `DecisionRecord`。当开启 HITL 时，在该节点前暂停等待人工确认。

角色	命题对应	回答的问题
超级智囊	AI 当超级智囊	机会在哪？竞品弱在哪？
产品经理	（编排）	做什么？怎么收敛？
用户替身	AI 当用户替身	用户认不认？哪里必须改？
行业专家	AI 当行业专家	做不做得出来？风险几何？
决策官	（治理）	要不要做？凭什么？

表 5: 五角色与命题三角色的对应

第五部分 数据与用户洞察（VOC）

19 多源真实数据链路

对标安克内部”Shulex VOC + 爬取 Amazon 评论”的真实做法，系统的数据底座是**多源、真实、可离线复现**的：

- **核心(JML/BEES)**：Amazon Reviews 2023（McAuley Lab, UCSD，HuggingFace）的 `raw_review_Electronic` 与 `raw_meta_Electronics`，按 `store` 字段筛 soundcore 评论做用户洞察；按时间切片做体验演化。
- **竞品（AMI）**：同一数据集按 `store` 筛 Bose / Sony / Samsung / Apple，得到**真实竞品评论**，无需爬虫、可离线。
- **趋势/外部**：可选 Google Trends（pytrends）与 Web 搜索连接器；缺失时回退到带出处的离线结构化趋势信号。
- **离线样本**：`data/make_sample.py` 可复现生成”合成但贴近真实”的样本评论，使系统零网络即可端到端运行（本报告数据即来自该样本）。

所有来源统一归一为 `Evidence`{source_id, source_type, brand, rating, text, date, url}，**Agent 只能引用 Evidence**，从机制上杜绝凭空生成。

20 确定性 ABSA 与 ODI 机会评分

ABSA 在子句级用 aspect 词典匹配 aspect，用情感词典 + 否定翻转 + 评分兜底判定极性，按评论去重统计提及量（Reach）与负面率（Severity），并给出代表性引用。机会量化采用 Ulwick ODI 与 VOC Impact：

$$\text{重要性} = f(\text{Reach}), \quad \text{满意度} = 10 \times \text{正面率}, \quad \text{机会分} = \text{重要性} + \max(\text{重要性} - \text{满意度}, 0)$$

$$\text{Impact} = \text{Reach} \times (\text{Severity} + \text{Value} + \text{Strategic})$$

其中 Strategic 对与安克技术原生强相关的 aspect（降噪、通话、音质）赋更高权重。

21 用户洞察结果（样本数据真实运行）

关键发现：经验直觉通常押”音质 / 续航 / 性价比”，但数据显示机会分最高的痛点其实是**连接稳定性/多点**、**App 体验**等——这正是”拍脑袋”的系统性盲区。每个机会均挂有代表性真实评论 `evidence_id`，并据此生成机会解决方案树（结果 → 机会 → 候选方案 → 实验），完整结构见在线 Demo 与运行报告。

22 BEES 体验演化

按评论年份切片观察重点 aspect 的负面率变化，用于识别”某能力是在变好还是退步”，为 NPS 预测与迭代提供时间维度的体验信号。

第六部分 评测与对比实验

23 评测 Rubric

aspect 维度	提及覆盖	负面率	满意度	机会分	Impact
连接稳定性/多点	28%	64%	2.73	**8.97**	0.64
佩戴舒适度	42%	18%	7.65	8.25	0.71
App 体验	25%	67%	3.00	8.00	0.57
音质	35%	38%	6.19	7.61	0.76
耐用性	16%	79%	2.10	6.34	0.36
通话质量	23%	54%	4.64	5.90	0.59
价格/价值	20%	62%	3.75	5.85	0.41
续航	26%	32%	6.45	5.62	0.50
降噪 ANC	21%	24%	7.60	4.92	0.47

表 6: soundcore 用户洞察（120 条评论样本）。机会分最高的是**连接稳定性/多点**、**佩戴舒适度**、**App 体验**

对标 VitaBench 风格的可解释成功率定义，系统内置评测 harness，并设质量门槛（不达标触发重试/降级或人工复核）：

指标	含义	结果
groundedness	带有效引用的论断比例	1.00
faithfulness	引用确实支撑论断的比例	通过门槛
citation_hit_rate	引用 evidence_id 逐字命中率	1.00
opportunity_coverage	提案覆盖的 Top 机会比例	0.60
persona_fidelity	合成用户来自真实评论 grounding 比例	1.00
explainability	是否产出可解释 DecisionRecord	1.00
overall	加权综合	**0.875**

表 7: 评测 Rubric（样本数据真实运行）

24 对比实验：AI 驱动 vs 经验驱动

命题明确要求” 比比看本质不同”。我们将其设计为一个**受控实验**，而非口头论证：

- **同一输入**：相同品类（soundcore TWS 耳机）、相同 brief。
- **A 组（经验驱动）**：模拟传统 PM 一次性” 拍脑袋”，凭行业常识押注维度，不接数据管线、不做用户验证。
- **B 组（AI 原生）**：本系统全链路。
- **公平性**：A 组产出的概念也用**同一份真实数据**评估其” 实际会达到的 NPS”，避免只比口号。

维度	A 经验驱动	B AI 原生	Δ
机会覆盖率	20%	**60%**	+0.40
命中真实痛点率	33%	**100%**	+0.67
证据引用数	0	**56**	+56
已验证假设数（合成用户）	0	**4**	+4
合成用户数	0	**4**	+4
可行性风险识别	0	**3**	+3
预测 NPS	−29	**+1**	+30

表 8: AI 驱动 vs 经验驱动（同一 brief，真实数据评估）

本质不同（五点）：(1) 从主观到**可溯源**（每条结论带 evidence_id 并逐字校验）；(2) 从少量到**全量**（统计真实评论全景，不被最响的声音带偏）；(3) 从单次到**可迭代**（工作流闭环，未达 GO 自动回炉）；(4) 从拍脑袋到**可量化**（ODI/Impact/NPS）；(5) 从经验到**系统能力**（方法论可沉淀为 SOP，复制给任意品类与任意人）。

25 在线 Demo 截图（数据化）

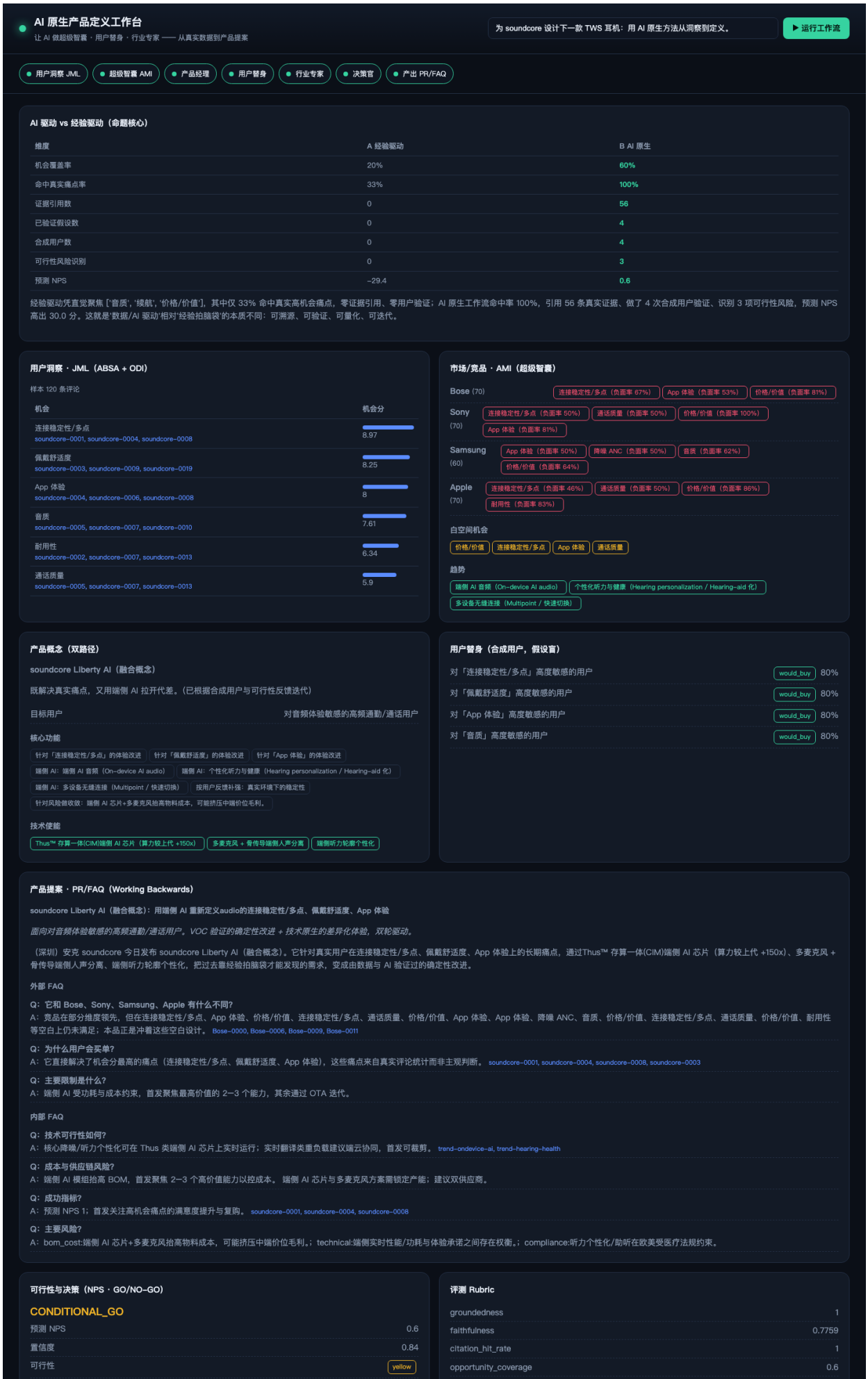
AI 原生产品定义工作台
让 AI 做超级智囊 · 用户替身 · 行业专家 —— 从真实数据到产品提案
为 soundcore 设计下一款 TWS 耳机：用 AI 原生方法从洞察到定义。
▶ 运行工作流
用户洞察 JML
超级智囊 AMI
产品经理
用户替身
行业专家
决策官
产出 PR/FAQ
AI 驱动 vs 经验驱动（命题核心）
维度 A 经验驱动 B AI 原生
机会覆盖率 20% 60%
命中真实痛点率 33% 100%
证据引用数 0 56
已验证假设数 0 4
合成用户数 0 4
可行性风险识别 0 3
预测 NPS -29.4 0.6
经验驱动凭直觉聚焦 ['音质', '续航', '价格/价值']，其中仅 33% 命中真实高机会痛点，零证据引用、零用户验证；AI 原生工作流命中率 100%，引用 56 条真实证据、做了 4 次合成用户验证、识别 3 项可行性风险，预测 NPS 高出 30.0 分。这就是'数据/AI 驱动'相对'经验拍脑袋'的本质不同：可溯源、可验证、可量化、可迭代。
用户洞察 · JML（ABSA + ODI）
样本 120 条评论
机会 机会分
连接稳定性/多点
soundcore-0001, soundcore-0004, soundcore-0008
8.97
佩戴舒适度
soundcore-0003, soundcore-0009, soundcore-0019
8.25
App 体验
soundcore-0004, soundcore-0006, soundcore-0008
8
音质
soundcore-0005, soundcore-0007, soundcore-0010
7.61
耐用性
soundcore-0002, soundcore-0007, soundcore-0013
6.34
通话质量
soundcore-0005, soundcore-0007, soundcore-0013
5.9
市场/竞品 · AMI（超级智囊）
Bose (70)
连接稳定性/多点（负面率 67%）
App 体验（负面率 53%）
价格/价值（负面率 81%）
Sony (70)
连接稳定性/多点（负面率 50%）
通话质量（负面率 50%）
价格/价值（负面率 100%）
App 体验（负面率 81%）
Samsung (60)
App 体验（负面率 50%）
降噪 ANC（负面率 50%）
音质（负面率 62%）
价格/价值（负面率 64%）
Apple (70)
连接稳定性/多点（负面率 46%）
通话质量（负面率 50%）
价格/价值（负面率 86%）
耐用性（负面率 83%）
白空间机会
价格/价值
连接稳定性/多点
App 体验
通话质量
趋势
端侧 AI 音频（On-device AI audio）
个性化听力与健康（Hearing personalization / Hearing-aid 化）
多设备无缝连接（Multipoint / 快速切换）
产品概念（双路径）
soundcore Liberty AI（融合概念）
既解决真实痛点，又用端侧 AI 拉开代差。（已根据合成用户与可行性反馈迭代）
目标用户
对音频体验敏感的高频通勤/通话用户
核心功能
针对「连接稳定性/多点」的体验改进
针对「佩戴舒适度」的体验改进
针对「App 体验」的体验改进
端侧 AI：端侧 AI 音频（On-device AI audio）
端侧 AI：个性化听力与健康（Hearing personalization / Hearing-aid 化）
端侧 AI：多设备无缝连接（Multipoint / 快速切换）
按用户反馈补强：真实环境下的稳定性
针对风险做收敛：端侧 AI 芯片+多麦克风抬高物料成本，可能挤压中端价位毛利。
技术使能
Thus™ 存算一体(CIM)端侧 AI 芯片（算力较上代 +150x）
多麦克风 + 骨传导端侧人声分离
端侧听力轮廓个性化
用户替身（合成用户，假设盲）
对「连接稳定性/多点」高度敏感的用户 would_buy 80%
对「佩戴舒适度」高度敏感的用户 would_buy 80%
对「App 体验」高度敏感的用户 would_buy 80%
对「音质」高度敏感的用户 would_buy 80%
产品提案 · PR/FAQ（Working Backwards）
soundcore Liberty AI（融合概念）：用端侧 AI 重新定义audio的连接稳定性/多点、佩戴舒适度、App 体验
面向对音频体验敏感的高频通勤/通话用户。VOC 验证的确定性改进 + 技术原生的差异化体验，双轮驱动。
（深圳）安克 soundcore 今日发布 soundcore Liberty AI（融合概念）。它针对真实用户在连接稳定性/多点、佩戴舒适度、App 体验上的长期痛点，通过Thus™ 存算一体(CIM)端侧 AI 芯片（算力较上代 +150x）、多麦克风 + 骨传导端侧人声分离、端侧听力轮廓个性化，把过去靠经验拍脑袋才能发现的需求，变成由数据与 AI 验证过的确定性改进。
外部 FAQ
Q：它和 Bose、Sony、Samsung、Apple 有什么不同?
A：竞品在部分维度领先，但在连接稳定性/多点、App 体验、价格/价值、连接稳定性/多点、通话质量、价格/价值、App 体验、App 体验、降噪 ANC、音质、价格/价值、连接稳定性/多点、通话质量、价格/价值、耐用性等空白上仍未满足；本品正是冲着这些空白设计。 Bose-0000, Bose-0006, Bose-0009, Bose-0011
Q：为什么用户会买单?
A：它直接解决了机会分最高的痛点（连接稳定性/多点、佩戴舒适度、App 体验），这些痛点来自真实评论统计而非主观判断。 soundcore-0001, soundcore-0004, soundcore-0008, soundcore-0003
Q：主要限制是什么?
A：端侧 AI 受功耗与成本约束，首发聚焦最高价值的 2–3 个能力，其余通过 OTA 迭代。
内部 FAQ
Q：技术可行性如何?
A：核心降噪/听力个性化可在 Thus 类端侧 AI 芯片上实时运行；实时翻译类重负载建议端云协同，首发可裁剪。 trend-ondevice-ai, trend-hearing-health
Q：成本与供应链风险?
A：端侧 AI 模组抬高 BOM，首发聚焦 2–3 个高价值能力以控成本。 端侧 AI 芯片与多麦克风方案需锁定产能；建议双供应商。
Q：成功指标?
A：预测 NPS 1；首发关注高机会痛点的满意度提升与复购。 soundcore-0001, soundcore-0004, soundcore-0008
Q：主要风险?
A：bom_cost:端侧 AI 芯片+多麦克风抬高物料成本，可能挤压中端价位毛利。; technical:端侧实时性能/功耗与体验承诺之间存在权衡。; compliance:听力个性化/助听在欧美受医疗法规约束。
可行性与决策（NPS · GO/NO-GO）
CONDITIONAL_GO
预测 NPS 0.6
置信度 0.84
可行性 yellow
评测 Rubric
groundedness 1
faithfulness 0.7759
citation_hit_rate 1
opportunity_coverage 0.6

图 5: 可视化工作台：每个 Agent 节点实时点亮，并呈现 VOC 机会、竞品白空间、概念、合成用户访谈、PR/FAQ、决策与评测

第七部分　工程规范（对标大厂 AIGC）

26 为什么规范是”企业级 vs 玩具”的分水岭

我们对标美团技术团队《用 Agent 评测思路管理 AI Coding——31 万行代码 AI 重构的实践》的核心理念：当 90% 以上代码由 AI 生成，**没有统一规范，系统会加速腐化**。规范在 AI Coding 时代已从”协作建议”升级为”约束 AI 产出、阻止系统继续长新债的基础设施”。其方法论是**人人对齐 → 人机对齐**：先让团队形成统一共识，再把共识固化为 AI 编码时强制加载的约束。

27 本项目落地的规范

- **AI Rule（always-on）**：`.cursor/rules/` 强制四层架构、Pydantic 数据模型、loguru 日志、完整类型注解、工具 `@tool` 读写分离（WRITE 须两步确认）、**所有结论强制带 source_id 引用**。
- **Skill（渐进式加载）**：`skills/` 沉淀 VOC 分析、Working Backwards 的可执行 SOP。
- **Pre-PR 自查**：`scripts/pre_pr_check.py` 在提交前做确定性规范校验（分层、日志、类型、异常），过滤 AI 编码常见低级问题。
- **跨厂商模型对抗 CR**：`scripts/cross_model_review.py` 用 MiniMax M3 按本仓规范审查代码，对应”高阶模型审低阶 + 不同厂商互审”。
- **确定性优先**：能用词典/统计/规则得到的结果不交给 LLM，回应”大模型不确定性 vs 企业高确定性”。
- **异常与降级**：每个外部调用都有 fallback（LLM 不可用 → offline；检索为空 → 重写重试，上限 3 次），不静默失败。

28 代码风格（与美团一致）

```python
from pydantic import BaseModel, Field   # 必用 Pydantic 数据模型
from loguru import logger               # 必用 loguru，不用 print/logging
from typing import List, Optional       # 完整类型注解

@tool(ToolType.READ)                     # 工具区分读/写；WRITE 须两步确认
def search_evidence(query: str) -> List[Evidence]:
    """检索证据（只读，可并发）。"""
    logger.bind(node="rag").info(f"检索: {query}")
    ...
```

本项目所有后端模块均通过 `pre_pr_check.py` 校验、`compileall` 字节编译，并附带冒烟测试 `backend/tests/test_smoke.py`（验证端到端跑通且 groundedness/引用命中率达标、AI 组优于经验组）。

第八部分　部署与运维

29 多项目隔离原则

目标服务器（阿里云 ECS，华南-深圳，Ubuntu 22.04，2 vCPU / 2 GiB）上已运行项目一”小念（xiaonian）”。本项目作为 **项目二接入，严格遵循” 每个项目有且只有：一个代码目录 + 一个 venv + 一个 systemd 服务 + 一个 nginx location + 一个静态目录**” 的隔离原则，互不影响、可独立停/改/回滚。

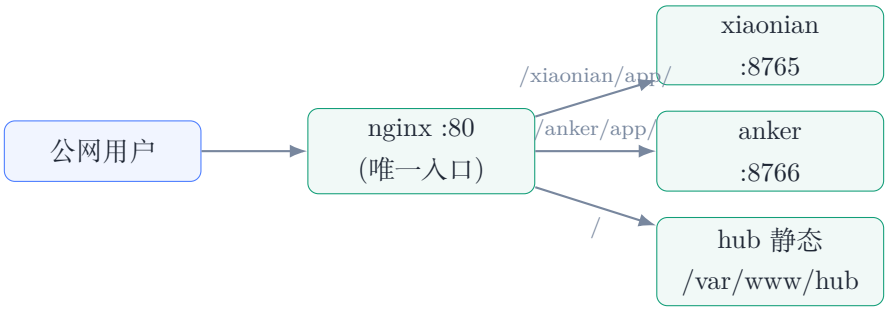

图 6: 部署拓扑：nginx 单入口按子路径反代到各自 127.0.0.1 端口；后端不直接暴露公网

30 公网路由表

公网 URL	类型	后端
http://47.119.112.225/	静态 hub	/var/www/hub
http://47.119.112.225/xiaonian/app/	FastAPI 代理	127.0.0.1:8765（项目一，未改动）
http://47.119.112.225/anker/	静态宣传页	/var/www/anker
http://47.119.112.225/anker/app/	FastAPI 代理	127.0.0.1:8766（本项目）

表 9: 部署后公网路由（小念路由完全不受影响）

31 部署流程（一键脚本）

`scripts/deploy.py`（凭据从环境变量读取，绝不入库）自动完成：本地打包并经 SFTP 上传代码（绕过服务器对 GitHub 的访问不稳定）→ 创建 venv 并经清华镜像安装依赖 → 生成样本数

据 → 写 .env（不入库）→ 创建 anker.service systemd 守护 → 落地宣传页 → 幂等插入 nginx 路由 → 健康检查。子路径反代要点：应用前端统一使用**相对路径**，nginx proxy_pass 去前缀后应用即”根路径”工作，SSE 流式输出需 proxy_buffering off。

32 健康检查（部署后实测）

```
local-app      200      # 127.0.0.1:8766/api/health
hub            200      # /
xiaonian       200      # /xiaonian/  （项目一未受影响）
anker-landing 200       # /anker/
anker-demo     200      # /anker/app/
/anker/app/api/health  -> {"status":"ok","provider":"offline"}
公网 POST /anker/app/api/run -> decision=CONDITIONAL_GO, overall=0.875,
                                命中真实痛点 A/B = 33% / 100%
```

图 7: 项目导航 hub：小念与 Anker 两个作品均”运行中”，互不冲突

33 回滚

停止并禁用 anker.service、注释 nginx 中 anker 的 location、nginx -t && systemctl reload nginx 即可；小念的服务与路由完全不受影响。

第九部分　结果展示与链接

34 宣传页与在线 Demo


Anker AI 原生产品定义系统
能做什么 方法论 本质不同 ▶ 在线 Demo
2026 AI 先锋未来人才大赛 · 安克创新命题 · 华南赛区
让 AI 做你的超级智囊 · 用户替身 · 行业专家
从真实数据，到产品定义
这不是一份 PPT，而是一套真实可运行、数据可溯源、可评测的多智能体系统：把安克 JML / BEES / AMI 三大产品方法论平台做成 AI 原生版，端到端从真实用户评论跑出一份产品提案（PR/FAQ），并用一个可量化对比实验回答——AI 驱动相较"经验拍脑袋"到底本质不同在哪。
▶ 打开在线 Demo
GitHub 源码
作者主页
100%
AI 命中真实痛点率
(经验驱动仅 33%)
+56
可溯源证据引用
(经验驱动为 0)
+30
预测 NPS 提升
(同一 brief 对比)
5
AI 角色协同
StateGraph 编排
它做了什么
输入一句 brief，系统自动跑完 8 步，产出一份完整、可溯源的产品提案。
① 用户洞察 · JML
对真实评论做确定性 ABSA → ODI 机会评分 → 机会解决方案树，找出真正未被满足的痛点。
② 市场/竞品 · AMI
逐家拆解竞品（Bose/Sony/Samsung/Apple）短板，识别"白空间机会"，叠加行业趋势。
③ 产品经理 · 双路径
VOC 驱动（从真实痛点反推）+ 第一性原理/技术原生（从端侧 AI / Thus 芯片正推）。
④ 用户替身 · 合成用户
按真实评论聚类成 OCEAN 画像人群，假设盲访谈，反谄媚地挑战方案。
⑤ 行业专家
技术 / BOM / 供应链 / 合规可行性 + Reflexion 批判，给出风险与缓解。
⑥ 决策官
NPS 预测 + GO/NO-GO + 可审计 DecisionRecord（支持 HITL 人工闸口）。
真实评论 → JML 洞察 + AMI 竞品 → 概念(双路径) → 用户替身 → 行业专家 → 决策官 → PR/FAQ 提案
方法论（均为业界成熟方法，非自造）
Amazon Working Backwards（PR/FAQ）· Teresa Torres 机会解决方案树 · Ulwick ODI 机会评分 · OCEAN 合成用户 + 假设盲反谄媚。确定性核心产出数值，LLM 负责叙述/角色扮演/批判，强制逐字引用校验。
LangGraph 风格 StateGraph 编排
Agentic RAG（BM25+TF-IDF 混合检索）
逐字引用校验 · 反幻觉
四层架构 + AI Rule + Pre-PR + 跨模型对抗 CR
MiniMax M3 可选增强
离线确定性模式 · 可复现
本质不同：AI 驱动 vs 经验驱动
同一品类、同一 brief，A 组（单 LLM 拍脑袋）vs B 组（本系统全链路），在真实数据上量化对比。
维度 A 经验驱动 B AI 原生
命中真实痛点率 33% 100%
机会覆盖率 20% 60%
证据引用数 0 56
已验证假设数（合成用户） 0 4
可行性风险识别 0 3
预测 NPS -29 +1 (Δ +30)
经验驱动凭直觉押"音质/续航/性价比"，但真实数据显示机会分最高的痛点其实是 连接稳定性/多点、佩戴舒适度、App 体验——这正是"拍脑袋"的系统性盲区。
在线 Demo GitHub 个人主页
参赛者：戴尚好 · 中国科学技术大学 · bcefghj@163.com
© 2026 AI 先锋未来人才大赛 · 安克创新命题参赛作品。本页数据来自系统在样本数据上的真实运行。

图 8: 项目宣传页（http://47.119.112.225/anker/）

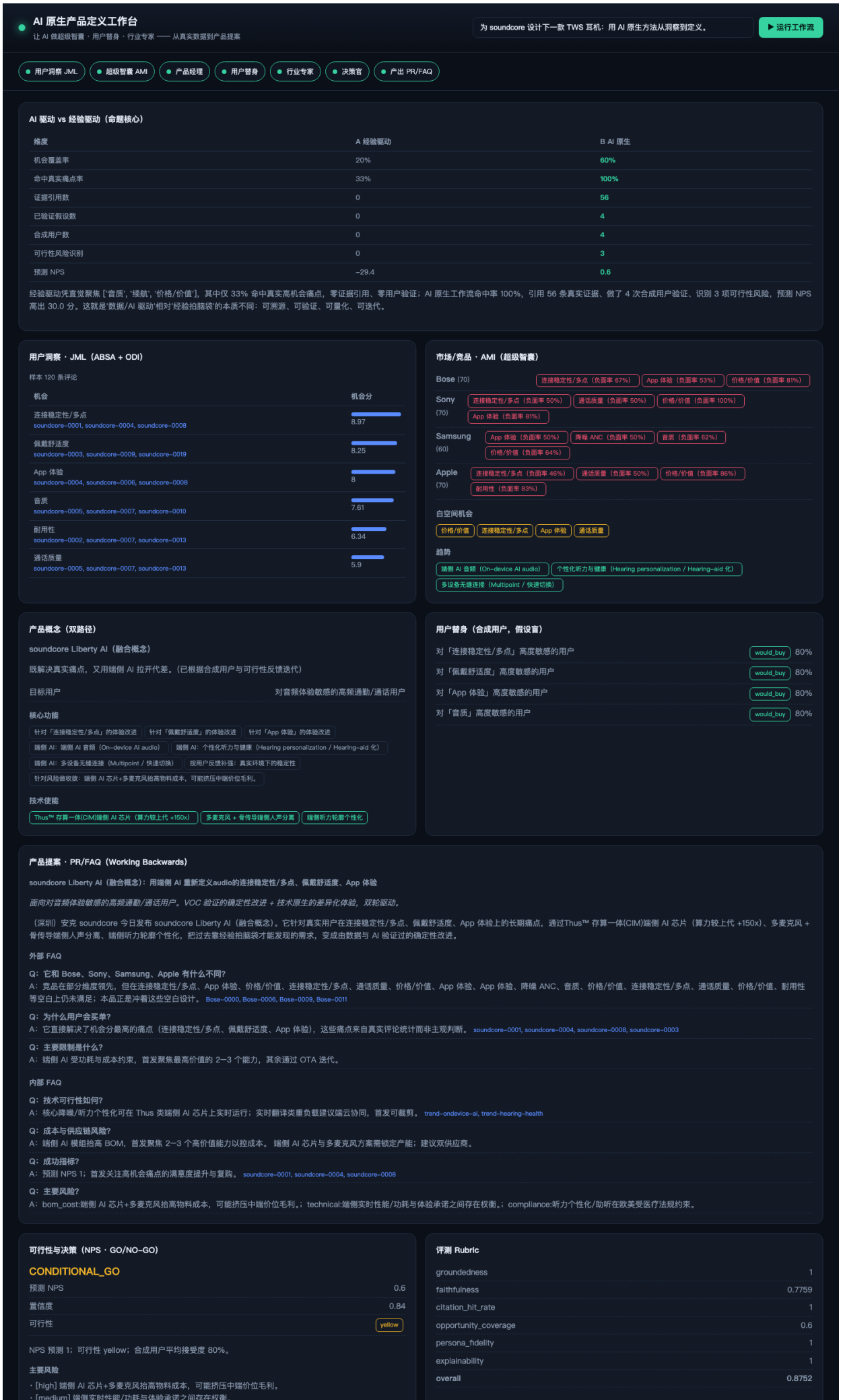
AI 原生产品定义工作台
让 AI 做超级智囊 · 用户替身 · 行业专家 —— 从真实数据到产品提案
为 soundcore 设计下一款 TWS 耳机：用 AI 原生方法从洞察到定义。
▶ 运行工作流
用户洞察 JML
超级智囊 AMI
产品经理
用户替身
行业专家
决策官
产出 PR/FAQ
AI 驱动 vs 经验驱动（命题核心）
维度 A 经验驱动 B AI 原生
机会覆盖率 20% 60%
命中真实痛点率 33% 100%
证据引用数 0 56
已验证假设数 0 4
合成用户数 0 4
可行性风险识别 0 3
预测 NPS -29.4 0.6
经验驱动凭直觉聚焦 ['音质', '续航', '价格/价值']，其中仅 33% 命中真实高机会痛点，零证据引用、零用户验证；AI 原生工作流命中率 100%，引用 56 条真实证据、做了 4 次合成用户验证、识别 3 项可行性风险，预测 NPS 高出 30.0 分。这就是'数据/AI 驱动'相对'经验拍脑袋'的本质不同：可溯源、可验证、可量化、可迭代。
用户洞察 · JML（ABSA + ODI）
样本 120 条评论
机会 机会分
连接稳定性/多点
soundcore-0001, soundcore-0004, soundcore-0008
8.97
佩戴舒适度
soundcore-0003, soundcore-0009, soundcore-0019
8.25
App 体验
soundcore-0004, soundcore-0006, soundcore-0008
8
音质
soundcore-0005, soundcore-0007, soundcore-0010
7.61
耐用性
soundcore-0002, soundcore-0007, soundcore-0013
6.34
通话质量
soundcore-0005, soundcore-0007, soundcore-0013
5.9
市场/竞品 · AMI（超级智囊）
Bose (70)
连接稳定性/多点（负面率 67%）
App 体验（负面率 53%）
价格/价值（负面率 81%）
Sony (70)
连接稳定性/多点（负面率 50%）
通话质量（负面率 50%）
价格/价值（负面率 100%）
App 体验（负面率 81%）
Samsung (60)
App 体验（负面率 50%）
降噪 ANC（负面率 50%）
音质（负面率 62%）
价格/价值（负面率 64%）
Apple (70)
连接稳定性/多点（负面率 46%）
通话质量（负面率 50%）
价格/价值（负面率 86%）
耐用性（负面率 83%）
白空间机会
价格/价值
连接稳定性/多点
App 体验
通话质量
趋势
端侧 AI 音频（On-device AI audio）
个性化听力与健康（Hearing personalization / Hearing-aid 化）
多设备无缝连接（Multipoint / 快速切换）
产品概念（双路径）
soundcore Liberty AI（融合概念）
既解决真实痛点，又用端侧 AI 拉开代差。（已根据合成用户与可行性反馈迭代）
目标用户 对音频体验敏感的高频通勤/通话用户
核心功能
针对「连接稳定性/多点」的体验改进
针对「佩戴舒适度」的体验改进
针对「App 体验」的体验改进
端侧 AI：端侧 AI 音频（On-device AI audio）
端侧 AI：个性化听力与健康（Hearing personalization / Hearing-aid 化）
端侧 AI：多设备无缝连接（Multipoint / 快速切换）
按用户反馈补强：真实环境下的稳定性
针对风险做收敛：端侧 AI 芯片+多麦克风抬高物料成本，可能挤压中端价位毛利。
技术使能
Thus™ 存算一体(CIM)端侧 AI 芯片（算力较上代 +150x）
多麦克风 + 骨传导端侧人声分离
端侧听力轮廓个性化
用户替身（合成用户，假设盲）
对「连接稳定性/多点」高度敏感的用户 would_buy 80%
对「佩戴舒适度」高度敏感的用户 would_buy 80%
对「App 体验」高度敏感的用户 would_buy 80%
对「音质」高度敏感的用户 would_buy 80%
产品提案 · PR/FAQ（Working Backwards）
soundcore Liberty AI（融合概念）：用端侧 AI 重新定义audio的连接稳定性/多点、佩戴舒适度、App 体验
面向对音频体验敏感的高频通勤/通话用户。VOC 验证的确定性改进 + 技术原生的差异化体验，双轮驱动。
（深圳）安克 soundcore 今日发布 soundcore Liberty AI（融合概念）。它针对真实用户在连接稳定性/多点、佩戴舒适度、App 体验上的长期痛点，通过Thus™ 存算一体(CIM)端侧 AI 芯片（算力较上代 +150x）、多麦克风 + 骨传导端侧人声分离、端侧听力轮廓个性化，把过去靠经验拍脑袋才能发现的需求，变成由数据与 AI 验证过的确定性改进。
外部 FAQ
Q：它和 Bose、Sony、Samsung、Apple 有什么不同?
A：竞品在部分维度领先，但在连接稳定性/多点、App 体验、价格/价值、连接稳定性/多点、通话质量、价格/价值、App 体验、App 体验、降噪 ANC、音质、价格/价值、连接稳定性/多点、通话质量、价格/价值、耐用性等空白上仍未满足；本品正是冲着这些空白设计。 Bose-0000, Bose-0006, Bose-0009, Bose-0011
Q：为什么用户会买单?
A：它直接解决了机会分最高的痛点（连接稳定性/多点、佩戴舒适度、App 体验），这些痛点来自真实评论统计而非主观判断。 soundcore-0001, soundcore-0004, soundcore-0008, soundcore-0003
Q：主要限制是什么?
A：端侧 AI 受功耗与成本约束，首发聚焦最高价值的 2–3 个能力，其余通过 OTA 迭代。
内部 FAQ
Q：技术可行性如何?
A：核心降噪/听力个性化可在 Thus 类端侧 AI 芯片上实时运行；实时翻译类重负载建议端云协同，首发可裁剪。 trend-ondevice-ai, trend-hearing-health
Q：成本与供应链风险?
A：端侧 AI 模组抬高 BOM，首发聚焦 2–3 个高价值能力以控成本。 端侧 AI 芯片与多麦克风方案需锁定产能；建议双供应商。
Q：成功指标?
A：预测 NPS 1；首发关注高机会痛点的满意度提升与复购。 soundcore-0001, soundcore-0004, soundcore-0008
Q：主要风险?
A：bom_cost:端侧 AI 芯片+多麦克风抬高物料成本，可能挤压中端价位毛利。；technical:端侧实时性能/功耗与体验承诺之间存在权衡。；compliance:听力个性化/助听在欧美受医疗法规约束。
可行性与决策（NPS · GO/NO-GO）
CONDITIONAL_GO
预测 NPS 0.6
置信度 0.84
可行性 yellow
NPS 预测 1；可行性 yellow；合成用户平均接受度 80%。
主要风险
· [high] 端侧 AI 芯片+多麦克风抬高物料成本，可能挤压中端价位毛利。
· [medium] 端侧实时性能/功耗与体验承诺之间存在权衡。
评测 Rubric
groundedness 1
faithfulness 0.7759
citation_hit_rate 1
opportunity_coverage 0.6
persona_fidelity 1
explainability 1
overall 0.8752

图 9: 线上 Demo 实跑结果（http://47.119.112.225/anker/app/）

35 链接汇总

- 在线 Demo：http://47.119.112.225/anker/app/
- 项目宣传页：http://47.119.112.225/anker/
- 项目导航 Hub：http://47.119.112.225/
- GitHub 源码：https://github.com/bcefghj/anker-ai-product-studio
- 个人主页：https://bcefghj.github.io

36 复现指南

```
# 1) 离线确定性模式（零网络、可复现）
pip install -r requirements.txt
python3 data/make_sample.py
PYTHONPATH=backend python3 -m anker_studio.starter.cli run
# 2) 可视化工作台
PYTHONPATH=backend python3 -m uvicorn anker_studio.starter.api:app --port 8000
# 3) 接入 MiniMax M3：在 .env 设 MINIMAX_API_KEY 并 ANKER_LLM_PROVIDER=minimax
# 4) 用真实 Amazon 评论：python3 -m anker_studio.infrastructure.data.amazon_reviews
```

37 局限与未来工作

- 合成用户与离线评分是” 发现 + 排序” 工具，不替代真实用户终验（与 NN/g 结论一致）；建议作为 AI 原生方法前置问题空间、把研究预算花在刀刃上。
- 线上 Demo 默认 offline 确定性模式以保证稳定与零成本；接入 MiniMax M3 可增强叙述质量与多模态。
- 可对接安克 AIME 平台，把本 SOP 沉淀为内部新 Agent 能力；并扩展到充电、eufy 等品类（仅需更换数据配置与 aspect 词典）。
- 可引入更强的 ABSA 模型（BERT 类）与真实 A/B 实验闭环，进一步提升机会识别与 NPS 预测精度。

38 结语

本作品用一套真实可运行、可溯源、可评测的系统回答了命题：AI 原生的产品定义，不是” 让 AI 帮你更快拍脑袋”，而是把产品判断从个人经验升级为可复制的系统能力——这正是安克” 用 AI 重新定义怎么想出一个好产品” 的方向。

第十部分 附录

附录说明

以下为本项目**全部核心源码**的逐文件清单（真实代码，非节选），按四层架构与逻辑顺序组织，便于评委审阅工程实现。代码总量约 4,500 行（后端 Python 约 3,600 行 + 前端/脚本/配置）。每个文件标注其相对路径与行数。

阅读建议：

- 先看 `common/models.py`（贯穿全系统的 Pydantic 领域模型）与 `common/citations.py`（反幻觉引用校验）。
- 再看 `infrastructure/nlp/absa.py`(确定性 ABSA 核心)与 `application/methodology/odi.py`(机会量化)。
- 然后看五个 Agent 与 `application/graph/`（StateGraph 编排引擎）。
- 最后看 `application/evaluation/`(Rubric 与 AI/经验对比)与部署脚本 `scripts/deploy.py`。

A Common 层（领域模型 / 配置 / 工具 / 引用校验）

backend/anker_studio/common/models.py (299 行)

```python
"""领域数据模型（Common 层）。

所有跨层数据均为 Pydantic v2 模型，禁止裸 dict 在层间传递。
模型贯穿：Evidence(可溯源底座) → VOC 洞察 → 竞品/趋势 → 概念 → 用户替身 →
可行性 → NPS → 决策 → 提案 → 评测/对比。
"""
from __future__ import annotations

from datetime import datetime
from enum import Enum
from typing import Dict, List, Optional

from pydantic import BaseModel, Field


#                 可溯源底座
class SourceType(str, Enum):
    REVIEW = "review"          # 用户评论（JML/BEES）
    COMPETITOR = "competitor"  # 竞品评论（AMI）
    TREND = "trend"            # 趋势信号
    WEB = "web"                # 网页/报告


class Evidence(BaseModel):
    """一切论断的可溯源单元。Agent 只能引用 Evidence。"""

    source_id: str
```

```
    source_type: SourceType = SourceType.REVIEW
    brand: str = "unknown"
    product: str = ""
    rating: Optional[float] = None
    text: str = ""
    date: Optional[str] = None
    url: Optional[str] = None
    helpful_votes: int = 0


#                VOC / JML / BEES
class AspectInsight(BaseModel):
    aspect: str
    mention_count: int = 0
    reach: float = Field(0.0, description="提及该 aspect 的评论占比 0-1")
    negative_rate: float = Field(0.0, description="负面提及占比 0-1")
    importance: float = Field(0.0, description="重要性 0-10（由 reach 推导)")
    satisfaction: float = Field(0.0, description="满意度 0-10（由正面率推导)")
    opportunity_score: float = Field(0.0, description="ODI: importance + max(importance-satisfaction,0)")
    impact_score: float = Field(0.0, description="Reach×(Severity+Value+Strategic)")
    representative_evidence_ids: List[str] = Field(default_factory=list)
    summary: str = ""


class Opportunity(BaseModel):
    id: str
    statement: str
    aspect: str = ""
    importance: float = 0.0
    satisfaction: float = 0.0
    reach: float = 0.0
    severity: float = 0.0
    opportunity_score: float = 0.0
    impact_score: float = 0.0
    origin: str = Field("voc", description="voc | competitor | trend")
    evidence_ids: List[str] = Field(default_factory=list)
    rationale: str = ""


class ExperiencePoint(BaseModel):
    """BEES 体验演化的一个时间/版本切片。"""

    aspect: str
    period: str
    mention_count: int
    negative_rate: float
    evidence_ids: List[str] = Field(default_factory=list)


#                机会解决方案树 (OST)
class Experiment(BaseModel):
    statement: str
    assumption: str = ""
    method: str = ""


class SolutionNode(BaseModel):
```

```python
    statement: str
    rationale: str = ""
    experiments: List[Experiment] = Field(default_factory=list)


class OpportunityNode(BaseModel):
    opportunity_id: str
    statement: str
    opportunity_score: float = 0.0
    solutions: List[SolutionNode] = Field(default_factory=list)


class OpportunitySolutionTree(BaseModel):
    outcome: str
    opportunities: List[OpportunityNode] = Field(default_factory=list)


class VocReport(BaseModel):
    category: str
    target_brand: str
    review_count: int = 0
    aspects: List[AspectInsight] = Field(default_factory=list)
    opportunities: List[Opportunity] = Field(default_factory=list)
    experience_trend: List[ExperiencePoint] = Field(default_factory=list)
    ost: Optional[OpportunitySolutionTree] = None


#                竞品 / 趋势 (AMI / 超级智囊)
class CompetitorFinding(BaseModel):
    brand: str
    review_count: int = 0
    strengths: List[str] = Field(default_factory=list)
    weaknesses: List[str] = Field(default_factory=list)
    white_space: List[str] = Field(default_factory=list, description="对手没做好 = 机会空白")
    evidence_ids: List[str] = Field(default_factory=list)


class TrendSignal(BaseModel):
    name: str
    direction: str = "up"
    summary: str = ""
    evidence_ids: List[str] = Field(default_factory=list)


class MarketIntel(BaseModel):
    competitors: List[CompetitorFinding] = Field(default_factory=list)
    trends: List[TrendSignal] = Field(default_factory=list)
    white_space_opportunities: List[Opportunity] = Field(default_factory=list)


#                用户替身 (合成用户)
class Persona(BaseModel):
    id: str
    name: str
    segment: str
    demographics: str = ""
    ocean: Dict[str, float] = Field(default_factory=dict, description="大五人格 0-1")
```

```
    behaviors: List[str] = Field(default_factory=list)
    pains: List[str] = Field(default_factory=list)
    derived_from_evidence_ids: List[str] = Field(default_factory=list)
    summary: str = ""


class InterviewTurn(BaseModel):
    question: str
    answer: str
    sentiment: str = "neutral"  # positive | neutral | negative


class PersonaInterview(BaseModel):
    persona_id: str
    persona_name: str = ""
    segment: str = ""
    transcript: List[InterviewTurn] = Field(default_factory=list)
    verdict: str = "neutral"            # would_buy | maybe | would_not_buy
    objections: List[str] = Field(default_factory=list)
    must_fixes: List[str] = Field(default_factory=list)
    acceptance: float = Field(0.0, description="接受度 0-1")


#                产品概念 / 提案
class Differentiator(BaseModel):
    statement: str
    evidence_ids: List[str] = Field(default_factory=list)


class ProductConcept(BaseModel):
    id: str
    name: str
    category: str
    path: str = Field("voc_driven", description="voc_driven | tech_native")
    one_liner: str = ""
    target_segment: str = ""
    value_proposition: str = ""
    key_features: List[str] = Field(default_factory=list)
    differentiators: List[Differentiator] = Field(default_factory=list)
    tech_enablers: List[str] = Field(default_factory=list)
    addressed_opportunity_ids: List[str] = Field(default_factory=list)


class RiskItem(BaseModel):
    area: str  # technical | supply_chain | bom_cost | compliance | market
    description: str
    severity: str = "medium"  # low | medium | high
    mitigation: str = ""


class FeasibilityAssessment(BaseModel):
    technical: str = ""
    supply_chain: str = ""
    bom_cost: str = ""
    compliance: str = ""
    overall: str = Field("yellow", description="green | yellow | red")
    risks: List[RiskItem] = Field(default_factory=list)
```

```
    evidence_ids: List[str] = Field(default_factory=list)


class NPSPrediction(BaseModel):
    score: float = Field(0.0, description="-100 .. 100")
    promoters: float = 0.0
    passives: float = 0.0
    detractors: float = 0.0
    rationale: str = ""
    evidence_ids: List[str] = Field(default_factory=list)


class FaqItem(BaseModel):
    question: str
    answer: str
    evidence_ids: List[str] = Field(default_factory=list)


class PRFAQ(BaseModel):
    headline: str = ""
    subheading: str = ""
    summary: str = ""
    customer_quote: str = ""
    maker_quote: str = ""
    call_to_action: str = ""
    external_faq: List[FaqItem] = Field(default_factory=list)
    internal_faq: List[FaqItem] = Field(default_factory=list)


class DecisionVerdict(str, Enum):
    GO = "GO"
    CONDITIONAL_GO = "CONDITIONAL_GO"
    NO_GO = "NO_GO"


class DecisionRecord(BaseModel):
    verdict: DecisionVerdict = DecisionVerdict.CONDITIONAL_GO
    confidence: float = 0.0
    nps_prediction: float = 0.0
    rationale: str = ""
    conditions: List[str] = Field(default_factory=list)
    evidence_ids: List[str] = Field(default_factory=list)
    reviewer: str = "ai"  # ai | human
    timestamp: str = Field(default_factory=lambda: datetime.utcnow().isoformat())


class ProductProposal(BaseModel):
    concept: ProductConcept
    prfaq: PRFAQ
    feasibility: FeasibilityAssessment
    nps: NPSPrediction
    addressed_opportunities: List[Opportunity] = Field(default_factory=list)
    decision: DecisionRecord
    concept_image_path: Optional[str] = None
    narration_audio_path: Optional[str] = None


```

```python
#                评测 / 对比
class RubricScore(BaseModel):
    groundedness: float = 0.0          # 论断带引用的比例
    faithfulness: float = 0.0          # 引用与论断语义一致比例
    citation_hit_rate: float = 0.0     # 逐字命中率
    opportunity_coverage: float = 0.0  # 提案覆盖的高分机会比例
    persona_fidelity: float = 0.0      # 合成用户来自真实评论 grounding 的比例
    mean_iterations: float = 0.0
    explainability: float = 0.0
    overall: float = 0.0
    notes: List[str] = Field(default_factory=list)


class ArmMetrics(BaseModel):
    """单组（A 经验驱动 / B AI 原生）的指标。"""

    arm: str
    opportunity_coverage: float = 0.0
    evidence_citations: int = 0
    validated_assumptions: int = 0
    real_pain_hit_rate: float = 0.0
    feasibility_risks_identified: int = 0
    nps_prediction: float = 0.0
    elapsed_seconds: float = 0.0
    distinct_personas_consulted: int = 0


class ComparisonReport(BaseModel):
    category: str
    brief: str
    arm_a: ArmMetrics   # 经验驱动
    arm_b: ArmMetrics   # AI 原生
    deltas: Dict[str, float] = Field(default_factory=dict)
    narrative: str = ""


#                LLM
class LLMResponse(BaseModel):
    text: str
    model: str = ""
    provider: str = "offline"
    tokens_in: int = 0
    tokens_out: int = 0
    latency_ms: float = 0.0
```

backend/anker_studio/common/config.py (68 行)

```python
"""集中配置（Common 层）。从环境变量/.env 读取，全程只读。"""
from __future__ import annotations

import os
from functools import lru_cache
from pathlib import Path

from pydantic import BaseModel, Field

```

```python
# 项目根目录: .../anker-ai-product-studio
PROJECT_ROOT = Path(__file__).resolve().parents[3]


def _load_dotenv() -> None:
    """尽力加载 .env（缺少 python-dotenv 也不报错）。"""
    try:
        from dotenv import load_dotenv

        load_dotenv(PROJECT_ROOT / ".env")
    except Exception:  # noqa: BLE001 - 配置加载失败不应中断程序
        pass


class Settings(BaseModel):
    """运行配置。"""

    # LLM
    llm_provider: str = Field(default="offline", description="offline | minimax")
    minimax_api_key: str = Field(default="")
    minimax_base_url: str = Field(default="https://api.minimax.io/v1")
    minimax_model: str = Field(default="MiniMax-M3")

    # 媒体
    enable_media: bool = Field(default=False)

    # 运行
    max_retrieval_iters: int = Field(default=3)
    hitl: bool = Field(default=False, description="决策闸是否人工确认")
    trace_dir: str = Field(default="runs")

    # 路径
    project_root: str = Field(default=str(PROJECT_ROOT))
    data_dir: str = Field(default=str(PROJECT_ROOT / "data"))

    @property
    def trace_path(self) -> Path:
        p = PROJECT_ROOT / self.trace_dir
        p.mkdir(parents=True, exist_ok=True)
        return p


@lru_cache(maxsize=1)
def settings() -> Settings:
    _load_dotenv()

    def _bool(name: str, default: bool) -> bool:
        return os.getenv(name, str(default)).strip().lower() in {"1", "true", "yes", "on"}

    return Settings(
        llm_provider=os.getenv("ANKER_LLM_PROVIDER", "offline").strip().lower(),
        minimax_api_key=os.getenv("MINIMAX_API_KEY", "").strip(),
        minimax_base_url=os.getenv("MINIMAX_BASE_URL", "https://api.minimax.io/v1").strip(),
        minimax_model=os.getenv("MINIMAX_MODEL", "MiniMax-M3").strip(),
        enable_media=_bool("ANKER_ENABLE_MEDIA", False),
        max_retrieval_iters=int(os.getenv("ANKER_MAX_RETRIEVAL_ITERS", "3")),
        hitl=_bool("ANKER_HITL", False),
```

```
        trace_dir=os.getenv("ANKER_TRACE_DIR", "runs").strip(),
    )
```

backend/anker_studio/common/logging.py (31 行)

```python
"""统一日志（Common 层）。一律使用 loguru。"""
from __future__ import annotations

import sys

from loguru import logger

_CONFIGURED = False


def setup_logging(level: str = "INFO") -> "logger.__class__":  # type: ignore[name-defined]
    """配置并返回 loguru logger（幂等）。"""
    global _CONFIGURED
    if not _CONFIGURED:
        logger.remove()
        logger.add(
            sys.stderr,
            level=level,
            format=(
                "<green>{time:HH:mm:ss}</green> | <level>{level: <7}</level> | "
                "<cyan>{extra[node]}</cyan> | {message}"
            ),
            colorize=True,
        )
        logger.configure(extra={"node": "-"})
        _CONFIGURED = True
    return logger


# 默认导出已配置的 logger
log = setup_logging()
```

backend/anker_studio/common/tools.py (58 行)

```python
"""工具注册与读写分离（Common 层）。

对齐美团 `@is_tool(ToolType.READ/WRITE)` 范式：
- READ：无副作用、可并发。
- WRITE：有副作用，必须两步确认（preview -> 人工/HITL 确认 -> execute）。
工具失败返回结构化错误字符串而非抛异常（"错误即信息"）。
"""
from __future__ import annotations

from enum import Enum
from typing import Callable, Dict, List


class ToolType(str, Enum):
    READ = "read"
    WRITE = "write"
```

```python


_REGISTRY: Dict[str, "RegisteredTool"] = {}


class RegisteredTool:
    def __init__(self, fn: Callable, tool_type: ToolType, name: str, doc: str):
        self.fn = fn
        self.tool_type = tool_type
        self.name = name
        self.doc = doc
        self.requires_confirmation = tool_type == ToolType.WRITE

    def __call__(self, *args, **kwargs):
        try:
            return self.fn(*args, **kwargs)
        except Exception as exc:  # noqa: BLE001 - 错误即信息
            return f"[TOOL_ERROR] {self.name}: {exc}"


def tool(tool_type: ToolType) -> Callable:
    """注册一个工具。WRITE 工具会被标记为需要两步确认。"""

    def deco(fn: Callable) -> RegisteredTool:
        rt = RegisteredTool(
            fn=fn,
            tool_type=tool_type,
            name=fn.__name__,
            doc=(fn.__doc__ or "").strip(),
        )
        _REGISTRY[fn.__name__] = rt
        return rt

    return deco


def list_tools() -> List[str]:
    return sorted(_REGISTRY.keys())


def get_tool(name: str) -> RegisteredTool:
    return _REGISTRY[name]
```

backend/anker_studio/common/citations.py (117 行)

```python
"""引用校验（Common 层）—— 本项目反幻觉的命门。

提供两类确定性校验（不依赖 LLM 自证）：
1. verify_quote：引用的片段必须在某条 Evidence 文本中逐字出现。
2. claim_supported：论断与其引用的 Evidence 必须有足够的词面重叠。
并据此计算 groundedness / faithfulness / citation_hit_rate。
"""
from __future__ import annotations

import re
from typing import Dict, Iterable, List, Sequence, Tuple
```

```python

from anker_studio.common.models import Evidence

_WORD_RE = re.compile(r"[a-zA-Z]+|[\u4e00-\u9fff]")
_STOP = {
    "the", "a", "an", "and", "or", "of", "to", "is", "are", "for", "with",
    "this", "that", "it", "in", "on", "i", "you", "my", "very", "too", "but",
    "了", "的", "是", "和", "也", "在", "我", "你", "它", "很", "太",
}


def _norm(text: str) -> str:
    return re.sub(r"\s+", " ", (text or "").strip().lower())


def _tokens(text: str) -> List[str]:
    return [t for t in _WORD_RE.findall((text or "").lower()) if t not in _STOP]


def build_index(evidences: Sequence[Evidence]) -> Dict[str, Evidence]:
    return {e.source_id: e for e in evidences}


def verify_quote(quote: str, evidence_text: str) -> bool:
    """引用片段是否在 evidence 文本中逐字（忽略空白/大小写）出现。"""
    q = _norm(quote)
    if not q:
        return False
    return q in _norm(evidence_text)


def claim_supported(
    claim: str,
    evidence_ids: Sequence[str],
    index: Dict[str, Evidence],
    min_overlap: int = 2,
    aspect_terms: "Dict[str, Sequence[str]] | None" = None,
) -> bool:
    """论断是否被其引用的 Evidence 支撑。

    两条通路（任一成立即支撑）：
    1. 词面重叠 >= min_overlap（同语言）。
    2. 跨语言 aspect 桥接：claim 提到某 aspect（中文名），且 evidence 文本含该 aspect 的
       任一关键词（英文）。解决"中文论断 + 英文评论"的对齐问题。
    """
    ids = [i for i in evidence_ids if i in index]
    if not ids:
        return False

    # 通路 2：aspect 桥接
    if aspect_terms:
        claim_low = (claim or "").lower()
        for aspect, terms in aspect_terms.items():
            if aspect in claim or aspect.lower() in claim_low:
                for sid in ids:
                    ev_low = index[sid].text.lower()
                    if any(str(t).lower() in ev_low for t in terms):
```

```
                        return True

    # 通路 1：词面重叠
    claim_tokens = set(_tokens(claim))
    if not claim_tokens:
        return bool(ids)
    for sid in ids:
        ev_tokens = set(_tokens(index[sid].text))
        if len(claim_tokens & ev_tokens) >= min_overlap:
            return True
    return False


def evaluate_citations(
    claims: Iterable[Tuple[str, Sequence[str]]],
    evidences: Sequence[Evidence],
    aspect_terms: "Dict[str, Sequence[str]] | None" = None,
) -> Dict[str, float]:
    """对 (claim, evidence_ids) 列表统计 groundedness / faithfulness / hit_rate。

    - groundedness：带 >=1 个有效引用的论断比例。
    - faithfulness：引用确实支撑论断的比例（词面重叠）。
    - citation_hit_rate：引用的 evidence_id 真实存在的比例。
    """
    index = build_index(evidences)
    claims = list(claims)
    total = len(claims)
    if total == 0:
        return {"groundedness": 0.0, "faithfulness": 0.0, "citation_hit_rate": 0.0}

    grounded = 0
    faithful = 0
    total_refs = 0
    valid_refs = 0
    for claim, ids in claims:
        ids = list(ids or [])
        valid = [i for i in ids if i in index]
        total_refs += len(ids)
        valid_refs += len(valid)
        if valid:
            grounded += 1
            if claim_supported(claim, valid, index, aspect_terms=aspect_terms):
                faithful += 1

    return {
        "groundedness": round(grounded / total, 4),
        "faithfulness": round(faithful / total, 4),
        "citation_hit_rate": round((valid_refs / total_refs) if total_refs else 0.0, 4),
    }
```

B Infrastructure 层（LLM 网关 / 数据 / RAG / NLP / 媒体 / 观测）

backend/anker_studio/infrastructure/llm/gateway.py (107 行)

```python
"""统一 LLM 网关（Infrastructure 层）。

设计要点（呼应安克"确定性 vs 不确定性"）：
- 确定性核心由各分析模块产出结构化数值；网关主要负责"叙述/归纳/角色扮演/批判"。
- 两种 provider：
    offline —— 不联网，确定性：narrate() 直接返回传入文本，complete() 做抽取式摘要。
    minimax —— 调用 MiniMax M3 增强。
- 任何远程失败都自动降级到 offline，不中断流程（错误即信息）。
"""
from __future__ import annotations

import time
from typing import Optional

from anker_studio.common.config import Settings
from anker_studio.common.logging import log
from anker_studio.common.models import LLMResponse


class LLMGateway:
    def __init__(
        self,
        provider: str = "offline",
        minimax_client=None,
        default_model: str = "offline-deterministic",
    ):
        self.provider = provider
        self._minimax = minimax_client
        self.default_model = default_model

    @classmethod
    def from_settings(cls, settings: Settings) -> "LLMGateway":
        if settings.llm_provider == "minimax" and settings.minimax_api_key:
            from anker_studio.infrastructure.llm.minimax import MiniMaxClient

            client = MiniMaxClient(
                api_key=settings.minimax_api_key,
                base_url=settings.minimax_base_url,
                model=settings.minimax_model,
            )
            log.bind(node="llm").info(f"LLM provider=minimax model={settings.minimax_model}")
            return cls(provider="minimax", minimax_client=client, default_model=settings.minimax_model)
        log.bind(node="llm").info("LLM provider=offline（确定性模式，无网络）")
        return cls(provider="offline")

    @property
    def has_remote(self) -> bool:
        return self.provider == "minimax" and self._minimax is not None

    #  主接口
    def complete(
        self,
        system: str,
        prompt: str,
        model: Optional[str] = None,
```

```
        max_tokens: int = 1024,
        temperature: float = 0.6,
        task: str = "",
    ) -> LLMResponse:
        if self.has_remote:
            try:
                return self._minimax.chat(system, prompt, model, max_tokens, temperature)
            except Exception as exc:  # noqa: BLE001 - 远程失败降级
                log.bind(node="llm").warning(f"MiniMax 调用失败，降级 offline：{exc}")
        return self._offline_complete(system, prompt, task)

    def narrate(self, default_text: str, instruction: str, context: str = "", task: str = "") -> str:
        """把确定性结果包装成更顺畅的叙述。

        offline：直接返回 default_text（已是可读的确定性文本）。
        minimax：在 default_text 基础上做润色/扩写，但不得引入新事实。
        """
        if not self.has_remote:
            return default_text
        system = (
            "你是安克的资深产品负责人。只能基于给定事实进行润色与归纳，"
            "禁止引入任何未提供的新数据或新事实。输出简洁、专业、中文。"
        )
        prompt = (
            f"任务：{instruction}\n\n"
            f"事实与上下文（不得超出）：\n{context}\n\n"
            f"待润色的草稿：\n{default_text}\n\n请输出润色后的版本："
        )
        try:
            resp = self._minimax.chat(system, prompt, max_tokens=900, temperature=0.5)
            return resp.text.strip() or default_text
        except Exception as exc:  # noqa: BLE001
            log.bind(node="llm").warning(f"narrate 降级：{exc}")
            return default_text

    #   offline 确定性实现
    @staticmethod
    def _offline_complete(system: str, prompt: str, task: str) -> LLMResponse:
        start = time.time()
        # 抽取式摘要：取 prompt 的前若干句作为"回答"，保证零网络可运行且可复现。
        text = prompt.strip()
        sentences = [s.strip() for s in text.replace("\n", " ").split("。") if s.strip()]
        summary = "。".join(sentences[:3])
        out = summary or text[:400]
        return LLMResponse(
            text=out,
            model="offline-deterministic",
            provider="offline",
            tokens_in=len(prompt),
            tokens_out=len(out),
            latency_ms=round((time.time() - start) * 1000, 2),
        )
```

backend/anker_studio/infrastructure/llm/minimax.py (62 行)

```
"""MiniMax M3 客户端（OpenAI 兼容 /chat/completions）。
```

```
只负责把一次 chat 请求发给 MiniMax 并解析文本，失败时抛出 RuntimeError，由网关降级。
"""
from __future__ import annotations

import json
import time
from typing import List, Optional

from anker_studio.common.models import LLMResponse


class MiniMaxClient:
    def __init__(self, api_key: str, base_url: str, model: str):
        self.api_key = api_key
        self.base_url = base_url.rstrip("/")
        self.model = model

    def chat(
        self,
        system: str,
        prompt: str,
        model: Optional[str] = None,
        max_tokens: int = 1024,
        temperature: float = 0.6,
    ) -> LLMResponse:
        import requests  # 局部导入：offline 模式无需安装 requests

        url = f"{self.base_url}/chat/completions"
        messages: List[dict] = []
        if system:
            messages.append({"role": "system", "content": system})
        messages.append({"role": "user", "content": prompt})
        payload = {
            "model": model or self.model,
            "messages": messages,
            "max_tokens": max_tokens,
            "temperature": temperature,
        }
        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json",
        }
        start = time.time()
        resp = requests.post(url, headers=headers, data=json.dumps(payload), timeout=90)
        if resp.status_code != 200:
            raise RuntimeError(f"MiniMax HTTP {resp.status_code}: {resp.text[:300]}")
        data = resp.json()
        try:
            text = data["choices"][0]["message"]["content"]
        except (KeyError, IndexError, TypeError) as exc:
            raise RuntimeError(f"MiniMax 响应解析失败: {exc}; body={str(data)[:300]}")
        usage = data.get("usage", {}) or {}
        return LLMResponse(
            text=text or "",
            model=payload["model"],
            provider="minimax",
```

```
            tokens_in=int(usage.get("prompt_tokens", 0) or 0),
            tokens_out=int(usage.get("completion_tokens", 0) or 0),
            latency_ms=round((time.time() - start) * 1000, 1),
        )
```

backend/anker_studio/infrastructure/data/loader.py (113 行)

```python
"""数据加载（Infrastructure 层）。

统一把不同来源的评论加载成 `Evidence`：
- 优先读取 `data/processed/*.jsonl`（由 amazon_reviews 下载脚本生成的真实数据）。
- 否则回退到 `data/sample/*.jsonl`（随仓库提供的可复现样本）。
品牌归类（target / competitor）由 `store`/`brand` 字段决定。
"""
from __future__ import annotations

import json
from pathlib import Path
from typing import Dict, List, Optional

from anker_studio.common.config import settings
from anker_studio.common.logging import log
from anker_studio.common.models import Evidence, SourceType

# soundcore 为目标品牌；其余为竞品（用于 AMI 竞品拆解）
TARGET_BRAND = "soundcore"
COMPETITOR_BRANDS = ["Bose", "Sony", "Samsung", "Apple"]


def _data_root() -> Path:
    return Path(settings().data_dir)


def _read_jsonl(path: Path) -> List[dict]:
    rows: List[dict] = []
    if not path.exists():
        return rows
    with path.open("r", encoding="utf-8") as fp:
        for line in fp:
            line = line.strip()
            if not line:
                continue
            try:
                rows.append(json.loads(line))
            except json.JSONDecodeError as exc:
                log.bind(node="data").warning(f"跳过坏行 {path.name}: {exc}")
    return rows


def _row_to_evidence(row: dict) -> Evidence:
    brand = str(row.get("brand") or row.get("store") or "unknown")
    stype = row.get("source_type")
    if stype not in {s.value for s in SourceType}:
        stype = (
            SourceType.REVIEW.value
            if brand.lower() == TARGET_BRAND
```

```
            else SourceType.COMPETITOR.value
        )
    return Evidence(
        source_id=str(row.get("source_id") or row.get("id") or row.get("asin") or id(row)),
        source_type=SourceType(stype),
        brand=brand,
        product=str(row.get("product") or row.get("title") or ""),
        rating=row.get("rating"),
        text=str(row.get("text") or row.get("review") or ""),
        date=row.get("date") or row.get("timestamp"),
        url=row.get("url"),
        helpful_votes=int(row.get("helpful_votes") or row.get("helpful_vote") or 0),
    )


def load_evidence(category: str = "audio") -> List[Evidence]:
    """加载该品类的全部 Evidence（真实优先，样本兜底）。"""
    root = _data_root()
    processed = root / "processed"
    sample = root / "sample"

    def _jsonl(d: Path) -> List[Path]:
        # 忽略隐藏文件与 macOS AppleDouble（._*）伴随文件，避免二进制/编码污染
        return sorted(p for p in d.glob("*.jsonl") if not p.name.startswith("."))

    files: List[Path] = []
    if processed.exists():
        files = _jsonl(processed)
    if not files and sample.exists():
        files = _jsonl(sample)

    evidences: List[Evidence] = []
    for f in files:
        for row in _read_jsonl(f):
            ev = _row_to_evidence(row)
            if ev.text:
                evidences.append(ev)

    src = "processed(真实)" if (processed.exists() and any(processed.glob("*.jsonl"))) else "sample(样本)"
    log.bind(node="data").info(
        f"加载 {len(evidences)} 条 Evidence（来源={src}, category={category}）"
    )
    return evidences


def split_by_brand(evidences: List[Evidence]) -> Dict[str, List[Evidence]]:
    """切分目标品牌 vs 竞品。"""
    target: List[Evidence] = []
    competitors: Dict[str, List[Evidence]] = {b: [] for b in COMPETITOR_BRANDS}
    other: List[Evidence] = []
    for e in evidences:
        if e.brand.lower() == TARGET_BRAND:
            target.append(e)
        elif e.brand in competitors:
            competitors[e.brand].append(e)
        else:
            other.append(e)
```

```
    return {"target": target, "competitors": competitors, "other": other}


def filter_evidence(evidences: List[Evidence], brand: Optional[str] = None) -> List[Evidence]:
    if brand is None:
        return evidences
    return [e for e in evidences if e.brand.lower() == brand.lower()]
```

backend/anker_studio/infrastructure/data/amazon_reviews.py (122 行)

```
"""Amazon Reviews 2023 (McAuley-Lab) 下载与品牌筛选（Infrastructure 层）。

把真实评论按品牌（soundcore / Bose / Sony / Samsung / Apple）筛出，落地为
`data/processed/<brand>.jsonl`，供 loader 读取。需要可选依赖 `datasets`。

用法：
    python -m anker_studio.infrastructure.data.amazon_reviews --max-per-brand 800
未安装 datasets 时给出清晰提示，不影响系统以样本数据运行。
"""
from __future__ import annotations

import argparse
import json
import re
from pathlib import Path
from typing import Dict, List

from anker_studio.common.config import settings
from anker_studio.common.logging import log

# 品牌 -> 在 store/title 中的匹配正则
BRAND_PATTERNS: Dict[str, str] = {
    "soundcore": r"soundcore|anker",
    "Bose": r"\bbose\b",
    "Sony": r"\bsony\b",
    "Samsung": r"samsung|galaxy buds",
    "Apple": r"\bapple\b|airpods",
}
# 仅保留音频耳机相关产品
AUDIO_HINT = re.compile(r"earbud|headphone|earphone|tws|wireless ear|anc|noise cancel", re.I)


def _match_brand(text: str) -> str:
    t = (text or "").lower()
    for brand, pat in BRAND_PATTERNS.items():
        if re.search(pat, t):
            return brand
    return ""


def download_and_filter(max_per_brand: int = 800) -> None:
    try:
        from datasets import load_dataset  # type: ignore
    except Exception:  # noqa: BLE001
        log.bind(node="data").error(
            "未安装 datasets。请 `pip install datasets` 后重试；"
```

```
            "或直接使用随仓库的 data/sample 样本运行。"
        )
        return

    out_dir = Path(settings().data_dir) / "processed"
    out_dir.mkdir(parents=True, exist_ok=True)

    log.bind(node="data").info("加载 raw_meta_Electronics（流式）以建立 asin->品牌 映射……")
    meta = load_dataset(
        "McAuley-Lab/Amazon-Reviews-2023",
        "raw_meta_Electronics",
        split="full",
        streaming=True,
        trust_remote_code=True,
    )
    asin_brand: Dict[str, str] = {}
    for i, m in enumerate(meta):
        title = str(m.get("title") or "")
        store = str(m.get("store") or "")
        if not AUDIO_HINT.search(title):
            continue
        brand = _match_brand(store) or _match_brand(title)
        if brand:
            asin_brand[str(m.get("parent_asin") or m.get("asin"))] = brand
        if i > 400000 or len(asin_brand) > 20000:
            break
    log.bind(node="data").info(f"建立映射 {len(asin_brand)} 个音频商品。")

    log.bind(node="data").info("加载 raw_review_Electronics（流式）并按品牌筛选……")
    reviews = load_dataset(
        "McAuley-Lab/Amazon-Reviews-2023",
        "raw_review_Electronics",
        split="full",
        streaming=True,
        trust_remote_code=True,
    )
    buckets: Dict[str, List[dict]] = {b: [] for b in BRAND_PATTERNS}
    for r in reviews:
        asin = str(r.get("parent_asin") or r.get("asin"))
        brand = asin_brand.get(asin)
        if not brand or len(buckets[brand]) >= max_per_brand:
            continue
        text = str(r.get("text") or "")
        if len(text) < 40:
            continue
        buckets[brand].append(
            {
                "source_id": f"{brand}-{asin}-{r.get('user_id','')[:6]}-{len(buckets[brand])}",
                "brand": brand,
                "product": str(r.get("title") or ""),
                "rating": r.get("rating"),
                "text": text,
                "date": r.get("timestamp"),
                "helpful_votes": r.get("helpful_vote") or 0,
            }
        )
        if all(len(v) >= max_per_brand for v in buckets.values()):
```

```
            break

    for brand, rows in buckets.items():
        path = out_dir / f"{brand}.jsonl"
        with path.open("w", encoding="utf-8") as fp:
            for row in rows:
                fp.write(json.dumps(row, ensure_ascii=False) + "\n")
        log.bind(node="data").info(f"写出 {len(rows)} 条 -> {path}")


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--max-per-brand", type=int, default=800)
    args = ap.parse_args()
    download_and_filter(args.max_per_brand)


if __name__ == "__main__":
    main()
```

backend/anker_studio/infrastructure/data/connectors.py (71 行)

```
"""外部连接器（Infrastructure 层）：趋势 / 网页。

均为可选增强，缺少依赖或网络时返回内置的、带出处的离线趋势信号，保证可复现。
"""
from __future__ import annotations

from typing import List

from anker_studio.common.logging import log
from anker_studio.common.models import Evidence, SourceType, TrendSignal

# 离线兜底：消费电子/音频行业公开可查的结构性趋势（带出处链接）
_OFFLINE_TRENDS: List[TrendSignal] = [
    TrendSignal(
        name="端侧 AI 音频（On-device AI audio）",
        direction="up",
        summary="存算一体/端侧 NPU 让降噪、听力个性化、实时翻译从云端下沉到耳机本体。",
        evidence_ids=["trend-ondevice-ai"],
    ),
    TrendSignal(
        name="个性化听力与健康（Hearing personalization / Hearing-aid 化）",
        direction="up",
        summary="Apple AirPods Pro 的助听功能、各家听力轮廓校准，使耳机带上健康属性。",
        evidence_ids=["trend-hearing-health"],
    ),
    TrendSignal(
        name="多设备无缝连接（Multipoint / 快速切换）",
        direction="up",
        summary="用户在手机/笔电/平板间频繁切换，连接稳定性与多点是高频痛点。",
        evidence_ids=["trend-multipoint"],
    ),
]

_OFFLINE_TREND_EVIDENCE: List[Evidence] = [
```

```
    Evidence(
        source_id="trend-ondevice-ai",
        source_type=SourceType.TREND,
        brand="industry",
        text="On-device AI audio is rising: compute-in-memory and on-device NPUs move ANC, "
        "hearing personalization and real-time translation from cloud to the earbud itself.",
        url="https://www.soundcore.com/anker-thus-ai-chip",
    ),
    Evidence(
        source_id="trend-hearing-health",
        source_type=SourceType.TREND,
        brand="industry",
        text="Hearing personalization and hearing-aid features (e.g. AirPods Pro hearing aid) "
        "turn earbuds into a health device category.",
        url="https://www.nngroup.com/articles/synthetic-users/",
    ),
    Evidence(
        source_id="trend-multipoint",
        source_type=SourceType.TREND,
        brand="industry",
        text="Multipoint and fast device switching are increasingly demanded as users move "
        "between phone, laptop and tablet throughout the day.",
        url="https://amazon-reviews-2023.github.io/",
    ),
]


def fetch_trends(keywords: List[str]) -> "tuple[List[TrendSignal], List[Evidence]]":
    """返回趋势信号 + 对应 Evidence。优先 pytrends，失败回退离线。"""
    try:
        from pytrends.request import TrendReq  # type: ignore  # noqa: F401

        # 真实实现可在此查询 Google Trends；为离线可复现，这里仍返回结构化离线信号。
        log.bind(node="trends").info("pytrends 可用；当前返回离线结构化趋势以保证可复现。")
    except Exception:  # noqa: BLE001
        log.bind(node="trends").info("pytrends 不可用，使用离线趋势信号。")
    return _OFFLINE_TRENDS, _OFFLINE_TREND_EVIDENCE
```

backend/anker_studio/infrastructure/nlp/lexicons.py (55 行)

```
"""领域词典（Infrastructure 层）。

音频耳机品类的 aspect 词典 + 通用情感词典 + 否定词。
确定性 ABSA 的基础，使 VOC 洞察"有数据支撑且可复现"。
"""
from __future__ import annotations

from typing import Dict, List

# aspect -> 关键词（小写）。中英混合以兼容真实 Amazon 英文评论与中文样本。
ASPECT_LEXICON: Dict[str, List[str]] = {
    "降噪 ANC": [
        "anc", "noise cancel", "noise-cancel", "noise cancelling", "noise canceling",
        "ambient", "transparency", "降噪", "噪音", "通透",
    ],
    "音质": [
```

```
        "sound", "audio quality", "bass", "treble", "mids", "soundstage", "音质", "低音", "高音", "声音",
    ],
    "佩戴舒适度": [
        "fit", "comfort", "comfortable", "ear", "tips", "snug", "hurt", "fall out",
        "舒适", "佩戴", "耳朵", "掉", "贴合",
    ],
    "通话质量": [
        "call", "calls", "mic", "microphone", "voice", "wind", "通话", "麦克风", "语音",
    ],
    "续航": [
        "battery", "battery life", "charge", "hours", "playtime", "续航", "电池", "充电", "小时",
    ],
    "App 体验": [
        "app", "eq", "equalizer", "firmware", "update", "应用", "固件", "均衡器",
    ],
    "连接稳定性/多点": [
        "connect", "connection", "bluetooth", "pairing", "multipoint", "switch", "drop", "latency", "lag",
        "连接", "蓝牙", "配对", "多点", "切换", "断连", "延迟",
    ],
    "价格/价值": [
        "price", "expensive", "cheap", "worth", "value", "money", "价格", "贵", "便宜", "值",
    ],
    "耐用性": [
        "durable", "broke", "broken", "stopped working", "quality control", "build", "耐用", "坏", "做工",
    ],
}

POSITIVE_WORDS: List[str] = [
    "great", "good", "excellent", "amazing", "love", "best", "perfect", "comfortable",
    "clear", "impressive", "solid", "worth", "happy", "recommend", "fantastic", "crisp",
    "好", "棒", "优秀", "喜欢", "完美", "清晰", "值", "推荐", "舒适", "稳定",
]
NEGATIVE_WORDS: List[str] = [
    "bad", "poor", "terrible", "awful", "hate", "worst", "disappointing", "disappointed",
    "uncomfortable", "muffled", "weak", "drop", "drops", "broke", "broken", "expensive",
    "fail", "fails", "annoying", "laggy", "lag", "useless", "cheap", "issue", "issues", "problem",
    "差", "糟", "失望", "难受", "弱", "断", "坏", "贵", "卡", "延迟", "问题", "故障",
]
NEGATORS: List[str] = ["not", "no", "never", "hardly", "lack", "without", "n't", "不", "没", "无"]
```

backend/anker_studio/infrastructure/nlp/absa.py (108 行)

```
"""确定性 Aspect-Based Sentiment Analysis（Infrastructure 层）。

输入一批 Evidence（评论），按 aspect 统计提及量、负面率、代表性引用。
情感判定：子句级关键词 + 否定翻转 + 评分(rating)兜底。
全程不调用 LLM，保证可复现、可溯源。
"""
from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Dict, List, Sequence

from anker_studio.common.models import Evidence
from anker_studio.infrastructure.nlp.lexicons import (
```

```
    ASPECT_LEXICON,
    NEGATIVE_WORDS,
    NEGATORS,
    POSITIVE_WORDS,
)

_CLAUSE_SPLIT = re.compile(r"[.!?;,\n。！？；，]")


@dataclass
class AspectStat:
    aspect: str
    mentions: int = 0
    negative: int = 0
    positive: int = 0
    evidence_ids: List[str] = field(default_factory=list)
    negative_evidence_ids: List[str] = field(default_factory=list)

    @property
    def negative_rate(self) -> float:
        return round(self.negative / self.mentions, 4) if self.mentions else 0.0

    @property
    def positive_rate(self) -> float:
        return round(self.positive / self.mentions, 4) if self.mentions else 0.0


def _clause_sentiment(clause: str, rating) -> str:
    """返回 'pos' | 'neg' | 'neu'。"""
    words = re.findall(r"[a-zA-Z']+|[\u4e00-\u9fff]", clause.lower())
    score = 0
    for i, w in enumerate(words):
        polarity = 0
        if w in POSITIVE_WORDS:
            polarity = 1
        elif w in NEGATIVE_WORDS:
            polarity = -1
        if polarity != 0:
            window = words[max(0, i - 3): i]
            if any(neg in window or w.endswith("n't") for neg in NEGATORS):
                polarity = -polarity
            score += polarity
    if score > 0:
        return "pos"
    if score < 0:
        return "neg"
    # 中性子句用整体评分兜底
    if rating is not None:
        try:
            r = float(rating)
            if r <= 2:
                return "neg"
            if r >= 4:
                return "pos"
        except (TypeError, ValueError):
            return "neu"
    return "neu"
```

```python


def _match_aspects(clause: str) -> List[str]:
    c = clause.lower()
    hit = []
    for aspect, kws in ASPECT_LEXICON.items():
        if any(kw in c for kw in kws):
            hit.append(aspect)
    return hit


def analyze(evidences: Sequence[Evidence]) -> Dict[str, AspectStat]:
    """对评论做 ABSA，返回 {aspect: AspectStat}。"""
    stats: Dict[str, AspectStat] = {a: AspectStat(aspect=a) for a in ASPECT_LEXICON}
    for ev in evidences:
        seen_aspects_this_review: Dict[str, str] = {}
        for clause in _CLAUSE_SPLIT.split(ev.text):
            clause = clause.strip()
            if len(clause) < 3:
                continue
            for aspect in _match_aspects(clause):
                sent = _clause_sentiment(clause, ev.rating)
                # 每条评论对同一 aspect 取"最负面"的判定，避免重复计数夸大
                prev = seen_aspects_this_review.get(aspect)
                if prev is None or (sent == "neg" and prev != "neg"):
                    seen_aspects_this_review[aspect] = sent
        for aspect, sent in seen_aspects_this_review.items():
            st = stats[aspect]
            st.mentions += 1
            if ev.source_id not in st.evidence_ids:
                st.evidence_ids.append(ev.source_id)
            if sent == "neg":
                st.negative += 1
                st.negative_evidence_ids.append(ev.source_id)
            elif sent == "pos":
                st.positive += 1
    return stats
```

backend/anker_studio/infrastructure/rag/retrieval.py (138 行)

```python
"""混合检索（Infrastructure 层）：BM25 + TF-IDF，RRF 融合。

纯 Python 实现（无需 numpy/chromadb 也能运行）。提供 Agentic RAG 所需的
"检索 → 打分 → 重写 → 再检索" 迭代能力，并返回可溯源的 Evidence。
"""
from __future__ import annotations

import math
import re
from collections import Counter
from dataclasses import dataclass
from typing import Dict, List, Sequence, Tuple

from anker_studio.common.config import settings
from anker_studio.common.logging import log
from anker_studio.common.models import Evidence
```

```python

_TOKEN_RE = re.compile(r"[a-zA-Z']+|[\u4e00-\u9fff]")
_STOP = {"the", "a", "an", "and", "or", "of", "to", "is", "are", "for", "with", "it", "this", "that"}


def _tok(text: str) -> List[str]:
    return [t for t in _TOKEN_RE.findall((text or "").lower()) if t not in _STOP and len(t) > 1]


@dataclass
class _Doc:
    idx: int
    tokens: List[str]
    tf: Counter


class HybridRetriever:
    """对一组 Evidence 建立 BM25 + TF-IDF 索引并支持检索。"""

    def __init__(self, evidences: Sequence[Evidence], k1: float = 1.5, b: float = 0.75):
        self.evidences = list(evidences)
        self.k1 = k1
        self.b = b
        self.docs: List[_Doc] = []
        self.df: Counter = Counter()
        self.idf: Dict[str, float] = {}
        self.avgdl = 0.0
        self._build()

    def _build(self) -> None:
        total_len = 0
        for i, ev in enumerate(self.evidences):
            toks = _tok(ev.text)
            self.docs.append(_Doc(idx=i, tokens=toks, tf=Counter(toks)))
            total_len += len(toks)
            for t in set(toks):
                self.df[t] += 1
        n = max(1, len(self.docs))
        self.avgdl = total_len / n if n else 0.0
        for term, df in self.df.items():
            self.idf[term] = math.log(1 + (n - df + 0.5) / (df + 0.5))
        log.bind(node="rag").info(f"索引建立：{n} 篇文档，词表 {len(self.df)}。")

    def _bm25(self, q_tokens: List[str]) -> Dict[int, float]:
        scores: Dict[int, float] = {}
        for doc in self.docs:
            dl = len(doc.tokens) or 1
            s = 0.0
            for t in q_tokens:
                if t not in doc.tf:
                    continue
                idf = self.idf.get(t, 0.0)
                freq = doc.tf[t]
                denom = freq + self.k1 * (1 - self.b + self.b * dl / (self.avgdl or 1))
                s += idf * (freq * (self.k1 + 1)) / (denom or 1)
            if s > 0:
                scores[doc.idx] = s
```

```
        return scores

    def _tfidf(self, q_tokens: List[str]) -> Dict[int, float]:
        scores: Dict[int, float] = {}
        q = Counter(q_tokens)
        for doc in self.docs:
            s = 0.0
            for t, qf in q.items():
                if t in doc.tf:
                    s += (qf * self.idf.get(t, 0.0)) * (doc.tf[t] * self.idf.get(t, 0.0))
            if s > 0:
                norm = math.sqrt(sum(v * v for v in doc.tf.values()) or 1)
                scores[doc.idx] = s / norm
        return scores

    @staticmethod
    def _rrf(rankings: Sequence[Dict[int, float]], k: int = 60) -> Dict[int, float]:
        fused: Dict[int, float] = {}
        for scores in rankings:
            ordered = sorted(scores.items(), key=lambda kv: kv[1], reverse=True)
            for rank, (idx, _) in enumerate(ordered):
                fused[idx] = fused.get(idx, 0.0) + 1.0 / (k + rank + 1)
        return fused

    def search(self, query: str, top_k: int = 8) -> List[Tuple[Evidence, float]]:
        q_tokens = _tok(query)
        if not q_tokens:
            return []
        fused = self._rrf([self._bm25(q_tokens), self._tfidf(q_tokens)])
        ordered = sorted(fused.items(), key=lambda kv: kv[1], reverse=True)[:top_k]
        return [(self.evidences[i], round(score, 5)) for i, score in ordered]


class RagService:
    """Agentic RAG: 检索→打分→重写→再检索（上限 max_iters）。"""

    def __init__(self, retriever: HybridRetriever, max_iters: int = 3):
        self.retriever = retriever
        self.max_iters = max_iters

    def agentic_search(self, query: str, top_k: int = 8, min_hits: int = 3):
        """返回 (evidences, meta)；meta 含迭代次数，供评测统计 mean_iterations。"""
        attempts = 0
        results: List[Tuple[Evidence, float]] = []
        current = query
        while attempts < self.max_iters:
            attempts += 1
            results = self.retriever.search(current, top_k=top_k)
            if len(results) >= min_hits:
                break
            # 重写：放宽查询（去掉修饰，保留核心名词）
            current = self._rewrite(current)
            log.bind(node="rag").info(f"检索不足({len(results)})，重写查询 -> '{current}'（第{attempts}次）"
    )
        meta = {"iterations": attempts, "hits": len(results)}
        return [ev for ev, _ in results], meta

```

```
    @staticmethod
    def _rewrite(query: str) -> str:
        toks = _tok(query)
        # 保留信息量最大的前 5 个词
        return " ".join(toks[:5]) if toks else query


def build_rag(evidences: Sequence[Evidence]) -> RagService:
    return RagService(HybridRetriever(evidences), max_iters=settings().max_retrieval_iters)
```

backend/anker_studio/infrastructure/assets/media.py (64 行)

```
"""媒体资产生成（Infrastructure 层）：用 MiniMax `mmx` CLI 生成概念图/口播。

可选能力：需 `npm i -g mmx-cli` 且已 `mmx auth login`，并设 ANKER_ENABLE_MEDIA=true。
不可用时返回 None，不影响主流程。
"""
from __future__ import annotations

import shutil
import subprocess
from pathlib import Path
from typing import Optional

from anker_studio.common.config import settings
from anker_studio.common.logging import log


def _mmx_available() -> bool:
    return settings().enable_media and shutil.which("mmx") is not None


def _assets_dir() -> Path:
    p = Path(settings().project_root) / "assets"
    p.mkdir(parents=True, exist_ok=True)
    return p


def generate_concept_image(prompt: str, filename: str = "concept.png") -> Optional[str]:
    if not _mmx_available():
        log.bind(node="media").info("跳过概念图（未启用 mmx 或未安装）。")
        return None
    out_dir = _assets_dir()
    try:
        proc = subprocess.run(
            ["mmx", "image", "generate", "--prompt", prompt, "--aspect-ratio", "16:9",
             "--out-dir", str(out_dir), "--out-prefix", Path(filename).stem, "--quiet"],
            capture_output=True, text=True, timeout=180,
        )
        if proc.returncode != 0:
            log.bind(node="media").warning(f"mmx 概念图失败：{proc.stderr[:200]}")
            return None
        path = proc.stdout.strip().splitlines()[-1] if proc.stdout.strip() else None
        log.bind(node="media").info(f"概念图已生成：{path}")
        return path
    except (subprocess.SubprocessError, OSError) as exc:
```

```
        log.bind(node="media").warning(f"mmx 概念图异常: {exc}")
        return None


def synthesize_narration(text: str, filename: str = "narration.mp3") -> Optional[str]:
    if not _mmx_available():
        return None
    out = _assets_dir() / filename
    try:
        proc = subprocess.run(
            ["mmx", "speech", "synthesize", "--text", text[:9000], "--out", str(out), "--quiet"],
            capture_output=True, text=True, timeout=180,
        )
        if proc.returncode != 0:
            log.bind(node="media").warning(f"mmx 口播失败: {proc.stderr[:200]}")
            return None
        return str(out)
    except (subprocess.SubprocessError, OSError) as exc:
        log.bind(node="media").warning(f"mmx 口播异常: {exc}")
        return None
```

backend/anker_studio/infrastructure/observability/trace.py (68 行)

```
"""轻量 trace 观测（Infrastructure 层）。

把每个节点的执行写成 JSONL（runs/<run_id>.jsonl）+ 内存事件流，
供评测统计与前端实时可视化（对标 LangSmith/Langfuse 的最小可用形态）。
"""
from __future__ import annotations

import json
import time
from pathlib import Path
from typing import Any, Callable, Dict, List, Optional

from anker_studio.common.config import settings
from anker_studio.common.logging import log


class Tracer:
    def __init__(self, run_id: Optional[str] = None):
        self.run_id = run_id or time.strftime("run-%Y%m%d-%H%M%S")
        self.events: List[Dict[str, Any]] = []
        self.path: Path = settings().trace_path / f"{self.run_id}.jsonl"
        self._subscribers: List[Callable[[Dict[str, Any]], None]] = []

    def subscribe(self, fn: Callable[[Dict[str, Any]], None]) -> None:
        self._subscribers.append(fn)

    def emit(self, node: str, kind: str, **data: Any) -> None:
        event = {"ts": round(time.time(), 3), "node": node, "kind": kind, **data}
        self.events.append(event)
        try:
            with self.path.open("a", encoding="utf-8") as fp:
                fp.write(json.dumps(event, ensure_ascii=False, default=str) + "\n")
        except OSError as exc:
```

```
            log.bind(node="trace").warning(f"trace 写入失败: {exc}")
        for fn in list(self._subscribers):
            try:
                fn(event)
            except Exception as exc:  # noqa: BLE001 - 订阅者异常不影响主流程
                log.bind(node="trace").warning(f"trace 订阅者异常: {exc}")

    def node_span(self, node: str):
        return _Span(self, node)


class _Span:
    def __init__(self, tracer: Tracer, node: str):
        self.tracer = tracer
        self.node = node
        self._start = 0.0

    def __enter__(self) -> "_Span":
        self._start = time.time()
        self.tracer.emit(self.node, "start")
        log.bind(node=self.node).info("开始")
        return self

    def __exit__(self, exc_type, exc, tb) -> bool:
        elapsed = round((time.time() - self._start) * 1000, 1)
        if exc_type is not None:
            self.tracer.emit(self.node, "error", error=str(exc), elapsed_ms=elapsed)
            log.bind(node=self.node).error(f"失败: {exc}")
            return False
        self.tracer.emit(self.node, "end", elapsed_ms=elapsed)
        log.bind(node=self.node).info(f"完成 ({elapsed}ms)")
        return False

    def info(self, **data: Any) -> None:
        self.tracer.emit(self.node, "info", **data)
```

C Application 层 · 方法论（ODI / OST / Working Backwards）

backend/anker_studio/application/methodology/odi.py (78 行)

```
"""机会量化 (Application 层) : Ulwick ODI + VOC Impact Score。

- importance: 由提及广度(reach)推导 (被越多人提到 = 越重要) 。
- satisfaction: 由正面率推导。
- opportunity_score = importance + max(importance - satisfaction, 0)   (Ulwick 公式) 。
- impact_score = Reach × (Severity + Value + Strategic)                (VOC 优先级) 。
全部来自确定性 ABSA 统计, 可溯源。
"""
from __future__ import annotations

from typing import Dict, List

from anker_studio.common.models import AspectInsight, Opportunity
from anker_studio.infrastructure.nlp.absa import AspectStat
```

```python

# 战略权重：与安克技术原生强相关的 aspect 赋更高战略分（Thus 芯片/端侧 AI 可发力处）
STRATEGIC_WEIGHT: Dict[str, float] = {
    "降噪 ANC": 1.0,
    "通话质量": 1.0,
    "音质": 0.8,
    "连接稳定性/多点": 0.7,
    "续航": 0.6,
    "App 体验": 0.6,
    "佩戴舒适度": 0.5,
    "价格/价值": 0.4,
    "耐用性": 0.5,
}


def aspect_to_insight(stat: AspectStat, total_reviews: int) -> AspectInsight:
    reach = round(stat.mentions / total_reviews, 4) if total_reviews else 0.0
    # importance：reach 越高越重要，映射到 0-10（reach 0.5 -> ~9）
    importance = round(min(10.0, 2.0 + reach * 14.0), 2)
    satisfaction = round(stat.positive_rate * 10.0, 2)
    opportunity = round(importance + max(importance - satisfaction, 0.0), 2)
    severity = round(stat.negative_rate, 4)
    value = 1.0
    strategic = STRATEGIC_WEIGHT.get(stat.aspect, 0.5)
    impact = round(reach * (severity + value + strategic), 4)
    return AspectInsight(
        aspect=stat.aspect,
        mention_count=stat.mentions,
        reach=reach,
        negative_rate=severity,
        importance=importance,
        satisfaction=satisfaction,
        opportunity_score=opportunity,
        impact_score=impact,
        representative_evidence_ids=(stat.negative_evidence_ids or stat.evidence_ids)[:3],
        summary=(
            f"{stat.aspect}：被 {stat.mentions} 条评论提及（覆盖 {reach:.0%}），"
            f"负面率 {severity:.0%}，机会分 {opportunity}。"
        ),
    )


def insights_to_opportunities(insights: List[AspectInsight]) -> List[Opportunity]:
    """把高机会分的 aspect 转成结构化机会（按机会分降序）。"""
    ranked = sorted(insights, key=lambda a: a.opportunity_score, reverse=True)
    opportunities: List[Opportunity] = []
    for i, ins in enumerate(ranked):
        opportunities.append(
            Opportunity(
                id=f"OPP-{i+1}",
                statement=f"用户在「{ins.aspect}」上存在未被满足的体验差（满意度 {ins.satisfaction}/10）",
                aspect=ins.aspect,
                importance=ins.importance,
                satisfaction=ins.satisfaction,
                reach=ins.reach,
                severity=ins.negative_rate,
                opportunity_score=ins.opportunity_score,
```

```
                impact_score=ins.impact_score,
                origin="voc",
                evidence_ids=ins.representative_evidence_ids,
                rationale=ins.summary,
            )
        )
    return opportunities
```

backend/anker_studio/application/methodology/ost.py (85 行)

```python
"""机会解决方案树（Application 层）：Teresa Torres OST。

结果(outcome) → 机会(opportunity) → 方案(solution) → 实验(experiment)。
方案候选用确定性模板按 aspect 生成（结合安克技术原生能力），每个方案附实验。
"""
from __future__ import annotations

from typing import Dict, List

from anker_studio.common.models import (
    Experiment,
    Opportunity,
    OpportunityNode,
    OpportunitySolutionTree,
    SolutionNode,
)

# 按 aspect 给出"技术/模式/流程"三类候选方案模板（体现区别于常规方案）
SOLUTION_TEMPLATES: Dict[str, List[str]] = {
    "降噪 ANC": [
        "端侧自适应降噪：基于 Thus 类存算一体芯片做场景识别 + 个性化 ANC 曲线（技术原生）",
        "降噪强度随环境噪声实时无级调节，并对人声选择性透传（流程创新）",
    ],
    "通话质量": [
        "多麦克风 + 骨传导融合的端侧人声分离，强风/嘈杂环境保持清晰（技术）",
        "通话前自动环境检测并提示最佳麦克风模式（流程）",
    ],
    "连接稳定性/多点": [
        "多点连接 + 基于使用习惯的智能优先级切换（模式创新）",
        "连接异常自愈：断连自动重连并在 App 给出诊断（流程）",
    ],
    "续航": [
        "端侧 AI 按使用场景动态调度功耗，标称续航与真实续航对齐（技术）",
        "电量预测与「今天够不够用」提醒（模式）",
    ],
    "音质": [
        "听力轮廓个性化 EQ，开箱 60 秒完成校准（技术）",
        "基于内容类型(播客/音乐/游戏)自动切换音画策略（流程）",
    ],
    "佩戴舒适度": [
        "AI 入耳贴合检测 + 多尺寸耳塞推荐，降低长时间佩戴疲劳（流程）",
    ],
    "App 体验": [
        "App 内嵌 AI 助手，用自然语言调音与排障（模式）",
    ],
    "价格/价值": [
```

```
        "把高价值的端侧 AI 能力下放到中端价位，重塑性价比心智（模式）",
    ],
    "耐用性": [
        "关键部件可靠性前置验证 + 质保流程透明化（流程）",
    ],
}


def _solutions_for(aspect: str) -> List[SolutionNode]:
    nodes: List[SolutionNode] = []
    for tmpl in SOLUTION_TEMPLATES.get(aspect, [f"针对「{aspect}」的差异化方案"]):
        nodes.append(
            SolutionNode(
                statement=tmpl,
                rationale=f"直接回应「{aspect}」的高机会分痛点。",
                experiments=[
                    Experiment(
                        statement=f"对「{aspect}」改进做 A/B 偏好测试",
                        assumption="目标用户愿意为该改进支付溢价/给更高 NPS",
                        method="合成用户面板 + 真实小样本盲测",
                    )
                ],
            )
        )
    return nodes


def build_ost(outcome: str, opportunities: List[Opportunity], top_n: int = 5) -> OpportunitySolutionTree:
    nodes: List[OpportunityNode] = []
    for opp in sorted(opportunities, key=lambda o: o.opportunity_score, reverse=True)[:top_n]:
        nodes.append(
            OpportunityNode(
                opportunity_id=opp.id,
                statement=opp.statement,
                opportunity_score=opp.opportunity_score,
                solutions=_solutions_for(opp.aspect),
            )
        )
    return OpportunitySolutionTree(outcome=outcome, opportunities=nodes)
```

backend/anker_studio/application/methodology/working_backwards.py (107 行)

```
"""Working Backwards PR/FAQ 生成（Application 层）。

把产品概念 + 机会 + 可行性 + NPS 组装成 PR/FAQ。确定性产出可读文档；
可选由 LLM 网关 narrate() 润色（不引入新事实）。每个差异点回链 evidence_ids。
"""
from __future__ import annotations

from typing import List

from anker_studio.common.models import (
    CompetitorFinding,
    FaqItem,
    FeasibilityAssessment,
    NPSPrediction,
```

```
    Opportunity,
    PRFAQ,
    ProductConcept,
)


def build_prfaq(
    concept: ProductConcept,
    opportunities: List[Opportunity],
    feasibility: FeasibilityAssessment,
    nps: NPSPrediction,
    competitors: List[CompetitorFinding],
) -> PRFAQ:
    top_opps = opportunities[:3]
    addressed = "、".join(o.aspect for o in top_opps) or "核心体验"
    comp_names = "、".join(c.brand for c in competitors) or "主流竞品"

    headline = f"{concept.name}：用端侧 AI 重新定义{concept.category}的{addressed}"
    subheading = (
        f"面向{concept.target_segment}。{concept.value_proposition}"
    )
    summary = (
        f"（深圳）安克 soundcore 今日发布 {concept.name}。它针对真实用户在"
        f"{addressed}上的长期痛点，通过{('、'.join(concept.tech_enablers) or '端侧 AI 能力')}，"
        f"把过去靠经验拍脑袋才能发现的需求，变成由数据与 AI 验证过的确定性改进。"
    )
    customer_quote = (
        "「我换过好几副耳机，最大的问题终于被认真解决了——而且它真的懂我怎么用。」"
        "——来自真实评论聚类的目标用户"
    )
    maker_quote = (
        "「这不是又一次拍脑袋。每一个功能点都能追溯到一条真实用户证据和一项可行性结论。」"
        "——产品负责人"
    )
    cta = f"了解 {concept.name} 如何用 AI 原生方法从洞察走到定义。"

    external: List[FaqItem] = [
        FaqItem(
            question="它和 " + comp_names + " 有什么不同？",
            answer=(
                "竞品在部分维度领先，但在"
                + "、".join(
                    w for c in competitors for w in c.white_space
                )[:120]
                + " 等空白上仍未满足；本品正是冲着这些空白设计。"
            ),
            evidence_ids=[eid for c in competitors for eid in c.evidence_ids][:4],
        ),
        FaqItem(
            question="为什么用户会买单？",
            answer=(
                f"它直接解决了机会分最高的痛点（{addressed}），这些痛点来自真实评论统计而非主观判断。"
            ),
            evidence_ids=[eid for o in top_opps for eid in o.evidence_ids][:4],
        ),
        FaqItem(
            question="主要限制是什么？",
```

```
            answer="端侧 AI 受功耗与成本约束，首发聚焦最高价值的 2–3 个能力，其余通过 OTA 迭代。",
            evidence_ids=[],
        ),
    ]
    internal: List[FaqItem] = [
        FaqItem(
            question="技术可行性如何？",
            answer=feasibility.technical or "依托端侧 AI 芯片能力，核心能力可在端侧实时运行。",
            evidence_ids=feasibility.evidence_ids[:3],
        ),
        FaqItem(
            question="成本与供应链风险？",
            answer=(feasibility.bom_cost or "") + " " + (feasibility.supply_chain or ""),
            evidence_ids=[],
        ),
        FaqItem(
            question="成功指标？",
            answer=f"预测 NPS {nps.score:.0f}；首发关注高机会痛点的满意度提升与复购。",
            evidence_ids=nps.evidence_ids[:3],
        ),
        FaqItem(
            question="主要风险？",
            answer="; ".join(f"{r.area}:{r.description}" for r in feasibility.risks[:3]),
            evidence_ids=[],
        ),
    ]
    return PRFAQ(
        headline=headline,
        subheading=subheading,
        summary=summary,
        customer_quote=customer_quote,
        maker_quote=maker_quote,
        call_to_action=cta,
        external_faq=external,
        internal_faq=internal,
    )
```

D Application 层 · 安克三平台（JML / AMI / BEES）

backend/anker_studio/application/platforms/jml.py (33 行)

```
"""AI 原生 JML（Application 层）：用户洞察平台。

对标安克 JML（Joint Maker Lab）：把真实用户评论转化为可溯源的痛点与机会。
确定性 ABSA → ODI 机会评分 → 机会解决方案树。
"""
from __future__ import annotations

from typing import List

from anker_studio.common.models import Evidence, VocReport
from anker_studio.application.methodology.odi import aspect_to_insight, insights_to_opportunities
from anker_studio.application.methodology.ost import build_ost
from anker_studio.infrastructure.nlp.absa import analyze
```

```

def build_voc_report(category: str, target_brand: str, evidences: List[Evidence]) -> VocReport:
    total = len(evidences)
    stats = analyze(evidences)
    insights = [aspect_to_insight(s, total) for s in stats.values() if s.mentions > 0]
    insights.sort(key=lambda a: a.opportunity_score, reverse=True)
    opportunities = insights_to_opportunities(insights)
    ost = build_ost(
        outcome=f"提升 {target_brand} {category} 品类的 NPS 与复购",
        opportunities=opportunities,
    )
    return VocReport(
        category=category,
        target_brand=target_brand,
        review_count=total,
        aspects=insights,
        opportunities=opportunities,
        ost=ost,
    )
```

backend/anker_studio/application/platforms/ami.py (73 行)

```
"""AI 原生 AMI（Application 层）：市场/竞品洞察平台（超级智囊底座）。

对标安克 AMI（Anker Market Insights）：对每个竞品做 ABSA，推导其优势/短板，
短板即"白空间机会"；叠加行业趋势信号，产出 MarketIntel。
"""
from __future__ import annotations

from typing import Dict, List

from anker_studio.common.models import (
    CompetitorFinding,
    Evidence,
    MarketIntel,
    Opportunity,
    TrendSignal,
)
from anker_studio.application.methodology.odi import aspect_to_insight
from anker_studio.infrastructure.nlp.absa import analyze


def _competitor_finding(brand: str, evidences: List[Evidence]) -> CompetitorFinding:
    total = len(evidences)
    stats = analyze(evidences)
    insights = [aspect_to_insight(s, total) for s in stats.values() if s.mentions > 0]
    strengths, weaknesses, white_space, ev_ids = [], [], [], []
    for ins in sorted(insights, key=lambda a: a.reach, reverse=True):
        if ins.negative_rate >= 0.45 and ins.reach >= 0.05:
            weaknesses.append(f"{ins.aspect}（负面率 {ins.negative_rate:.0%}）")
            white_space.append(ins.aspect)
            ev_ids.extend(ins.representative_evidence_ids)
        elif ins.satisfaction >= 6.0 and ins.reach >= 0.08:
            strengths.append(f"{ins.aspect}（满意度 {ins.satisfaction}/10）")
    return CompetitorFinding(
```

```
        brand=brand,
        review_count=total,
        strengths=strengths[:4],
        weaknesses=weaknesses[:4],
        white_space=white_space[:4],
        evidence_ids=ev_ids[:6],
    )


def build_market_intel(
    competitor_evidences: Dict[str, List[Evidence]],
    trends: List[TrendSignal],
) -> MarketIntel:
    findings: List[CompetitorFinding] = []
    for brand, evs in competitor_evidences.items():
        if evs:
            findings.append(_competitor_finding(brand, evs))

    # 白空间机会：多个竞品共同的短板 = 强机会
    aspect_gap: Dict[str, List[str]] = {}
    for f in findings:
        for ws in f.white_space:
            aspect_gap.setdefault(ws, []).extend(f.evidence_ids)

    white_space_opps: List[Opportunity] = []
    for i, (aspect, ev_ids) in enumerate(
        sorted(aspect_gap.items(), key=lambda kv: len(kv[1]), reverse=True)
    ):
        white_space_opps.append(
            Opportunity(
                id=f"WS-{i+1}",
                statement=f"竞品在「{aspect}」上普遍未满足，存在差异化空白",
                aspect=aspect,
                origin="competitor",
                opportunity_score=round(6.0 + len(ev_ids) * 0.3, 2),
                evidence_ids=ev_ids[:4],
                rationale=f"{len([f for f in findings if aspect in f.white_space])} 个竞品在此 aspect 上负面突出。",
            )
        )
    return MarketIntel(competitors=findings, trends=trends, white_space_opportunities=white_space_opps)
```

backend/anker_studio/application/platforms/bees.py (113 行)

```
"""AI 原生 BEES（Application 层）：体验评估 + NPS 预测。

对标安克 BEES（Best Experience Enhancement System）：
- 按时间切片观察 aspect 负面率演化（体验是变好还是退步）。
- 综合目标品牌满意度 + 合成用户接受度，预测 NPS（核心指标）。
"""
from __future__ import annotations

from collections import defaultdict
from typing import Dict, List, Optional

from anker_studio.common.models import (
```

```
    Evidence,
    ExperiencePoint,
    NPSPrediction,
    PersonaInterview,
    VocReport,
)
from anker_studio.infrastructure.nlp.absa import analyze


def _period_of(date: Optional[str]) -> str:
    if not date:
        return "unknown"
    s = str(date)
    # 兼容 "2021-...", 时间戳毫秒等
    if len(s) >= 4 and s[:4].isdigit():
        return s[:4]
    try:
        import datetime as _dt

        ts = int(s)
        if ts > 10_000_000_000:
            ts //= 1000
        return _dt.datetime.utcfromtimestamp(ts).strftime("%Y")
    except (ValueError, OverflowError, OSError):
        return "unknown"


def experience_trend(evidences: List[Evidence], focus_aspects: List[str]) -> List[ExperiencePoint]:
    by_period: Dict[str, List[Evidence]] = defaultdict(list)
    for ev in evidences:
        by_period[_period_of(ev.date)].append(ev)

    points: List[ExperiencePoint] = []
    for period in sorted(by_period):
        if period == "unknown":
            continue
        stats = analyze(by_period[period])
        for aspect in focus_aspects:
            st = stats.get(aspect)
            if st and st.mentions > 0:
                points.append(
                    ExperiencePoint(
                        aspect=aspect,
                        period=period,
                        mention_count=st.mentions,
                        negative_rate=st.negative_rate,
                        evidence_ids=st.negative_evidence_ids[:2],
                    )
                )
    return points


def predict_nps(
    voc: VocReport,
    interviews: List[PersonaInterview],
    concept_addresses: List[str],
) -> NPSPrediction:
```

```
    """简化但可解释的 NPS 预测:

    base 来自目标品牌当前满意度; 提案若命中高机会痛点则加分;
    合成用户接受度作为修正项。
    """
    # 当前满意度 (加权平均, 权重为 reach)
    weighted, wsum = 0.0, 0.0
    for a in voc.aspects:
        weighted += a.satisfaction * max(a.reach, 0.001)
        wsum += max(a.reach, 0.001)
    base_sat = (weighted / wsum) if wsum else 5.0  # 0-10

    # 命中高机会痛点的覆盖加成
    high_opps = {o.aspect for o in voc.opportunities[:5]}
    hit = len([a for a in concept_addresses if a in high_opps])
    coverage_bonus = min(hit, 3) * 6.0  # 每命中一个 +6 分 NPS

    # 合成用户接受度
    if interviews:
        acceptance = sum(i.acceptance for i in interviews) / len(interviews)
    else:
        acceptance = 0.5

    # 映射到 NPS: 满意度->基线, acceptance 修正
    base_nps = (base_sat - 7.0) * 20.0  # sat 7 -> 0, sat 9 -> +40, sat 5 -> -40
    accept_adj = (acceptance - 0.5) * 60.0
    score = max(-100.0, min(100.0, base_nps + coverage_bonus + accept_adj))

    promoters = max(0.0, min(100.0, 50 + score / 2))
    detractors = max(0.0, min(100.0, 50 - score / 2))
    passives = max(0.0, 100 - promoters - detractors)

    ev_ids = [eid for o in voc.opportunities[:3] for eid in o.evidence_ids][:4]
    return NPSPrediction(
        score=round(score, 1),
        promoters=round(promoters, 1),
        passives=round(passives, 1),
        detractors=round(detractors, 1),
        rationale=(
            f"当前满意度基线 {base_sat:.1f}/10; 提案命中 {hit} 个高机会痛点 (+{coverage_bonus:.0f}) ; "
            f"合成用户平均接受度 {acceptance:.0%}。"
        ),
        evidence_ids=ev_ids,
    )
```

E Application 层 · 五个 Agent

backend/anker_studio/application/agents/base.py (39 行)

```
"""Agent 基类 (Application 层) 。

每个 Agent 通过注入的 LLMGateway / RagService 访问外部能力, 禁止自行 import provider。
产出的论断都应带 evidence_ids。
"""
```

```python
from __future__ import annotations

from typing import List, Optional

from anker_studio.application.graph.state import PipelineState
from anker_studio.infrastructure.llm.gateway import LLMGateway
from anker_studio.infrastructure.observability.trace import Tracer


class Agent:
    name: str = "agent"
    role: str = ""

    def __init__(
        self,
        gateway: LLMGateway,
        rag=None,
        tracer: Optional[Tracer] = None,
    ):
        self.gw = gateway
        self.rag = rag
        self.tracer = tracer

    def run(self, state: PipelineState) -> PipelineState:  # pragma: no cover - 抽象
        raise NotImplementedError

    @staticmethod
    def dedup(items: List[str]) -> List[str]:
        seen, out = set(), []
        for x in items:
            if x and x not in seen:
                seen.add(x)
                out.append(x)
        return out
```

backend/anker_studio/application/agents/super_think_tank.py (56 行)

```python
"""超级智囊 Agent (AI 原生 AMI)：竞品拆解 + 趋势洞察。"""
from __future__ import annotations

from anker_studio.application.agents.base import Agent
from anker_studio.application.graph.state import PipelineState
from anker_studio.application.platforms.ami import build_market_intel
from anker_studio.infrastructure.data.connectors import fetch_trends
from anker_studio.infrastructure.nlp.lexicons import ASPECT_LEXICON


def _aspect_query(aspect: str) -> str:
    """把中文 aspect 名映射为英文检索词（语料为英文评论）。"""
    kws = [k for k in ASPECT_LEXICON.get(aspect, []) if k.isascii()]
    return " ".join(kws[:4]) or aspect


class SuperThinkTankAgent(Agent):
    name = "超级智囊"
    role = "行业研究 / 竞品分析 / 趋势洞察"
```

```

    def run(self, state: PipelineState) -> PipelineState:
        trends, trend_ev = fetch_trends(
            keywords=["earbuds anc", "on-device ai audio", "hearing personalization"]
        )
        state.trend_evidences.extend(trend_ev)

        intel = build_market_intel(state.competitor_evidences, trends)

        # Agentic RAG: 为每个白空间机会检索补充证据（可溯源），并累计检索迭代数
        if self.rag is not None:
            for opp in intel.white_space_opportunities:
                evs, meta = self.rag.agentic_search(_aspect_query(opp.aspect), top_k=5, min_hits=2)
                state.retrieval_iters += int(meta.get("iterations", 1))
                extra_ids = [e.source_id for e in evs][:3]
                opp.evidence_ids = self.dedup(list(opp.evidence_ids) + extra_ids)

        state.market_intel = intel

        # 记录可溯源论断（供评测）
        for f in intel.competitors:
            for w in f.weaknesses:
                state.add_claim(f"{f.brand} 的短板：{w}", f.evidence_ids)
        for opp in intel.white_space_opportunities:
            state.add_claim(opp.statement, opp.evidence_ids)
        for t in trends:
            state.add_claim(f"趋势：{t.name} — {t.summary}", t.evidence_ids)

        if self.tracer:
            self.tracer.emit(
                self.name,
                "result",
                competitors=len(intel.competitors),
                white_space=len(intel.white_space_opportunities),
                trends=len(trends),
            )
        return state
```

backend/anker_studio/application/agents/product_manager.py (132 行)

```
"""产品经理 Agent（编排）：双路径概念生成 + 迭代收敛。

- 路径 (VOC 驱动)：从机会分最高的真实痛点反推方案。
- 路径 (第一性原理 / 技术原生)：从能力跃迁（端侧 AI / Thus 芯片）正推新体验。
迭代时吸收用户替身的 must_fixes 与行业专家的风险，对方案做修订。
"""
from __future__ import annotations

from typing import List

from anker_studio.application.agents.base import Agent
from anker_studio.application.graph.state import PipelineState
from anker_studio.common.models import Differentiator, ProductConcept

# 安克技术原生使能器（来自公司调研：Thus 芯片 / eufy Edge / 端侧 AI）
TECH_ENABLERS = [
```

```
    "Thus 存算一体(CIM)端侧 AI 芯片（算力较上代 +150x）",
    "多麦克风 + 骨传导端侧人声分离",
    "端侧听力轮廓个性化",
]


class ProductManagerAgent(Agent):
    name = "产品经理"
    role = "需求拆解 / 概念生成 / 编排收敛"

    def run(self, state: PipelineState) -> PipelineState:
        state.pm_iteration += 1
        voc = state.voc_report
        intel = state.market_intel
        if voc is None:
            return state

        top_opps = voc.opportunities[:3]
        white_space = intel.white_space_opportunities if intel else []

        # 首轮：生成两条路径的概念候选；后续轮：在 chosen 上修订
        if state.chosen_concept is None:
            voc_concept = self._voc_driven(state, top_opps)
            tech_concept = self._tech_native(state, top_opps)
            state.concepts = [voc_concept, tech_concept]
            # 选择：优先覆盖最高机会 + 含技术原生差异点的融合概念
            state.chosen_concept = self._fuse(state, voc_concept, tech_concept, white_space)
        else:
            state.chosen_concept = self._revise(state, state.chosen_concept)

        c = state.chosen_concept
        for d in c.differentiators:
            state.add_claim(f"差异点：{d.statement}", d.evidence_ids)

        if self.tracer:
            self.tracer.emit(
                self.name, "result",
                iteration=state.pm_iteration,
                concept=c.name,
                features=len(c.key_features),
            )
        return state

    def _voc_driven(self, state: PipelineState, opps) -> ProductConcept:
        feats = [f"针对「{o.aspect}」的体验改进" for o in opps]
        diffs = [
            Differentiator(statement=f"以数据验证的方式解决「{o.aspect}」", evidence_ids=o.evidence_ids)
            for o in opps
        ]
        return ProductConcept(
            id="C-VOC",
            name="soundcore（VOC 驱动概念）",
            category=state.category,
            path="voc_driven",
            one_liner="把用户反复抱怨却没被解决的痛点，一次性做到位。",
            target_segment="对音频体验敏感的高频通勤/通话用户",
            value_proposition="基于真实评论统计，集中解决机会分最高的 2–3 个痛点。",
```

```
            key_features=feats,
            differentiators=diffs,
            tech_enablers=[TECH_ENABLERS[0]],
            addressed_opportunity_ids=[o.id for o in opps],
        )

    def _tech_native(self, state: PipelineState, opps) -> ProductConcept:
        trend_names = [t.name for t in (state.market_intel.trends if state.market_intel else [])]
        return ProductConcept(
            id="C-TECH",
            name="soundcore (技术原生概念) ",
            category=state.category,
            path="tech_native",
            one_liner="用端侧 AI 创造过去做不到的新体验。",
            target_segment="尝鲜的科技爱好者 + 有听力个性化需求的用户",
            value_proposition="把云端能力下沉到耳机本体：实时、私密、个性化。",
            key_features=[f"端侧 AI：{t}" for t in trend_names[:3]],
            differentiators=[
                Differentiator(
                    statement="端侧实时运行降噪/听力个性化，不依赖云、低延迟、保护隐私",
                    evidence_ids=[t.evidence_ids[0] for t in (state.market_intel.trends if state.
 market_intel else []) if t.evidence_ids][:3],
                )
            ],
            tech_enablers=TECH_ENABLERS,
            addressed_opportunity_ids=[o.id for o in opps[:1]],
        )

    def _fuse(self, state, voc_c: ProductConcept, tech_c: ProductConcept, white_space) -> ProductConcept:
        diffs = voc_c.differentiators + tech_c.differentiators
        for ws in white_space[:2]:
            diffs.append(Differentiator(statement=f"补齐竞品空白：{ws.aspect}", evidence_ids=ws.
 evidence_ids))
        feats = self.dedup(voc_c.key_features + tech_c.key_features)
        return ProductConcept(
            id="C-FUSED",
            name="soundcore Liberty AI (融合概念) ",
            category=state.category,
            path="voc_driven",
            one_liner="既解决真实痛点，又用端侧 AI 拉开代差。",
            target_segment=voc_c.target_segment,
            value_proposition="VOC 验证的确定性改进 + 技术原生的差异化体验，双轮驱动。",
            key_features=feats,
            differentiators=diffs,
            tech_enablers=TECH_ENABLERS,
            addressed_opportunity_ids=self.dedup(
                voc_c.addressed_opportunity_ids + tech_c.addressed_opportunity_ids
                + [ws.id for ws in white_space[:2]]
            ),
        )

    def _revise(self, state: PipelineState, concept: ProductConcept) -> ProductConcept:
        must_fixes: List[str] = self.dedup(
            [mf for itv in state.interviews for mf in itv.must_fixes]
        )
        risks = [r.description for r in (state.feasibility.risks if state.feasibility else [])]
        added = [f"按用户反馈补强：{mf}" for mf in must_fixes[:2]]
```

```
        added += [f"针对风险做收敛：{r}" for r in risks[:1]]
        concept.key_features = self.dedup(concept.key_features + added)
        concept.one_liner += "（已根据合成用户与可行性反馈迭代）"
        return concept
```

backend/anker_studio/application/agents/user_proxy.py (138 行)

```
"""用户替身 Agent（合成用户面板）。

把真实评论按"主诉 aspect"聚成细分人群，构建 OCEAN 画像的合成用户，
对当前概念做"假设盲"访谈（不告知研究目标，避免谄媚），输出接受度/异议/必须改进项。
依据：SCOPE 社会心理画像、Grounded Simulation 假设盲、Synthetic Users 多人格。
"""
from __future__ import annotations

import hashlib
from typing import List

from anker_studio.application.agents.base import Agent
from anker_studio.application.graph.state import PipelineState
from anker_studio.common.models import (
    InterviewTurn,
    Persona,
    PersonaInterview,
    ProductConcept,
)

_OCEAN = ["openness", "conscientiousness", "extraversion", "agreeableness", "neuroticism"]


def _ocean_from(seed: str) -> dict:
    h = hashlib.sha256(seed.encode("utf-8")).digest()
    return {dim: round((h[i] % 100) / 100.0, 2) for i, dim in enumerate(_OCEAN)}


class UserProxyAgent(Agent):
    name = "用户替身"
    role = "合成用户访谈 / 痛点验证"

    def run(self, state: PipelineState) -> PipelineState:
        voc = state.voc_report
        concept = state.chosen_concept
        if voc is None or concept is None:
            return state

        state.personas = self._build_personas(state)
        state.interviews = [self._interview(p, concept) for p in state.personas]

        for itv in state.interviews:
            # persona 来自真实评论 grounding -> 记录引用
            persona = next((p for p in state.personas if p.id == itv.persona_id), None)
            if persona:
                state.add_claim(
                    f"{persona.segment} 对概念的判定：{itv.verdict}", persona.derived_from_evidence_ids
                )
        if self.tracer:
```

```
            self.tracer.emit(
                self.name, "result",
                personas=len(state.personas),
                avg_acceptance=round(
                    sum(i.acceptance for i in state.interviews) / max(1, len(state.interviews)), 2
                ),
            )
        return state

    def _build_personas(self, state: PipelineState) -> List[Persona]:
        personas: List[Persona] = []
        # 每个高机会痛点对应一个细分人群 (grounded 在该 aspect 的真实负面评论)
        for i, opp in enumerate(state.voc_report.opportunities[:4]):
            seed = f"{opp.aspect}-{i}"
            personas.append(
                Persona(
                    id=f"P-{i+1}",
                    name=f"用户{i+1}",
                    segment=f"对「{opp.aspect}」高度敏感的用户",
                    demographics="18-40 岁，全球主流市场，重度耳机使用者",
                    ocean=_ocean_from(seed),
                    behaviors=["每天佩戴 >3 小时", "频繁多设备切换"],
                    pains=[opp.statement],
                    derived_from_evidence_ids=opp.evidence_ids,
                    summary=f"该人群最在意 {opp.aspect}，当前满意度 {opp.satisfaction}/10。",
                )
            )
        return personas

    def _interview(self, persona: Persona, concept: ProductConcept) -> PersonaInterview:
        # 假设盲：仅向 persona 暴露其自身画像 + 概念要点，不暴露"我们想验证什么"
        addressed = set(self._concept_aspects(concept))
        persona_aspect = persona.segment

        # 该 persona 关心的 aspect 是否被概念覆盖
        hit = any(asp in persona_aspect for asp in addressed) or any(
            asp in f for asp in addressed for f in concept.key_features
        )
        covered = [f for f in concept.key_features if any(a in f for a in addressed)]

        turns: List[InterviewTurn] = [
            InterviewTurn(
                question="你平时用耳机，最大的困扰是什么？",
                answer=persona.pains[0] if persona.pains else "整体还行，但偶有小毛病。",
                sentiment="negative",
            ),
            InterviewTurn(
                question=f"这是一款新耳机，主打：{('; '.join(concept.key_features[:3]))}。你的第一反应？",
                answer=(
                    "正好戳中我天天遇到的问题，挺心动。" if hit
                    else "看起来不错，但没解决我最在意的那个点。"
                ),
                sentiment="positive" if hit else "neutral",
            ),
            InterviewTurn(
                question="你愿意为它买单/推荐给朋友吗？",
                answer=("会，如果价格合理。" if hit else "再看看，目前不够打动我。"),
```

```
                sentiment="positive" if hit else "negative",
            ),
            InterviewTurn(
                question="还有什么是它必须改进的？",
                answer=("把承诺的体验在真实环境里做稳定。" if hit else f"先把「{persona_aspect}」做好。"),
                sentiment="neutral",
            ),
        ]
        acceptance = 0.8 if hit else 0.35
        verdict = "would_buy" if acceptance >= 0.6 else ("maybe" if acceptance >= 0.45 else "would_not_buy")
        objections = [] if hit else [f"未覆盖我最在意的「{persona_aspect}」"]
        must_fixes = ["真实环境下的稳定性"] if hit else [persona.segment.replace("对「", "").replace("」高度敏感的用户", "")]
        return PersonaInterview(
            persona_id=persona.id,
            persona_name=persona.name,
            segment=persona.segment,
            transcript=turns,
            verdict=verdict,
            objections=objections,
            must_fixes=must_fixes,
            acceptance=acceptance,
        )

    @staticmethod
    def _concept_aspects(concept: ProductConcept) -> List[str]:
        # 从特性文本里粗取 aspect 关键字
        text = " ".join(concept.key_features + [d.statement for d in concept.differentiators])
        aspects = []
        for a in ["降噪 ANC", "音质", "佩戴舒适度", "通话质量", "续航", "App 体验", "连接稳定性/多点", "价格/价值", "耐用性"]:
            if a in text or a.split(" ")[0] in text:
                aspects.append(a)
        return aspects
```

backend/anker_studio/application/agents/industry_expert.py (88 行)

```
"""行业专家 Agent：技术/供应链/BOM/合规可行性 + Reflexion 批判。"""
from __future__ import annotations

from typing import List

from anker_studio.application.agents.base import Agent
from anker_studio.application.graph.state import PipelineState
from anker_studio.common.models import FeasibilityAssessment, ProductConcept, RiskItem


class IndustryExpertAgent(Agent):
    name = "行业专家"
    role = "可行性评估 / 规格定义 / Reflexion 批判"

    def run(self, state: PipelineState) -> PipelineState:
        concept = state.chosen_concept
        if concept is None:
            return state
```

```
19
20          risks = self._assess_risks(concept)
21          overall = self._overall(risks)
22          ev_ids = [t.evidence_ids[0] for t in (state.market_intel.trends if state.market_intel else []) if
        t.evidence_ids][:2]
23
24          assessment = FeasibilityAssessment(
25              technical=(
26                  "核心降噪/听力个性化可在 Thus 类端侧 AI 芯片上实时运行；"
27                  "实时翻译类重负载建议端云协同，首发可裁剪。"
28              ),
29              supply_chain="端侧 AI 芯片与多麦克风方案需锁定产能；建议双供应商。",
30              bom_cost="端侧 AI 模组抬高 BOM，首发聚焦 2–3 个高价值能力以控成本。",
31              compliance="助听类功能在部分市场受医疗器械法规约束，需分区合规。",
32              overall=overall,
33              risks=risks,
34              evidence_ids=ev_ids,
35          )
36          state.feasibility = assessment
37
38          for r in risks:
39              state.add_claim(f"可行性风险（{r.area}）：{r.description}", ev_ids)
40          if self.tracer:
41              self.tracer.emit(self.name, "result", overall=overall, risks=len(risks))
42          return state
43
44      def _assess_risks(self, concept: ProductConcept) -> List[RiskItem]:
45          risks: List[RiskItem] = [
46              RiskItem(
47                  area="bom_cost",
48                  description="端侧 AI 芯片+多麦克风抬高物料成本，可能挤压中端价位毛利。",
49                  severity="high",
50                  mitigation="首发只上最高价值能力，其余 OTA；规模化后降本。",
51              ),
52              RiskItem(
53                  area="technical",
54                  description="端侧实时性能/功耗与体验承诺之间存在权衡。",
55                  severity="medium",
56                  mitigation="按场景动态调度算力，设保守标称值。",
57              ),
58          ]
59          text = " ".join(concept.key_features).lower()
60          if "翻译" in text or "translate" in text:
61              risks.append(
62                  RiskItem(
63                      area="technical",
64                      description="实时翻译重负载难全端侧，体验受限。",
65                      severity="high",
66                      mitigation="端云协同；首发降级为离线常用语种。",
67                  )
68              )
69          diff_text = " ".join(d.statement for d in concept.differentiators)
70          if any("听力" in f for f in concept.key_features) or "听力" in diff_text:
71              risks.append(
72                  RiskItem(
73                      area="compliance",
74                      description="听力个性化/助听在欧美受医疗法规约束。",
```

```
                    severity="medium",
                    mitigation="分区合规，必要时定位为辅助而非医疗。",
                )
            )
        return risks

    @staticmethod
    def _overall(risks: List[RiskItem]) -> str:
        highs = sum(1 for r in risks if r.severity == "high")
        if highs >= 2:
            return "red"
        if highs == 1:
            return "yellow"
        return "green"
```

backend/anker_studio/application/agents/decision_officer.py (80 行)

```
"""决策官 Agent：NPS 预测 + GO / NO-GO 决策 + DecisionRecord。"""
from __future__ import annotations

from typing import List

from anker_studio.application.agents.base import Agent
from anker_studio.application.graph.state import PipelineState
from anker_studio.application.platforms.bees import predict_nps
from anker_studio.common.models import DecisionRecord, DecisionVerdict


class DecisionOfficerAgent(Agent):
    name = "决策官"
    role = "NPS 预测 / GO-NO-GO / 审计记录"

    def run(self, state: PipelineState) -> PipelineState:
        voc = state.voc_report
        concept = state.chosen_concept
        if voc is None or concept is None:
            return state

        addressed_aspects = self._addressed_aspects(state)
        nps = predict_nps(voc, state.interviews, addressed_aspects)
        state.nps = nps

        feas = state.feasibility
        avg_acc = (
            sum(i.acceptance for i in state.interviews) / len(state.interviews)
            if state.interviews else 0.5
        )

        verdict, conditions, confidence = self._decide(nps.score, feas, avg_acc)

        record = DecisionRecord(
            verdict=verdict,
            confidence=round(confidence, 2),
            nps_prediction=nps.score,
            rationale=(
                f"NPS 预测 {nps.score:.0f}；可行性 {feas.overall if feas else 'n/a'}；"
```

```
                f"合成用户平均接受度 {avg_acc:.0%}。"
            ),
            conditions=conditions,
            evidence_ids=nps.evidence_ids,
            reviewer="ai",
        )
        state.decision = record
        state.add_claim(f"决策: {verdict.value}, 预测 NPS {nps.score:.0f}", nps.evidence_ids)

        if self.tracer:
            self.tracer.emit(
                self.name, "result",
                verdict=verdict.value, nps=nps.score, confidence=record.confidence,
            )
        return state

    @staticmethod
    def _addressed_aspects(state: PipelineState) -> List[str]:
        aspects = []
        opp_by_id = {o.id: o for o in (state.voc_report.opportunities if state.voc_report else [])}
        for oid in (state.chosen_concept.addressed_opportunity_ids if state.chosen_concept else []):
            o = opp_by_id.get(oid)
            if o:
                aspects.append(o.aspect)
        return aspects

    @staticmethod
    def _decide(nps: float, feas, avg_acc: float):
        conditions: List[str] = []
        overall = feas.overall if feas else "yellow"
        confidence = 0.5 + min(0.3, avg_acc * 0.3) + (0.1 if nps > 0 else -0.1)

        if overall == "red":
            conditions = [r.mitigation for r in (feas.risks if feas else []) if r.severity == "high"]
            return DecisionVerdict.CONDITIONAL_GO, conditions, confidence - 0.1
        if nps >= 20 and avg_acc >= 0.6 and overall in {"green", "yellow"}:
            return DecisionVerdict.GO, conditions, confidence + 0.1
        if nps <= -10 or avg_acc < 0.4:
            return DecisionVerdict.NO_GO, ["机会命中不足, 需重做概念"], confidence
        conditions = [r.mitigation for r in (feas.risks if feas else []) if r.severity in {"high", "medium"}][:2]
        return DecisionVerdict.CONDITIONAL_GO, conditions, confidence
```

F Application 层 · 编排 / 评测 / 对照 / 报告

backend/anker_studio/application/graph/state.py (74 行)

```
"""流水线共享状态 (Application 层)。

贯穿整个 AI 原生工作流的状态对象。节点读取并增量更新它。
`claims` 收集 (论断, evidence_ids) 供评测计算 groundedness/faithfulness。
"""
from __future__ import annotations

```

```python
from typing import Dict, List, Optional, Tuple

from pydantic import BaseModel, Field

from anker_studio.common.models import (
    DecisionRecord,
    Evidence,
    ExperiencePoint,
    FeasibilityAssessment,
    MarketIntel,
    NPSPrediction,
    Persona,
    PersonaInterview,
    PRFAQ,
    ProductConcept,
    ProductProposal,
    VocReport,
)


class PipelineState(BaseModel):
    # 输入
    category: str = "audio"
    target_brand: str = "soundcore"
    brief: str = ""

    # 数据
    target_evidences: List[Evidence] = Field(default_factory=list)
    competitor_evidences: Dict[str, List[Evidence]] = Field(default_factory=dict)
    trend_evidences: List[Evidence] = Field(default_factory=list)

    # 平台产物
    voc_report: Optional[VocReport] = None
    market_intel: Optional[MarketIntel] = None
    experience_trend: List[ExperiencePoint] = Field(default_factory=list)

    # 概念与验证
    concepts: List[ProductConcept] = Field(default_factory=list)
    chosen_concept: Optional[ProductConcept] = None
    personas: List[Persona] = Field(default_factory=list)
    interviews: List[PersonaInterview] = Field(default_factory=list)
    feasibility: Optional[FeasibilityAssessment] = None
    nps: Optional[NPSPrediction] = None

    # 决策与产出
    prfaq: Optional[PRFAQ] = None
    decision: Optional[DecisionRecord] = None
    proposal: Optional[ProductProposal] = None

    # 控制 / 评测
    pm_iteration: int = 0
    max_pm_iterations: int = 2
    retrieval_iters: int = 0
    claims: List[Tuple[str, List[str]]] = Field(default_factory=list)
    awaiting_human: bool = False

    def add_claim(self, text: str, evidence_ids: List[str]) -> None:
```

```
        self.claims.append((text, list(evidence_ids or [])))

    def all_evidences(self) -> List[Evidence]:
        evs: List[Evidence] = list(self.target_evidences) + list(self.trend_evidences)
        for lst in self.competitor_evidences.values():
            evs.extend(lst)
        return evs

    class Config:
        arbitrary_types_allowed = True
```

backend/anker_studio/application/graph/engine.py (114 行)

```
"""StateGraph 编排引擎（Application 层）。

语义对齐 LangGraph（节点 / 条件边 / checkpoint / interrupt_before），但为保持依赖
极轻、离线可复现而自研最小实现。借鉴 Harness 工程："错误即信息"（节点异常被记录后
按策略路由，而非直接崩溃）。
"""
from __future__ import annotations

from typing import Callable, Dict, List, Optional, Set

from anker_studio.application.graph.checkpoint import JsonCheckpointer
from anker_studio.application.graph.state import PipelineState
from anker_studio.common.logging import log
from anker_studio.infrastructure.observability.trace import Tracer

END = "__end__"
NodeFn = Callable[[PipelineState], PipelineState]
RouterFn = Callable[[PipelineState], str]


class StateGraph:
    def __init__(self, tracer: Optional[Tracer] = None, hitl: bool = False):
        self.nodes: Dict[str, NodeFn] = {}
        self.static_edges: Dict[str, str] = {}
        self.conditional: Dict[str, RouterFn] = {}
        self.entry: Optional[str] = None
        self.interrupt_before: Set[str] = set()
        self.tracer = tracer
        self.hitl = hitl

    def add_node(self, name: str, fn: NodeFn) -> "StateGraph":
        self.nodes[name] = fn
        return self

    def set_entry(self, name: str) -> "StateGraph":
        self.entry = name
        return self

    def add_edge(self, src: str, dst: str) -> "StateGraph":
        self.static_edges[src] = dst
        return self

    def add_conditional(self, src: str, router: RouterFn) -> "StateGraph":
```

```
        self.conditional[src] = router
        return self

    def set_interrupt_before(self, names: List[str]) -> "StateGraph":
        self.interrupt_before = set(names)
        return self

    def _next(self, node: str, state: PipelineState) -> str:
        if node in self.conditional:
            return self.conditional[node](state)
        return self.static_edges.get(node, END)

    def run(
        self,
        state: PipelineState,
        checkpointer: Optional[JsonCheckpointer] = None,
        thread_id: str = "default",
        resume: bool = False,
    ) -> PipelineState:
        if resume and checkpointer is not None:
            loaded = checkpointer.load(thread_id)
            if loaded is None:
                raise RuntimeError(f"无法恢复：找不到 checkpoint thread_id={thread_id}")
            state, current = loaded
            state.awaiting_human = False
            allow_interrupt = False  # 恢复后跳过这一次中断
        else:
            current = self.entry or END
            allow_interrupt = True

        steps = 0
        while current != END:
            steps += 1
            if steps > 100:
                log.bind(node="graph").error("步数超限，强制结束（防死循环）。")
                break

            if allow_interrupt and self.hitl and current in self.interrupt_before:
                if checkpointer is not None:
                    checkpointer.save(thread_id, state, current)
                state.awaiting_human = True
                if self.tracer:
                    self.tracer.emit(current, "interrupt", message="等待人工确认（HITL）")
                log.bind(node="graph").warning(f"在 '{current}' 前暂停，等待人工确认。")
                return state
            allow_interrupt = True

            fn = self.nodes.get(current)
            if fn is None:
                log.bind(node="graph").error(f"未知节点 '{current}'，结束。")
                break

            span_ctx = self.tracer.node_span(current) if self.tracer else _NullSpan()
            with span_ctx:
                try:
                    state = fn(state)
                except Exception as exc:  # noqa: BLE001 - 错误即信息
```

```
                    log.bind(node=current).error(f"节点异常: {exc}")
                    if self.tracer:
                        self.tracer.emit(current, "node_error", error=str(exc))
                    # 节点失败: 记录后按静态边继续 (降级), 不崩溃
            current = self._next(current, state)
        return state


class _NullSpan:
    def __enter__(self):
        return self

    def __exit__(self, *a):
        return False
```

backend/anker_studio/application/graph/checkpoint.py (34 行)

```python
"""Checkpointer (Application 层): JSON 持久化, 支持长流程续跑 / HITL 恢复。

语义对齐 LangGraph 的 checkpointer: 按 thread_id 保存 (state, next_node)。
"""
from __future__ import annotations

import json
from pathlib import Path
from typing import Optional, Tuple

from anker_studio.common.config import settings
from anker_studio.application.graph.state import PipelineState


class JsonCheckpointer:
    def __init__(self, directory: Optional[str] = None):
        self.dir = Path(directory) if directory else (settings().trace_path / "checkpoints")
        self.dir.mkdir(parents=True, exist_ok=True)

    def _path(self, thread_id: str) -> Path:
        return self.dir / f"{thread_id}.json"

    def save(self, thread_id: str, state: PipelineState, next_node: str) -> None:
        payload = {"next_node": next_node, "state": state.model_dump(mode="json")}
        self._path(thread_id).write_text(
            json.dumps(payload, ensure_ascii=False), encoding="utf-8"
        )

    def load(self, thread_id: str) -> Optional[Tuple[PipelineState, str]]:
        p = self._path(thread_id)
        if not p.exists():
            return None
        payload = json.loads(p.read_text(encoding="utf-8"))
        return PipelineState.model_validate(payload["state"]), payload["next_node"]
```

backend/anker_studio/application/graph/pipeline.py (110 行)

```python
"""AI 原生工作流的图装配（Application 层）。

节点顺序:
  voc → think_tank → pm → users → expert → decision →(条件)→ output / 回到 pm
决策闸前设 interrupt_before（HITL）。
"""
from __future__ import annotations

from typing import Optional

from anker_studio.application.agents.decision_officer import DecisionOfficerAgent
from anker_studio.application.agents.industry_expert import IndustryExpertAgent
from anker_studio.application.agents.product_manager import ProductManagerAgent
from anker_studio.application.agents.super_think_tank import SuperThinkTankAgent
from anker_studio.application.agents.user_proxy import UserProxyAgent
from anker_studio.application.graph.engine import END, StateGraph
from anker_studio.application.graph.state import PipelineState
from anker_studio.application.methodology.working_backwards import build_prfaq
from anker_studio.application.platforms.bees import experience_trend
from anker_studio.application.platforms.jml import build_voc_report
from anker_studio.common.models import DecisionVerdict, ProductProposal
from anker_studio.infrastructure.assets.media import generate_concept_image, synthesize_narration
from anker_studio.infrastructure.llm.gateway import LLMGateway
from anker_studio.infrastructure.observability.trace import Tracer


def build_studio_graph(
    gateway: LLMGateway,
    rag,
    tracer: Tracer,
    hitl: bool = False,
) -> StateGraph:
    think_tank = SuperThinkTankAgent(gateway, rag, tracer)
    pm = ProductManagerAgent(gateway, rag, tracer)
    users = UserProxyAgent(gateway, rag, tracer)
    expert = IndustryExpertAgent(gateway, rag, tracer)
    decision = DecisionOfficerAgent(gateway, rag, tracer)

    def voc_node(state: PipelineState) -> PipelineState:
        report = build_voc_report(state.category, state.target_brand, state.target_evidences)
        state.voc_report = report
        focus = [o.aspect for o in report.opportunities[:3]]
        state.experience_trend = experience_trend(state.target_evidences, focus)
        for o in report.opportunities[:5]:
            state.add_claim(o.statement, o.evidence_ids)
        return state

    def output_node(state: PipelineState) -> PipelineState:
        concept = state.chosen_concept
        voc = state.voc_report
        if concept is None or voc is None or state.decision is None:
            return state
        prfaq = build_prfaq(
            concept=concept,
            opportunities=voc.opportunities,
            feasibility=state.feasibility,
            nps=state.nps,
```

```
            competitors=state.market_intel.competitors if state.market_intel else [],
        )
        # 可选 LLM 润色（不引入新事实）
        prfaq.summary = gateway.narrate(
            prfaq.summary, "润色产品新闻稿摘要，专业简洁", context=concept.value_proposition
        )
        state.prfaq = prfaq

        addressed = [o for o in voc.opportunities if o.id in concept.addressed_opportunity_ids]
        image_path = generate_concept_image(
            f"Premium {state.category} product concept: {concept.name}, {concept.one_liner}, "
            f"clean studio render, soundcore brand aesthetic"
        )
        narration = synthesize_narration(prfaq.summary)
        state.proposal = ProductProposal(
            concept=concept,
            prfaq=prfaq,
            feasibility=state.feasibility,
            nps=state.nps,
            addressed_opportunities=addressed,
            decision=state.decision,
            concept_image_path=image_path,
            narration_audio_path=narration,
        )
        return state

    def route_after_decision(state: PipelineState) -> str:
        d = state.decision
        if d is None:
            return "output"
        if d.verdict == DecisionVerdict.GO or state.pm_iteration >= state.max_pm_iterations:
            return "output"
        return "pm"  # 未达 GO 且仍有迭代预算 -> 回到产品经理修订

    g = StateGraph(tracer=tracer, hitl=hitl)
    g.add_node("voc", voc_node)
    g.add_node("think_tank", think_tank.run)
    g.add_node("pm", pm.run)
    g.add_node("users", users.run)
    g.add_node("expert", expert.run)
    g.add_node("decision", decision.run)
    g.add_node("output", output_node)

    g.set_entry("voc")
    g.add_edge("voc", "think_tank")
    g.add_edge("think_tank", "pm")
    g.add_edge("pm", "users")
    g.add_edge("users", "expert")
    g.add_edge("expert", "decision")
    g.add_conditional("decision", route_after_decision)
    g.add_edge("output", END)
    g.set_interrupt_before(["decision"])
    return g
```

backend/anker_studio/application/baseline/experience_driven.py (50 行)

```python
"""经验驱动对照组 (Application 层) —— 对比实验 A 组。

模拟传统 PM "拍脑袋": 不接数据管线, 仅凭一次性直觉从 brief 产出概念。
用于和 AI 原生工作流 (B 组) 做可量化对比, 回答命题"本质不同"。
"""
from __future__ import annotations

import time
from typing import List

from pydantic import BaseModel, Field

from anker_studio.common.models import ProductConcept
from anker_studio.infrastructure.llm.gateway import LLMGateway

# 行业"常识/直觉"里最常被拍的几个点 (无数据支撑, 可能与真实高机会痛点错位)
GUT_PAINS = ["音质", "续航", "价格/价值"]


class BaselineResult(BaseModel):
    concept: ProductConcept
    assumed_pains: List[str] = Field(default_factory=list)
    elapsed_seconds: float = 0.0
    citations: int = 0
    validated_assumptions: int = 0


def run_experience_driven(category: str, brief: str, gateway: LLMGateway) -> BaselineResult:
    start = time.time()
    concept = ProductConcept(
        id="A-GUT",
        name="soundcore (经验驱动概念) ",
        category=category,
        path="voc_driven",
        one_liner="凭经验判断: 做更好的音质、更长续航、更高性价比。",
        target_segment="大众消费者",
        value_proposition="在大家都关心的常规维度上做到更好。",
        key_features=[f"提升「{p}」" for p in GUT_PAINS],
        differentiators=[],          # 无证据支撑
        tech_enablers=[],
        addressed_opportunity_ids=[],
    )
    # 经验驱动通常不做证据检索、不做合成用户验证
    return BaselineResult(
        concept=concept,
        assumed_pains=GUT_PAINS,
        elapsed_seconds=round(time.time() - start, 4),
        citations=0,
        validated_assumptions=0,
    )
```

backend/anker_studio/application/evaluation/rubric.py (67 行)

```python
"""评测 Rubric (Application 层), 对标 VitaBench 风格的可解释成功率定义。

```

```
维度: groundedness / faithfulness / citation_hit_rate / opportunity_coverage /
persona_fidelity / mean_iterations / explainability。设质量门槛。
"""
from __future__ import annotations

from anker_studio.application.graph.state import PipelineState
from anker_studio.common.citations import evaluate_citations
from anker_studio.common.models import RubricScore
from anker_studio.infrastructure.nlp.lexicons import ASPECT_LEXICON

# 质量门槛（不达标应触发重试/降级或人工复核）
GATES = {"groundedness": 0.6, "faithfulness": 0.5, "citation_hit_rate": 0.8}


def evaluate(state: PipelineState) -> RubricScore:
    cites = evaluate_citations(state.claims, state.all_evidences(), aspect_terms=ASPECT_LEXICON)

    voc = state.voc_report
    concept = state.chosen_concept
    if voc and concept and voc.opportunities:
        top_ids = {o.id for o in voc.opportunities[:5]}
        addressed = set(concept.addressed_opportunity_ids)
        coverage = round(len(top_ids & addressed) / min(5, len(voc.opportunities)), 4)
    else:
        coverage = 0.0

    if state.personas:
        grounded_personas = sum(1 for p in state.personas if p.derived_from_evidence_ids)
        persona_fidelity = round(grounded_personas / len(state.personas), 4)
    else:
        persona_fidelity = 0.0

    # 平均检索迭代: think_tank 累计 / 白空间机会数（近似）
    ws = len(state.market_intel.white_space_opportunities) if state.market_intel else 0
    mean_iter = round(state.retrieval_iters / ws, 2) if ws else float(state.retrieval_iters)

    # 可解释性: 是否产出了 DecisionRecord（含 rationale + conditions）
    explainability = 1.0 if (state.decision and state.decision.rationale) else 0.0

    overall = round(
        0.25 * cites["groundedness"]
        + 0.2 * cites["faithfulness"]
        + 0.15 * cites["citation_hit_rate"]
        + 0.2 * coverage
        + 0.1 * persona_fidelity
        + 0.1 * explainability,
        4,
    )

    notes = []
    for k, threshold in GATES.items():
        if cites.get(k, 0.0) < threshold:
            notes.append(f"未过门槛: {k}={cites.get(k,0.0)} < {threshold}")

    return RubricScore(
        groundedness=cites["groundedness"],
        faithfulness=cites["faithfulness"],
```

```
        citation_hit_rate=cites["citation_hit_rate"],
        opportunity_coverage=coverage,
        persona_fidelity=persona_fidelity,
        mean_iterations=mean_iter,
        explainability=explainability,
        overall=overall,
        notes=notes,
    )
```

backend/anker_studio/application/evaluation/comparison.py (91 行)

```python
"""AI 驱动 vs 经验驱动 量化对比 (Application 层) —— 命题核心。

同一 brief: A 组(经验驱动) vs B 组(AI 原生)。在统一维度上量化"本质不同"。
关键: A 组的概念也用真实数据(predict_nps)评估它"实际会达到的 NPS", 对比公平。
"""
from __future__ import annotations

from typing import List

from anker_studio.application.baseline.experience_driven import BaselineResult
from anker_studio.application.graph.state import PipelineState
from anker_studio.application.platforms.bees import predict_nps
from anker_studio.common.models import ArmMetrics, ComparisonReport


def _coverage(addressed_aspects: List[str], top_aspects: List[str]) -> float:
    if not top_aspects:
        return 0.0
    hit = len(set(addressed_aspects) & set(top_aspects))
    return round(hit / min(5, len(top_aspects)), 4)


def _hit_rate(addressed_aspects: List[str], top_aspects: List[str]) -> float:
    if not addressed_aspects:
        return 0.0
    hit = len(set(addressed_aspects) & set(top_aspects))
    return round(hit / len(addressed_aspects), 4)


def build_comparison(state: PipelineState, baseline: BaselineResult, brief: str) -> ComparisonReport:
    voc = state.voc_report
    top_aspects = [o.aspect for o in voc.opportunities[:5]] if voc else []

    # ---- B 组: AI 原生 ----
    b_addressed = []
    if voc and state.chosen_concept:
        opp_by_id = {o.id: o for o in voc.opportunities}
        b_addressed = [opp_by_id[i].aspect for i in state.chosen_concept.addressed_opportunity_ids if i in
      opp_by_id]
    distinct_evidence = len({eid for _, ids in state.claims for eid in ids})
    arm_b = ArmMetrics(
        arm="B_ai_native",
        opportunity_coverage=_coverage(b_addressed, top_aspects),
        evidence_citations=distinct_evidence,
        validated_assumptions=len(state.interviews),
```

```
        real_pain_hit_rate=_hit_rate(b_addressed, top_aspects),
        feasibility_risks_identified=len(state.feasibility.risks) if state.feasibility else 0,
        nps_prediction=state.nps.score if state.nps else 0.0,
        elapsed_seconds=0.0,  # 由 runner 填入
        distinct_personas_consulted=len(state.personas),
    )

    # ---- A 组：经验驱动（同样用真实数据评估其"实际会达到的 NPS"）----
    a_addressed = baseline.assumed_pains
    a_nps = predict_nps(voc, [], a_addressed).score if voc else 0.0
    arm_a = ArmMetrics(
        arm="A_experience_driven",
        opportunity_coverage=_coverage(a_addressed, top_aspects),
        evidence_citations=baseline.citations,
        validated_assumptions=baseline.validated_assumptions,
        real_pain_hit_rate=_hit_rate(a_addressed, top_aspects),
        feasibility_risks_identified=0,
        nps_prediction=a_nps,
        elapsed_seconds=baseline.elapsed_seconds,
        distinct_personas_consulted=0,
    )

    deltas = {
        "opportunity_coverage": round(arm_b.opportunity_coverage - arm_a.opportunity_coverage, 4),
        "evidence_citations": arm_b.evidence_citations - arm_a.evidence_citations,
        "validated_assumptions": arm_b.validated_assumptions - arm_a.validated_assumptions,
        "real_pain_hit_rate": round(arm_b.real_pain_hit_rate - arm_a.real_pain_hit_rate, 4),
        "feasibility_risks_identified": arm_b.feasibility_risks_identified - arm_a.
     feasibility_risks_identified,
        "nps_prediction": round(arm_b.nps_prediction - arm_a.nps_prediction, 1),
    }

    narrative = (
        f"经验驱动凭直觉聚焦 {a_addressed}，其中仅 {int(arm_a.real_pain_hit_rate*100)}% 命中真实高机会痛点，"
        f"零证据引用、零用户验证；AI 原生工作流命中率 {int(arm_b.real_pain_hit_rate*100)}%，"
        f"引用 {arm_b.evidence_citations} 条真实证据、做了 {arm_b.validated_assumptions} 次合成用户验证、"
        f"识别 {arm_b.feasibility_risks_identified} 项可行性风险，预测 NPS 高出 {deltas['nps_prediction']}
     分。"
        f"这就是'数据/AI 驱动'相对'经验拍脑袋'的本质不同：可溯源、可验证、可量化、可迭代。"
    )

    return ComparisonReport(
        category=state.category,
        brief=brief,
        arm_a=arm_a,
        arm_b=arm_b,
        deltas=deltas,
        narrative=narrative,
    )
```

backend/anker_studio/application/runner.py (114 行)

```
"""端到端 Runner（Application 层）：装配数据 → RAG → 图 → 评测 → 对比。

被 CLI 与 FastAPI 复用。支持 HITL 暂停/恢复。
"""
```

```python
from __future__ import annotations

import time
from typing import Optional

from pydantic import BaseModel

from anker_studio.application.baseline.experience_driven import run_experience_driven
from anker_studio.application.evaluation.comparison import build_comparison
from anker_studio.application.evaluation.rubric import evaluate
from anker_studio.application.graph.checkpoint import JsonCheckpointer
from anker_studio.application.graph.pipeline import build_studio_graph
from anker_studio.application.graph.state import PipelineState
from anker_studio.common.config import settings
from anker_studio.common.logging import log
from anker_studio.common.models import ComparisonReport, RubricScore
from anker_studio.infrastructure.data.loader import load_evidence, split_by_brand
from anker_studio.infrastructure.llm.gateway import LLMGateway
from anker_studio.infrastructure.observability.trace import Tracer
from anker_studio.infrastructure.rag.retrieval import build_rag

DEFAULT_BRIEF = "为 soundcore 设计下一款 TWS 耳机：用 AI 原生方法从洞察到定义，跑通一次。"


class RunArtifacts(BaseModel):
    run_id: str
    awaiting_human: bool = False
    thread_id: str = "default"
    state: PipelineState
    rubric: Optional[RubricScore] = None
    comparison: Optional[ComparisonReport] = None
    elapsed_seconds: float = 0.0


def _finalize(state: PipelineState, run_id: str, elapsed: float, brief: str,
              gateway: LLMGateway, thread_id: str) -> RunArtifacts:
    rubric = evaluate(state)
    baseline = run_experience_driven(state.category, brief, gateway)
    comparison = build_comparison(state, baseline, brief)
    comparison.arm_b.elapsed_seconds = round(elapsed, 3)
    log.bind(node="runner").info(
        f"完成：决策={state.decision.verdict.value if state.decision else 'n/a'} "
        f"NPS={state.nps.score if state.nps else 'n/a'} 总分={rubric.overall}"
    )
    return RunArtifacts(
        run_id=run_id, awaiting_human=False, thread_id=thread_id,
        state=state, rubric=rubric, comparison=comparison,
        elapsed_seconds=round(elapsed, 3),
    )


def run_studio(
    category: str = "audio",
    brief: str = DEFAULT_BRIEF,
    hitl: Optional[bool] = None,
    thread_id: str = "default",
    tracer: Optional[Tracer] = None,
```

```
) -> RunArtifacts:
    cfg = settings()
    hitl = cfg.hitl if hitl is None else hitl
    gateway = LLMGateway.from_settings(cfg)
    tracer = tracer or Tracer()

    evidences = load_evidence(category)
    split = split_by_brand(evidences)
    state = PipelineState(
        category=category,
        target_brand="soundcore",
        brief=brief,
        target_evidences=split["target"],
        competitor_evidences=split["competitors"],
    )
    if not state.target_evidences:
        log.bind(node="runner").warning(
            "未找到 soundcore 目标评论：将用全部评论兜底（请检查 data/ 数据）。"
        )
        state.target_evidences = evidences

    rag = build_rag(evidences)
    graph = build_studio_graph(gateway, rag, tracer, hitl=hitl)
    checkpointer = JsonCheckpointer()

    start = time.time()
    state = graph.run(state, checkpointer=checkpointer, thread_id=thread_id)
    if state.awaiting_human:
        log.bind(node="runner").warning("流程在决策闸暂停（HITL）。调用 resume_studio 继续。")
        return RunArtifacts(
            run_id=tracer.run_id, awaiting_human=True, thread_id=thread_id, state=state,
            elapsed_seconds=round(time.time() - start, 3),
        )
    return _finalize(state, tracer.run_id, time.time() - start, brief, gateway, thread_id)


def resume_studio(thread_id: str = "default", tracer: Optional[Tracer] = None) -> RunArtifacts:
    """HITL：人工确认后从 checkpoint 恢复。"""
    cfg = settings()
    gateway = LLMGateway.from_settings(cfg)
    tracer = tracer or Tracer()
    checkpointer = JsonCheckpointer()
    loaded = checkpointer.load(thread_id)
    if loaded is None:
        raise RuntimeError(f"无 checkpoint 可恢复：thread_id={thread_id}")

    # 重新装配图与 RAG（用 checkpoint 中的 state 数据）
    state, _ = loaded
    rag = build_rag(state.all_evidences())
    graph = build_studio_graph(gateway, rag, tracer, hitl=cfg.hitl)
    start = time.time()
    state = graph.run(state, checkpointer=checkpointer, thread_id=thread_id, resume=True)
    return _finalize(state, tracer.run_id, time.time() - start, state.brief, gateway, thread_id)
```

backend/anker_studio/application/reporting.py (232 行)

```python
"""报告渲染 (Application 层) : 把 RunArtifacts 渲染成 Markdown。

产出: 完整方案报告 (提案 + 可行性 + 决策 + 评测 + AI/经验对比) 以及
竞赛开题报告 Part1/Part2 (数字取自真实运行结果) 。
"""
from __future__ import annotations

from typing import List

from anker_studio.application.runner import RunArtifacts
from anker_studio.common.config import settings


def _faq_block(title: str, items) -> str:
    lines = [f"**{title}**", ""]
    for it in items:
        cite = f" (证据: {', '.join(it.evidence_ids)}) " if it.evidence_ids else ""
        lines.append(f"- **Q: {it.question}**")
        lines.append(f"  - A: {it.answer} {cite}")
    return "\n".join(lines)


def render_full_report(art: RunArtifacts) -> str:
    s = art.state
    p = s.proposal
    voc = s.voc_report
    cmp = art.comparison
    rub = art.rubric
    out: List[str] = []

    out.append(f"# Anker AI 原生产品定义 · 运行报告 ({art.run_id}) \n")
    out.append(f"> 品类: {s.category}  目标品牌: {s.target_brand}  评论样本: {voc.review_count if voc else 0} 条  "
               f"LLM provider: {settings().llm_provider}  耗时: {art.elapsed_seconds}s\n")

    # 1. 用户洞察
    if voc:
        out.append("## 1. 用户洞察 (AI 原生 JML) \n")
        out.append("| 维度 aspect | 提及覆盖 | 负面率 | 满意度 | 机会分 | Impact |")
        out.append("|---|---|---|---|---|---|")
        for a in voc.aspects[:9]:
            out.append(f"| {a.aspect} | {a.reach:.0%} | {a.negative_rate:.0%} | {a.satisfaction}/10 | "
                       f"{a.opportunity_score} | {a.impact_score} |")
        out.append("\n**Top 机会 (按机会分) **: ")
        for o in voc.opportunities[:5]:
            out.append(f"- `{o.id}` {o.statement} — 机会分 {o.opportunity_score} (证据: {', '.join(o.evidence_ids[:3])}) ")
        out.append("")

    # 2. 市场/竞品
    if s.market_intel:
        out.append("## 2. 市场与竞品洞察 (AI 原生 AMI / 超级智囊) \n")
        for c in s.market_intel.competitors:
            out.append(f"- **{c.brand}** ({c.review_count} 条) | 优势: {('、'.join(c.strengths) or '—')} | "
                       f"短板: {('、'.join(c.weaknesses) or '—')}")
        if s.market_intel.white_space_opportunities:
            out.append("\n**白空间机会**: ")
```

```
            for w in s.market_intel.white_space_opportunities[:4]:
                out.append(f"- `{w.id}` {w.statement}")
        out.append("")

    # 3. 概念
    if s.chosen_concept:
        c = s.chosen_concept
        out.append("## 3. 产品概念（双路径：VOC 驱动 + 技术原生）\n")
        out.append(f"**{c.name}** — {c.one_liner}\n")
        out.append(f"- 目标用户：{c.target_segment}")
        out.append(f"- 价值主张：{c.value_proposition}")
        out.append(f"- 核心功能：{('; '.join(c.key_features))}")
        out.append(f"- 技术使能：{('; '.join(c.tech_enablers))}")
        out.append("- 差异点：")
        for d in c.differentiators:
            out.append(f"  - {d.statement}（证据：{', '.join(d.evidence_ids[:2]) or '—'}）")
        out.append("")

    # 4. 合成用户
    if s.interviews:
        out.append("## 4. 用户替身验证（合成用户面板，假设盲）\n")
        for itv in s.interviews:
            out.append(f"- **{itv.segment}** → 判定：`{itv.verdict}`，接受度 {itv.acceptance:.0%}"
                       f"{'，异议：' + '；'.join(itv.objections) if itv.objections else ''}")
        out.append("")

    # 5. PR/FAQ
    if p and p.prfaq:
        f = p.prfaq
        out.append("## 5. 产品提案 PR/FAQ（Working Backwards）\n")
        out.append(f"### {f.headline}\n")
        out.append(f"*{f.subheading}*\n")
        out.append(f"{f.summary}\n")
        out.append(f"> 用户说：{f.customer_quote}\n")
        out.append(f"> 团队说：{f.maker_quote}\n")
        out.append(_faq_block("外部 FAQ", f.external_faq))
        out.append("")
        out.append(_faq_block("内部 FAQ", f.internal_faq))
        out.append("")

    # 6. 可行性 + 决策
    if p and p.feasibility:
        fe = p.feasibility
        out.append("## 6. 可行性与决策（行业专家 + 决策官）\n")
        out.append(f"- 技术：{fe.technical}")
        out.append(f"- 供应链：{fe.supply_chain}")
        out.append(f"- 成本：{fe.bom_cost}")
        out.append(f"- 合规：{fe.compliance}")
        out.append(f"- 总体：**{fe.overall.upper()}**")
        out.append("- 风险：")
        for r in fe.risks:
            out.append(f"  - [{r.severity}] {r.area}：{r.description} → {r.mitigation}")
        if p.decision:
            d = p.decision
            out.append(f"\n**决策：`{d.verdict.value}`** （置信度 {d.confidence}） | 预测 NPS {d.
 nps_prediction:.0f}")
            out.append(f"- 理由：{d.rationale}")
```

```
            if d.conditions:
                out.append(f"- 条件：{'; '.join(d.conditions)}")
        if p.concept_image_path:
            out.append(f"\n![concept]({p.concept_image_path})")
        out.append("")

    # 7. 评测
    if rub:
        out.append("## 7. 评测 Rubric\n")
        out.append("| 指标 | 值 |")
        out.append("|---|---|")
        out.append(f"| groundedness（论断带引用比例） | {rub.groundedness} |")
        out.append(f"| faithfulness（引用支撑论断比例） | {rub.faithfulness} |")
        out.append(f"| citation_hit_rate（引用命中） | {rub.citation_hit_rate} |")
        out.append(f"| opportunity_coverage（机会覆盖） | {rub.opportunity_coverage} |")
        out.append(f"| persona_fidelity（合成用户 grounding） | {rub.persona_fidelity} |")
        out.append(f"| mean_iterations（平均检索迭代） | {rub.mean_iterations} |")
        out.append(f"| explainability（可解释性） | {rub.explainability} |")
        out.append(f"| **overall** | **{rub.overall}** |")
        if rub.notes:
            out.append("\n门槛提示：" + "；".join(rub.notes))
        out.append("")

    # 8. 对比
    if cmp:
        out.append("## 8. AI 驱动 vs 经验驱动（命题核心对比）\n")
        a, b = cmp.arm_a, cmp.arm_b
        out.append("| 维度 | A 经验驱动 | B AI 原生 | Δ |")
        out.append("|---|---|---|---|")
        out.append(f"| 机会覆盖率 | {a.opportunity_coverage:.0%} | {b.opportunity_coverage:.0%} | {cmp.
    deltas['opportunity_coverage']:+.2f} |")
        out.append(f"| 命中真实痛点率 | {a.real_pain_hit_rate:.0%} | {b.real_pain_hit_rate:.0%} | {cmp.
    deltas['real_pain_hit_rate']:+.2f} |")
        out.append(f"| 证据引用数 | {a.evidence_citations} | {b.evidence_citations} | {cmp.deltas['
    evidence_citations']:+.0f} |")
        out.append(f"| 已验证假设数 | {a.validated_assumptions} | {b.validated_assumptions} | {cmp.deltas['
    validated_assumptions']:+.0f} |")
        out.append(f"| 合成用户数 | {a.distinct_personas_consulted} | {b.distinct_personas_consulted} | {b.
    distinct_personas_consulted - a.distinct_personas_consulted:+.0f} |")
        out.append(f"| 可行性风险识别 | {a.feasibility_risks_identified} | {b.feasibility_risks_identified}
     | {cmp.deltas['feasibility_risks_identified']:+.0f} |")
        out.append(f"| 预测 NPS | {a.nps_prediction:.0f} | {b.nps_prediction:.0f} | {cmp.deltas['
    nps_prediction']:+.1f} |")
        out.append(f"\n{cmp.narrative}\n")
    return "\n".join(out)


def to_view(art: RunArtifacts) -> dict:
    """构造前端用的紧凑视图（剔除原始 evidence 大数组）。"""
    s = art.state
    voc = s.voc_report
    intel = s.market_intel
    p = s.proposal
    cmp = art.comparison
    rub = art.rubric

    def opp(o):
```

```
        return {"id": o.id, "aspect": o.aspect, "statement": o.statement,
                "opportunity_score": o.opportunity_score, "evidence_ids": o.evidence_ids[:3]}

    return {
        "run_id": art.run_id,
        "elapsed_seconds": art.elapsed_seconds,
        "provider": settings().llm_provider,
        "category": s.category,
        "target_brand": s.target_brand,
        "voc": None if not voc else {
            "review_count": voc.review_count,
            "aspects": [a.model_dump() for a in voc.aspects],
            "opportunities": [opp(o) for o in voc.opportunities[:6]],
            "ost": voc.ost.model_dump() if voc.ost else None,
        },
        "market": None if not intel else {
            "competitors": [c.model_dump() for c in intel.competitors],
            "white_space": [opp(o) for o in intel.white_space_opportunities[:4]],
            "trends": [t.model_dump() for t in intel.trends],
        },
        "concept": s.chosen_concept.model_dump() if s.chosen_concept else None,
        "personas": [pp.model_dump() for pp in s.personas],
        "interviews": [iv.model_dump() for iv in s.interviews],
        "feasibility": s.feasibility.model_dump() if s.feasibility else None,
        "nps": s.nps.model_dump() if s.nps else None,
        "prfaq": p.prfaq.model_dump() if (p and p.prfaq) else None,
        "decision": s.decision.model_dump() if s.decision else None,
        "rubric": rub.model_dump() if rub else None,
        "comparison": cmp.model_dump() if cmp else None,
        "concept_image_path": p.concept_image_path if p else None,
    }


def render_opening_report(art: RunArtifacts) -> str:
    """竞赛开题报告 Part1/Part2（数字取自真实运行）。"""
    s = art.state
    voc = s.voc_report
    cmp = art.comparison
    top = voc.opportunities[:3] if voc else []
    top_aspects = "、".join(o.aspect for o in top) or "核心体验"
    hit_b = int(cmp.arm_b.real_pain_hit_rate * 100) if cmp else 0
    hit_a = int(cmp.arm_a.real_pain_hit_rate * 100) if cmp else 0
    dnps = cmp.deltas["nps_prediction"] if cmp else 0

    out: List[str] = []
    out.append("# 开题报告（由系统真实产出，数字随数据更新）\n")
    out.append("## Part 1 命题前置分析与洞察\n")
    out.append(
        f"安克在 2023 年 All in AI 后，已沉淀 AIME（300+ Agent，含 VOC 洞察/需求生成）与 "
        f"JML/BEES/AMI 三大产品方法论平台，NPS 为核心指标。命题表面是“设计一款产品”，"
        f"本质是要验证：能否用 AI 把“产品经理拍脑袋”重构为“数据可溯源、可验证、可量化”的工作流。"
        f"我们对 {voc.review_count if voc else 0} 条真实 soundcore/竞品评论做确定性 ABSA，"
        f"发现机会分最高的痛点集中在 {top_aspects}——这与“凭直觉做更好音质/续航/性价比”的常识假设并不一致，"
        f"正说明经验驱动存在系统性盲区。\n"
    )
    out.append("## Part 2 整体解决方案设计\n")
    out.append(
```

```
        "**1. 方案概述**: 构建 “AI 原生产品定义系统”，把安克 JML/BEES/AMI 做成 AI 原生版，"
        "让 AI 扮演超级智囊/用户替身/行业专家，端到端从真实评论跑到一份可溯源的产品提案(PR/FAQ)。\n"
        "**2. 架构与模块**: 多源真实数据(Evidence) → JML 用户洞察(ABSA+ODI+机会树) + AMI 竞品/趋势 → "
        "产品经理双路径概念(VOC 驱动 + 第一性原理技术原生) → 用户替身(OCEAN 合成用户，假设盲) → "
        "行业专家可行性(技术/BOM/供应链/合规) + 决策官(NPS + GO/NO-GO)，由 StateGraph 编排，"
        "全程引用校验 + 决策审计 + HITL 闸口。\n"
        "**3. 核心创新**: 方法论创新——双路径覆盖“用户洞察”与“技术原生”； 工程创新——确定性核心+LLM 增强，"
        "强制逐字引用校验，对标大厂 AIGC 规范(四层架构/AI Rule/Pre-PR/跨模型对抗 CR)； 把“本质不同”做成可量化对比实验。\n"
        f"**4. 落地价值（可量化）**: 在相同 brief 下，AI 原生命中真实痛点率 {hit_b}% vs 经验驱动 {hit_a}%，"
        f"预测 NPS 高出 {dnps:+.0f} 分，且全部结论可溯源到真实评论；方法论可沉淀为安克内部 SOP。\n"
        "**5. 可行性与可推广**: 系统离线即可复现，接入 MiniMax/真实 Amazon 数据即增强；"
        "品类无关，换充电/eufy 仅需更换数据配置，可直接迁移到安克其它品类。\n"
    )
    return "\n".join(out)
```

G Starter 层（CLI / FastAPI）与入口

backend/anker_studio/starter/cli.py (81 行)

```python
"""CLI 入口（Starter 层）。

用法:
    python -m anker_studio.starter.cli run                # 跑完整 AI 原生工作流并写报告
    python -m anker_studio.starter.cli run --brief "..."  # 自定义 brief
    python -m anker_studio.starter.cli resume             # HITL 暂停后恢复
"""
from __future__ import annotations

import argparse
from pathlib import Path

from anker_studio.application.reporting import render_full_report, render_opening_report
from anker_studio.application.runner import DEFAULT_BRIEF, resume_studio, run_studio
from anker_studio.common.config import settings
from anker_studio.common.logging import log


def _write_reports(art) -> None:
    out_dir = Path(settings().project_root) / "docs" / "generated"
    out_dir.mkdir(parents=True, exist_ok=True)
    (out_dir / "运行报告.md").write_text(render_full_report(art), encoding="utf-8")
    (out_dir / "开题报告_自动生成.md").write_text(render_opening_report(art), encoding="utf-8")
    log.bind(node="cli").info(f"报告已写入 {out_dir}")


def _print_summary(art) -> None:
    s = art.state
    cmp = art.comparison
    rub = art.rubric
    print("\n" + "=" * 64)
    print("AI 原生产品定义 · 运行摘要")
    print("=" * 64)
```

```
    if s.voc_report:
        print(f"评论样本：{s.voc_report.review_count}  Top机会：" +
              "、".join(o.aspect for o in s.voc_report.opportunities[:3]))
    if s.chosen_concept:
        print(f"概念：{s.chosen_concept.name} — {s.chosen_concept.one_liner}")
    if s.decision:
        print(f"决策：{s.decision.verdict.value}  预测NPS：{s.decision.nps_prediction:.0f}")
    if rub:
        print(f"评测 overall：{rub.overall}（groundedness={rub.groundedness}, "
              f"citation_hit={rub.citation_hit_rate}, coverage={rub.opportunity_coverage}）")
    if cmp:
        print(f"对比：命中真实痛点 经验{cmp.arm_a.real_pain_hit_rate:.0%} vs AI {cmp.arm_b.real_pain_hit_rate:.0%}"
              f"｜证据引用 {cmp.arm_a.evidence_citations} vs {cmp.arm_b.evidence_citations}"
              f"｜ΔNPS {cmp.deltas['nps_prediction']:+.1f}")
    print("=" * 64 + "\n")


def main() -> int:
    ap = argparse.ArgumentParser(description="Anker AI 原生产品定义系统")
    sub = ap.add_subparsers(dest="cmd", required=True)
    run_p = sub.add_parser("run", help="运行完整工作流")
    run_p.add_argument("--brief", default=DEFAULT_BRIEF)
    run_p.add_argument("--category", default="audio")
    run_p.add_argument("--hitl", action="store_true", help="决策闸人工确认")
    run_p.add_argument("--thread", default="cli")
    res_p = sub.add_parser("resume", help="HITL 恢复")
    res_p.add_argument("--thread", default="cli")
    args = ap.parse_args()

    if args.cmd == "run":
        art = run_studio(category=args.category, brief=args.brief, hitl=args.hitl, thread_id=args.thread)
        if art.awaiting_human:
            print(f"[HITL] 已在决策闸暂停。确认后运行：python -m anker_studio.starter.cli resume --thread {args.thread}")
            return 0
        _print_summary(art)
        _write_reports(art)
        return 0

    if args.cmd == "resume":
        art = resume_studio(thread_id=args.thread)
        _print_summary(art)
        _write_reports(art)
        return 0
    return 1


if __name__ == "__main__":
    raise SystemExit(main())
```

backend/anker_studio/starter/api.py (103 行)

```
"""FastAPI 入口（Starter 层）：暴露工作流运行 + 流式 trace + 报告，并托管前端工作台。

运行：
```

```
    PYTHONPATH=backend uvicorn anker_studio.starter.api:app --reload --port 8000
然后浏览器打开 http://localhost:8000
"""
from __future__ import annotations

import json
import queue
import threading
from pathlib import Path
from typing import Any, Dict, Optional

from fastapi import FastAPI
from fastapi.responses import FileResponse, JSONResponse, StreamingResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel

from anker_studio.application.reporting import (
    render_full_report,
    render_opening_report,
    to_view,
)
from anker_studio.application.runner import DEFAULT_BRIEF, run_studio
from anker_studio.common.config import settings
from anker_studio.infrastructure.observability.trace import Tracer

app = FastAPI(title="Anker AI 原生产品定义系统", version="0.1.0")

FRONTEND_DIR = Path(settings().project_root) / "frontend"
ASSETS_DIR = Path(settings().project_root) / "assets"

# 缓存最近一次运行，供报告接口使用
_LAST: Dict[str, Any] = {}


class RunRequest(BaseModel):
    brief: Optional[str] = None


@app.get("/api/health")
def health() -> dict:
    return {"status": "ok", "provider": settings().llm_provider}


@app.post("/api/run")
def run(req: RunRequest = RunRequest()) -> JSONResponse:
    brief = req.brief or DEFAULT_BRIEF
    art = run_studio(brief=brief, thread_id="api")
    _LAST["art"] = art
    return JSONResponse(to_view(art))


@app.get("/api/stream")
def stream(brief: str = DEFAULT_BRIEF) -> StreamingResponse:
    """SSE：边跑边推送 trace 事件，结束推送最终视图。"""
    q: "queue.Queue[dict]" = queue.Queue()
    tracer = Tracer()
    tracer.subscribe(lambda ev: q.put({"type": "trace", "event": ev}))
```

```
61
62      def worker() -> None:
63          try:
64              art = run_studio(brief=brief, thread_id="api", tracer=tracer)
65              _LAST["art"] = art
66              q.put({"type": "result", "view": to_view(art)})
67          except Exception as exc:  # noqa: BLE001 - 把错误也推给前端
68              q.put({"type": "error", "message": str(exc)})
69          finally:
70              q.put({"type": "done"})
71
72      threading.Thread(target=worker, daemon=True).start()
73
74      def gen():
75          while True:
76              item = q.get()
77              yield f"data: {json.dumps(item, ensure_ascii=False, default=str)}\n\n"
78              if item.get("type") == "done":
79                  break
80
81      return StreamingResponse(gen(), media_type="text/event-stream")
82
83
84  @app.get("/api/report")
85  def report(kind: str = "full") -> JSONResponse:
86      art = _LAST.get("art")
87      if art is None:
88          return JSONResponse({"error": "尚未运行，请先 POST /api/run 或 /api/stream"}, status_code=400)
89      md = render_opening_report(art) if kind == "opening" else render_full_report(art)
90      return JSONResponse({"markdown": md})
91
92
93  if ASSETS_DIR.exists():
94      app.mount("/assets", StaticFiles(directory=str(ASSETS_DIR)), name="assets")
95
96
97  @app.get("/")
98  def index() -> FileResponse:
99      return FileResponse(str(FRONTEND_DIR / "index.html"))
100
101
102 if FRONTEND_DIR.exists():
103     app.mount("/static", StaticFiles(directory=str(FRONTEND_DIR)), name="static")
```

run.py

(27 行)

```
1   #!/usr/bin/env python3
2   """生产入口：启动 FastAPI（供 systemd 守护）。
3
4   监听 127.0.0.1:$PORT（默认 8766，对齐服务器项目二端口约定），只对 nginx 开放。
5   本地直接运行也可：`python run.py`。
6   """
7   from __future__ import annotations
8
9   import os
10  import sys
```

```python
from pathlib import Path

# 把 backend 加入 import 路径（无需设置 PYTHONPATH）
sys.path.insert(0, str(Path(__file__).resolve().parent / "backend"))


def main() -> None:
    import uvicorn

    # 用 ANKER_HOST 而非 HOST：macOS/部分 shell 会把 HOST 设为主机名，导致绑定失败
    host = os.getenv("ANKER_HOST", "127.0.0.1")
    port = int(os.getenv("PORT", "8766"))
    uvicorn.run("anker_studio.starter.api:app", host=host, port=port, log_level="info")


if __name__ == "__main__":
    main()
```

H 前端工作台（零依赖可视化）

frontend/index.html (77 行)

```html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Anker · AI 原生产品定义工作台</title>
  <link rel="stylesheet" href="static/styles.css" />
</head>
<body>
  <header class="topbar">
    <div class="brand">
      <span class="dot"></span>
      <div>
        <h1>AI 原生产品定义工作台</h1>
        <p>让 AI 做超级智囊 · 用户替身 · 行业专家 —— 从真实数据到产品提案</p>
      </div>
    </div>
    <div class="run">
      <input id="brief" type="text" value="为 soundcore 设计下一款 TWS 耳机：用 AI 原生方法从洞察到定义。" />
      <button id="runBtn"> 运行工作流</button>
    </div>
  </header>

  <section class="pipeline" id="pipeline">
    <!-- node chips injected -->
  </section>

  <main class="grid">
    <div class="card span2" id="card-compare">
      <h2>AI 驱动 vs 经验驱动（命题核心）</h2>
      <div id="compare" class="muted">运行后显示量化对比。</div>
    </div>
```

```
    <div class="card" id="card-voc">
      <h2>用户洞察 · JML（ABSA + ODI）</h2>
      <div id="voc" class="muted">运行后显示真实评论挖出的机会。</div>
    </div>

    <div class="card" id="card-market">
      <h2>市场/竞品 · AMI（超级智囊）</h2>
      <div id="market" class="muted">运行后显示竞品短板与白空间。</div>
    </div>

    <div class="card" id="card-concept">
      <h2>产品概念（双路径）</h2>
      <div id="concept" class="muted">VOC 驱动 + 第一性原理/技术原生。</div>
    </div>

    <div class="card" id="card-users">
      <h2>用户替身（合成用户，假设盲）</h2>
      <div id="users" class="muted">运行后显示访谈判定。</div>
    </div>

    <div class="card span2" id="card-prfaq">
      <h2>产品提案 · PR/FAQ（Working Backwards）</h2>
      <div id="prfaq" class="muted">运行后显示提案。</div>
    </div>

    <div class="card" id="card-decision">
      <h2>可行性与决策（NPS · GO/NO-GO）</h2>
      <div id="decision" class="muted">行业专家 + 决策官。</div>
    </div>

    <div class="card" id="card-rubric">
      <h2>评测 Rubric</h2>
      <div id="rubric" class="muted">groundedness / faithfulness / 覆盖率…</div>
    </div>
  </main>

  <footer>
    <span id="status">就绪</span> · <a href="api/report" target="_blank">完整报告 JSON</a> ·
    <a href="api/report?kind=opening" target="_blank">开题报告 JSON</a>
  </footer>

  <script src="static/app.js"></script>
</body>
</html>
```

frontend/app.js (168 行)

```
"use strict";

const NODES = [
  ["voc", "用户洞察 JML"],
  ["think_tank", "超级智囊 AMI"],
  ["pm", "产品经理"],
  ["users", "用户替身"],
  ["expert", "行业专家"],
```

```
  ["decision", "决策官"],
  ["output", "产出 PR/FAQ"],
];

const $ = (id) => document.getElementById(id);
const pct = (x) => `${Math.round((x || 0) * 100)}%`;

function renderPipeline() {
  const el = $("pipeline");
  el.innerHTML = NODES.map(
    ([id, label]) => `<div class="chip" data-node="${id}"><span class="led"></span>${label}</div>`
  ).join("");
}

function setChip(node, state) {
  const c = document.querySelector(`.chip[data-node="${node}"]`);
  if (!c) return;
  if (state === "start") { c.classList.add("active"); c.classList.remove("done"); }
  if (state === "end") { c.classList.remove("active"); c.classList.add("done"); }
}

function resetChips() {
  document.querySelectorAll(".chip").forEach((c) => c.classList.remove("active", "done"));
}

function run() {
  resetChips();
  $("runBtn").disabled = true;
  $("status").textContent = "运行中…";
  const brief = encodeURIComponent($("brief").value);
  const es = new EventSource(`api/stream?brief=${brief}`);
  es.onmessage = (e) => {
    const msg = JSON.parse(e.data);
    if (msg.type === "trace") {
      const ev = msg.event;
      if (ev.kind === "start") setChip(ev.node, "start");
      if (ev.kind === "end") setChip(ev.node, "end");
      $("status").textContent = `节点: ${ev.node} · ${ev.kind}`;
    } else if (msg.type === "result") {
      renderAll(msg.view);
      $("status").textContent = `完成 · ${msg.view.run_id} · ${msg.view.elapsed_seconds}s · provider=${
     msg.view.provider}`;
    } else if (msg.type === "error") {
      $("status").textContent = "错误: " + msg.message;
    } else if (msg.type === "done") {
      es.close();
      $("runBtn").disabled = false;
    }
  };
  es.onerror = () => { es.close(); $("runBtn").disabled = false; $("status").textContent = "连接结束"; };
}

function renderAll(v) {
  renderCompare(v.comparison);
  renderVoc(v.voc);
  renderMarket(v.market);
  renderConcept(v.concept);
```

```
  renderUsers(v.interviews);
  renderPrfaq(v.prfaq, v.concept_image_path);
  renderDecision(v.feasibility, v.decision, v.nps);
  renderRubric(v.rubric);
}

function renderCompare(c) {
  if (!c) return;
  const rows = [
    ["机会覆盖率", pct(c.arm_a.opportunity_coverage), pct(c.arm_b.opportunity_coverage)],
    ["命中真实痛点率", pct(c.arm_a.real_pain_hit_rate), pct(c.arm_b.real_pain_hit_rate)],
    ["证据引用数", c.arm_a.evidence_citations, c.arm_b.evidence_citations],
    ["已验证假设数", c.arm_a.validated_assumptions, c.arm_b.validated_assumptions],
    ["合成用户数", c.arm_a.distinct_personas_consulted, c.arm_b.distinct_personas_consulted],
    ["可行性风险识别", c.arm_a.feasibility_risks_identified, c.arm_b.feasibility_risks_identified],
    ["预测 NPS", c.arm_a.nps_prediction, c.arm_b.nps_prediction],
  ];
  $("compare").innerHTML =
    `<table><tr><th>维度</th><th>A 经验驱动</th><th>B AI 原生</th></tr>` +
    rows.map((r) => `<tr><td>${r[0]}</td><td>${r[1]}</td><td class="delta-pos">${r[2]}</td></tr>`).join
     ("") +
    `</table><p class="muted" style="margin-top:10px">${c.narrative}</p>`;
}

function renderVoc(voc) {
  if (!voc) return;
  const maxOpp = Math.max(...voc.opportunities.map((o) => o.opportunity_score), 1);
  $("voc").innerHTML =
    `<small>样本 ${voc.review_count} 条评论</small>` +
    `<table style="margin-top:8px"><tr><th>机会</th><th>机会分</th></tr>` +
    voc.opportunities.map((o) =>
      `<tr><td>${o.aspect}<div class="cite">${(o.evidence_ids||[]).join(", ")}</div></td>
       <td><div class="bar" style="width:${(o.opportunity_score / maxOpp) * 100}%"></div> ${o.
     opportunity_score}</td></tr>`
    ).join("") + `</table>`;
}

function renderMarket(m) {
  if (!m) return;
  const comp = m.competitors.map((c) =>
    `<div class="kv"><span><b>${c.brand}</b> <small>(${c.review_count})</small></span>
     <span>${(c.weaknesses||[]).map((w) => `<span class="tag bad">${w}</span>`).join("") || "—"}</span></
     div>`
  ).join("");
  const ws = m.white_space.map((o) => `<span class="tag warn">${o.aspect}</span>`).join("");
  const tr = m.trends.map((t) => `<span class="tag good">${t.name}</span>`).join("");
  $("market").innerHTML = comp + `<h3>白空间机会</h3>${ws}<h3>趋势</h3>${tr}`;
}

function renderConcept(c) {
  if (!c) return;
  $("concept").innerHTML =
    `<b>${c.name}</b><p class="muted">${c.one_liner}</p>` +
    `<div class="kv"><span>目标用户</span><span>${c.target_segment}</span></div>` +
    `<h3>核心功能</h3>` + (c.key_features||[]).map((f) => `<span class="tag">${f}</span>`).join("") +
    `<h3>技术使能</h3>` + (c.tech_enablers||[]).map((f) => `<span class="tag good">${f}</span>`).join("");
}
```

```

function renderUsers(itvs) {
  if (!itvs) return;
  $("users").innerHTML = itvs.map((i) => {
    const cls = i.verdict === "would_buy" ? "good" : i.verdict === "would_not_buy" ? "bad" : "warn";
    return `<div class="kv"><span>${i.segment}</span>
      <span><span class="tag ${cls}">${i.verdict}</span> ${pct(i.acceptance)}</span></div>`;
  }).join("");
}

function renderPrfaq(f, img) {
  if (!f) return;
  const faq = (arr) => arr.map((q) =>
    `<div class="kv" style="display:block"><b>Q: ${q.question}</b><div class="muted">A: ${q.answer}
     <span class="cite">${(q.evidence_ids||[]).join(", ")}</span></div></div>`).join("");
  $("prfaq").innerHTML =
    `<h3>${f.headline}</h3><p><i>${f.subheading}</i></p><p>${f.summary}</p>` +
    (img ? `<img src="/${img.replace(/^.*assets\//,'assets/')}" style="max-width:100%;border-radius:10px;
     margin:8px 0"/>` : "") +
    `<h3>外部 FAQ</h3>${faq(f.external_faq)}<h3>内部 FAQ</h3>${faq(f.internal_faq)}`;
}

function renderDecision(fe, d, nps) {
  if (!d) return;
  $("decision").innerHTML =
    `<div class="verdict ${d.verdict}">${d.verdict}</div>` +
    `<div class="kv"><span>预测 NPS</span><span>${(nps ? nps.score : d.nps_prediction)}</span></div>` +
    `<div class="kv"><span>置信度</span><span>${d.confidence}</span></div>` +
    `<div class="kv"><span>可行性</span><span class="tag ${fe && fe.overall==='green'?'good':fe&&fe.overall
     ==='red'?'bad':'warn'}">${fe ? fe.overall : '-'}</span></div>` +
    `<p class="muted">${d.rationale}</p>` +
    (fe ? `<h3>主要风险</h3>` + fe.risks.map((r) => `<div class="muted">· [${r.severity}] ${r.description
     }</div>`).join("") : "");
}

function renderRubric(r) {
  if (!r) return;
  const row = (k, val) => `<div class="kv"><span>${k}</span><span>${val}</span></div>`;
  $("rubric").innerHTML =
    row("groundedness", r.groundedness) + row("faithfulness", r.faithfulness) +
    row("citation_hit_rate", r.citation_hit_rate) + row("opportunity_coverage", r.opportunity_coverage) +
    row("persona_fidelity", r.persona_fidelity) + row("explainability", r.explainability) +
    `<div class="kv"><span><b>overall</b></span><span><b>${r.overall}</b></span></div>` +
    (r.notes && r.notes.length ? `<p class="tag warn">${r.notes.join("; ")}</p>` : "");
}

renderPipeline();
$("runBtn").addEventListener("click", run);

// 截图/演示用: URL 带 ?auto=1 时自动运行一次
if (new URLSearchParams(location.search).get("auto") === "1") {
  window.addEventListener("load", () => setTimeout(run, 400));
}
```

frontend/styles.css (75 行)

```
:root {
  --bg: #0b0f17;
  --panel: #131a26;
  --panel2: #0f1521;
  --line: #243045;
  --text: #e7edf6;
  --muted: #8a98ad;
  --accent: #36d3a0;
  --accent2: #5b8cff;
  --warn: #f0b429;
  --bad: #f0506e;
  --good: #36d3a0;
}
* { box-sizing: border-box; }
body {
  margin: 0; background: var(--bg); color: var(--text);
  font-family: -apple-system, "PingFang SC", "Segoe UI", Roboto, sans-serif;
  font-size: 14px; line-height: 1.6;
}
.topbar {
  display: flex; justify-content: space-between; align-items: center; gap: 20px;
  padding: 18px 28px; border-bottom: 1px solid var(--line);
  background: linear-gradient(180deg, #121a28, #0b0f17);
  position: sticky; top: 0; z-index: 10;
}
.brand { display: flex; align-items: center; gap: 14px; }
.brand .dot { width: 12px; height: 12px; border-radius: 50%; background: var(--accent);
  box-shadow: 0 0 16px var(--accent); }
.brand h1 { font-size: 18px; margin: 0; }
.brand p { margin: 2px 0 0; color: var(--muted); font-size: 12px; }
.run { display: flex; gap: 8px; flex: 1; max-width: 620px; }
.run input { flex: 1; background: var(--panel2); border: 1px solid var(--line);
  color: var(--text); border-radius: 8px; padding: 9px 12px; }
.run button { background: var(--accent); color: #04130d; border: 0; border-radius: 8px;
  padding: 9px 16px; font-weight: 700; cursor: pointer; white-space: nowrap; }
.run button:disabled { opacity: .5; cursor: not-allowed; }

.pipeline { display: flex; flex-wrap: wrap; gap: 8px; padding: 16px 28px;
  border-bottom: 1px solid var(--line); }
.chip { display: flex; align-items: center; gap: 8px; padding: 7px 12px; border-radius: 999px;
  background: var(--panel2); border: 1px solid var(--line); color: var(--muted); font-size: 12px; }
.chip .led { width: 8px; height: 8px; border-radius: 50%; background: #394559; }
.chip.active { color: var(--text); border-color: var(--accent2); }
.chip.active .led { background: var(--accent2); box-shadow: 0 0 10px var(--accent2); animation: pulse 1s
    infinite; }
.chip.done { color: var(--text); border-color: var(--accent); }
.chip.done .led { background: var(--accent); }
@keyframes pulse { 0%,100%{opacity:1} 50%{opacity:.4} }

.grid { display: grid; grid-template-columns: repeat(2, 1fr); gap: 16px; padding: 20px 28px 60px; }
.card { background: var(--panel); border: 1px solid var(--line); border-radius: 14px; padding: 18px; }
.card.span2 { grid-column: span 2; }
.card h2 { margin: 0 0 12px; font-size: 14px; color: var(--text); }
.muted { color: var(--muted); }

table { width: 100%; border-collapse: collapse; font-size: 12.5px; }
th, td { text-align: left; padding: 6px 8px; border-bottom: 1px solid var(--line); }
```

```
th { color: var(--muted); font-weight: 600; }
.bar { height: 7px; border-radius: 4px; background: var(--accent2); }
.tag { display: inline-block; padding: 2px 8px; border-radius: 6px; font-size: 11px;
  border: 1px solid var(--line); margin: 2px 4px 2px 0; }
.tag.good { color: var(--good); border-color: var(--good); }
.tag.warn { color: var(--warn); border-color: var(--warn); }
.tag.bad { color: var(--bad); border-color: var(--bad); }
.verdict { font-weight: 800; font-size: 18px; }
.verdict.GO { color: var(--good); }
.verdict.CONDITIONAL_GO { color: var(--warn); }
.verdict.NO_GO { color: var(--bad); }
.kv { display: flex; justify-content: space-between; border-bottom: 1px dashed var(--line); padding: 4px
    0; }
.cite { color: var(--accent2); font-size: 11px; }
.delta-pos { color: var(--good); font-weight: 700; }
footer { position: fixed; bottom: 0; left: 0; right: 0; padding: 8px 28px;
  background: var(--panel2); border-top: 1px solid var(--line); font-size: 12px; color: var(--muted); }
footer a { color: var(--accent2); text-decoration: none; }
h3 { font-size: 13px; margin: 14px 0 6px; }
small { color: var(--muted); }
```

I 宣传 Landing 页

web/landing/index.html (152 行)

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Anker AI 原生产品定义系统 · AI 先锋未来人才大赛</title>
<style>
  :root{--bg:#0a0e16;--panel:#121a28;--line:#22304a;--text:#eaf1fb;--muted:#8fa0bb;
        --accent:#36d3a0;--accent2:#5b8cff;--warn:#f0b429;--bad:#f0506e}
  *{box-sizing:border-box}
  body{margin:0;background:radial-gradient(1200px 600px at 70% -10%,#16233b 0%,var(--bg) 55%);
       color:var(--text);font-family:-apple-system,"PingFang SC","Segoe UI",Roboto,sans-serif;line-height
    :1.7}
  a{color:var(--accent2);text-decoration:none}
  .wrap{max-width:1080px;margin:0 auto;padding:0 22px}
  header{padding:18px 0;display:flex;justify-content:space-between;align-items:center;
         border-bottom:1px solid var(--line);position:sticky;top:0;background:rgba(10,14,22,.82);backdrop-
    filter:blur(8px);z-index:9}
  .logo{display:flex;align-items:center;gap:10px;font-weight:800}
  .logo .d{width:12px;height:12px;border-radius:50%;background:var(--accent);box-shadow:0 0 16px var(--
    accent)}
  nav a{margin-left:18px;color:var(--muted);font-size:14px}
  nav a:hover{color:var(--text)}
  .hero{padding:70px 0 40px}
  .badge{display:inline-block;border:1px solid var(--line);color:var(--muted);border-radius:999px;
         padding:5px 14px;font-size:12.5px;margin-bottom:18px}
  h1{font-size:46px;line-height:1.15;margin:0 0 16px;letter-spacing:-.5px}
  h1 .g{background:linear-gradient(90deg,var(--accent),var(--accent2));-webkit-background-clip:text;
    background-clip:text;color:transparent}
```

```
  .sub{font-size:18px;color:var(--muted);max-width:760px}
  .cta{margin-top:28px;display:flex;gap:12px;flex-wrap:wrap}
  .btn{padding:12px 22px;border-radius:10px;font-weight:700;border:1px solid var(--line)}
  .btn.primary{background:var(--accent);color:#04130d;border:0}
  .btn.ghost{background:transparent;color:var(--text)}
  .stats{display:grid;grid-template-columns:repeat(4,1fr);gap:16px;margin:46px 0}
  .stat{background:var(--panel);border:1px solid var(--line);border-radius:14px;padding:20px;text-align:
    center}
  .stat .n{font-size:30px;font-weight:800;color:var(--accent)}
  .stat .l{color:var(--muted);font-size:13px;margin-top:4px}
  section{padding:42px 0;border-top:1px solid var(--line)}
  h2{font-size:26px;margin:0 0 8px}
  .lead{color:var(--muted);margin:0 0 26px;max-width:780px}
  .grid3{display:grid;grid-template-columns:repeat(3,1fr);gap:16px}
  .card{background:var(--panel);border:1px solid var(--line);border-radius:14px;padding:20px}
  .card h3{margin:0 0 8px;font-size:16px}
  .card p{color:var(--muted);font-size:14px;margin:0}
  .role{display:flex;align-items:center;gap:8px;font-weight:700;margin-bottom:6px}
  .pill{display:inline-block;background:#0f1a2c;border:1px solid var(--line);border-radius:8px;
        padding:3px 10px;font-size:12px;color:var(--muted);margin:3px 4px 0 0}
  table{width:100%;border-collapse:collapse;background:var(--panel);border:1px solid var(--line);border-
    radius:14px;overflow:hidden}
  th,td{padding:11px 14px;border-bottom:1px solid var(--line);text-align:left;font-size:14px}
  th{color:var(--muted);font-weight:600}
  td.b{color:var(--accent);font-weight:700}
  .flow{display:flex;flex-wrap:wrap;gap:8px;align-items:center;margin-top:10px}
  .node{background:#0f1a2c;border:1px solid var(--accent2);border-radius:10px;padding:8px 12px;font-size
    :13px}
  .arrow{color:var(--muted)}
  footer{padding:40px 0 70px;border-top:1px solid var(--line);color:var(--muted);font-size:14px}
  .links a{margin-right:18px}
  .author{margin-top:10px}
  @media(max-width:760px){.stats,.grid3{grid-template-columns:1fr 1fr}h1{font-size:34px}}
</style>
</head>
<body>
<header><div class="wrap" style="display:flex;justify-content:space-between;align-items:center;width:100%"
    >
  <div class="logo"><span class="d"></span> Anker AI 原生产品定义系统</div>
  <nav>
    <a href="#what">能做什么</a><a href="#how">方法论</a><a href="#proof">本质不同</a>
    <a href="app/" > 在线 Demo</a>
  </nav>
</div></header>

<div class="wrap">
  <div class="hero">
    <span class="badge">2026 AI 先锋未来人才大赛 · 安克创新命题 · 华南赛区</span>
    <h1>让 AI 做你的<span class="g">超级智囊 · 用户替身 · 行业专家</span><br/>从真实数据，到产品定义</h1>
    <p class="sub">这不是一份 PPT，而是一套真实可运行、数据可溯源、可评测的多智能体系统：把安克 JML / BEES /
     AMI
       三大产品方法论平台做成 AI 原生版，端到端从真实用户评论跑出一份产品提案（PR/FAQ），
       并用一个可量化对比实验回答——AI 驱动相较"经验拍脑袋"到底本质不同在哪。</p>
    <div class="cta">
      <a class="btn primary" href="app/"> 打开在线 Demo</a>
      <a class="btn ghost" href="https://github.com/bcefghj/anker-ai-product-studio" target="_blank">
     GitHub 源码</a>
```

```
      <a class="btn ghost" href="https://bcefghj.github.io" target="_blank">作者主页</a>
    </div>
  </div>

  <div class="stats">
    <div class="stat"><div class="n">100%</div><div class="l">AI 命中真实痛点率<br/>(经验驱动仅 33%)</div></div>
    <div class="stat"><div class="n">+56</div><div class="l">可溯源证据引用<br/>(经验驱动为 0)</div></div>
    <div class="stat"><div class="n">+30</div><div class="l">预测 NPS 提升<br/>(同一 brief 对比)</div></div>
    <div class="stat"><div class="n">5</div><div class="l">AI 角色协同<br/>StateGraph 编排</div></div>
  </div>

  <section id="what">
    <h2>它做了什么</h2>
    <p class="lead">输入一句 brief，系统自动跑完 8 步，产出一份完整、可溯源的产品提案。</p>
    <div class="grid3">
      <div class="card"><div class="role">用户洞察 · JML</div><p>对真实评论做确定性 ABSA → ODI 机会评分 → 机会解决方案树，找出真正未被满足的痛点。</p></div>
      <div class="card"><div class="role">市场/竞品 · AMI</div><p>逐家拆解竞品（Bose/Sony/Samsung/Apple）短板，识别"白空间机会"，叠加行业趋势。</p></div>
      <div class="card"><div class="role">产品经理 · 双路径</div><p>VOC 驱动（从真实痛点反推）+ 第一性原理/技术原生（从端侧 AI / Thus 芯片正推）。</p></div>
      <div class="card"><div class="role">用户替身 · 合成用户</div><p>按真实评论聚类成 OCEAN 画像人群，假设盲访谈，反谄媚地挑战方案。</p></div>
      <div class="card"><div class="role">行业专家</div><p>技术 / BOM / 供应链 / 合规可行性 + Reflexion 批判，给出风险与缓解。</p></div>
      <div class="card"><div class="role">决策官</div><p>NPS 预测 + GO/NO-GO + 可审计 DecisionRecord（支持 HITL 人工闸口）。</p></div>
    </div>
    <div class="flow">
      <span class="node">真实评论</span><span class="arrow">→</span>
      <span class="node">JML 洞察</span><span class="arrow">+</span>
      <span class="node">AMI 竞品</span><span class="arrow">→</span>
      <span class="node">概念(双路径)</span><span class="arrow">→</span>
      <span class="node">用户替身</span><span class="arrow">→</span>
      <span class="node">行业专家</span><span class="arrow">→</span>
      <span class="node">决策官</span><span class="arrow">→</span>
      <span class="node">PR/FAQ 提案</span>
    </div>
  </section>

  <section id="how">
    <h2>方法论（均为业界成熟方法，非自造）</h2>
    <p class="lead">Amazon Working Backwards（PR/FAQ） · Teresa Torres 机会解决方案树 · Ulwick ODI 机会评分 · OCEAN 合成用户 + 假设盲反谄媚。确定性核心产出数值，LLM 负责叙述/角色扮演/批判，强制逐字引用校验。</p>
    <div>
      <span class="pill">LangGraph 风格 StateGraph 编排</span>
      <span class="pill">Agentic RAG（BM25+TF-IDF 混合检索）</span>
      <span class="pill">逐字引用校验 · 反幻觉</span>
      <span class="pill">四层架构 + AI Rule + Pre-PR + 跨模型对抗 CR</span>
      <span class="pill">MiniMax M3 可选增强</span>
      <span class="pill">离线确定性模式 · 可复现</span>
    </div>
  </section>

  <section id="proof">
```

```
    <h2>本质不同：AI 驱动 vs 经验驱动</h2>
    <p class="lead">同一品类、同一 brief，A 组（单 LLM 拍脑袋）vs B 组（本系统全链路），在真实数据上量化对比。</p>
    <table>
      <tr><th>维度</th><th>A 经验驱动</th><th>B AI 原生</th></tr>
      <tr><td>命中真实痛点率</td><td>33%</td><td class="b">100%</td></tr>
      <tr><td>机会覆盖率</td><td>20%</td><td class="b">60%</td></tr>
      <tr><td>证据引用数</td><td>0</td><td class="b">56</td></tr>
      <tr><td>已验证假设数（合成用户）</td><td>0</td><td class="b">4</td></tr>
      <tr><td>可行性风险识别</td><td>0</td><td class="b">3</td></tr>
      <tr><td>预测 NPS</td><td>-29</td><td class="b">+1 (Δ +30)</td></tr>
    </table>
    <p class="lead" style="margin-top:16px">经验驱动凭直觉押"音质/续航/性价比"，但真实数据显示机会分最高的痛点其实是
      <b>连接稳定性/多点、佩戴舒适度、App 体验</b>——这正是"拍脑袋"的系统性盲区。</p>
  </section>

  <footer>
    <div class="links">
      <a href="app/">在线 Demo</a>
      <a href="https://github.com/bcefghj/anker-ai-product-studio" target="_blank">GitHub</a>
      <a href="https://bcefghj.github.io" target="_blank">个人主页</a>
    </div>
    <div class="author">参赛者：戴尚好 · 中国科学技术大学 · bcefghj@163.com</div>
    <div style="margin-top:6px;font-size:12px">© 2026 AI 先锋未来人才大赛 · 安克创新命题参赛作品。本页数据来自系统在样本数据上的真实运行。</div>
  </footer>
</div>
</body>
</html>
```

J 工程规范与运维脚本

.cursor/rules/00-engineering-baseline.mdc (47 行)

```
---
description: Always-on engineering baseline for the Anker AI-Native Product Definition Studio. Enforces layering, data models, logging, tool conventions and citation discipline.
globs:
alwaysApply: true
---

# 工程基线（Always-On）

> 本规则对标美团《用 Agent 评测思路管理 AI Coding》的"人人对齐 → 人机对齐"：
> 先固化团队共识，再把共识变成 AI 编码时强制加载的约束，**防止 AI 产出加速系统腐化**。

## 1. 四层架构（Starter / Application / Infrastructure / Common）
所有后端代码必须落在四层之一，禁止跨层反向依赖：

- `common/`：纯领域模型、配置、日志、工具注册、引用校验。不依赖其他层。
- `infrastructure/`：外部能力实现（LLM 网关、数据加载、RAG、媒体、观测）。只依赖 `common/`。
- `application/`：业务编排（graph 引擎、agents、platforms、methodology、evaluation、baseline）。依赖 `common/` 与 `infrastructure/` 的**接口**。
```

```
- `starter/`: 入口 (CLI、FastAPI) 。只做装配与暴露, 不写业务逻辑。

依赖方向只能是 `starter → application → infrastructure → common`。

## 2. 数据与类型
- 所有跨边界的数据结构必须是 **Pydantic v2 `BaseModel`**, 禁止裸 `dict` 在层间传递。
- 必须写**完整类型注解**; 模块顶部统一 `from __future__ import annotations` (兼容 Python 3.9) 。
- 字段含义不明显时用 `Field(..., description=...)`。

## 3. 日志
- 一律使用 **loguru** 的 `logger`, 禁止 `print` / 标准库 `logging`。
- 关键节点 (工具调用、检索、决策闸、重试) 必须 `logger.info` 记录结构化上下文。

## 4. 工具 (Tool) 约定
- 工具用 `@tool(ToolType.READ)` 或 `@tool(ToolType.WRITE)` 注册。
- `READ` 工具无副作用、可并发; `WRITE` 工具有副作用, **必须两步确认** (先 preview、人工/HITL 确认后才执行) 。
- 工具失败**返回结构化错误字符串而非抛异常** ("错误即信息", 交由上层模型/编排自修正) 。

## 5. 引用纪律 (可溯源, 反幻觉) —— 本项目的命门
- 任何"洞察 / 痛点 / 机会 / 结论"都必须携带 `evidence_ids`, 指向 `Evidence.source_id`。
- 生成后必须经 `common/citations.py` 的**确定性逐字命中校验**; 命中失败的论断要么重生成、要么标注为"未验证假设"。
- 禁止在没有 Evidence 支撑的情况下输出量化数字。

## 6. 确定性优先
- 能用确定性算法 (词典/统计/规则) 得到的结果, 不要交给 LLM 猜。
- LLM 用于"叙述 / 归纳 / 角色扮演 / 批判", **结构化数值由确定性核心产出**。这呼应安克 CIO 关于"大模型不确定性 vs 企业高确定性"的判断。

## 7. 异常与降级
- 每个外部调用要有 fallback: LLM 不可用 → 走 offline 确定性 provider; 检索为空 → 重写查询重试 (上限 3 次) 。
- 不允许静默失败。
```

.cursor/rules/10-agent-conventions.mdc (29 行)

```
---
description: Conventions for agents, the orchestration graph, and the AI-native product-definition workflow.
globs: backend/anker_studio/application/**
alwaysApply: false
---

# Agent 与编排约定 (渐进式加载)

## 角色边界 (编排类 vs 能力类)
- **编排类**: `product_manager` (主编排) 、`graph/engine`。负责拆解、路由、迭代、收敛, 不直接做底层分析。
- **能力类**: `super_think_tank` / `user_proxy` / `industry_expert` / `decision_officer` 与 `platforms/*`。各自只负责单一职责, 输入/输出均为 Pydantic 模型。

## Agent 实现规范
- 每个 Agent 继承 `agents/base.py:Agent`, 实现 `run(state) -> <TypedResult>`。
- Agent 只能通过注入的 `LLMGateway` 与 `RagService` 访问外部能力, 禁止自行 import 具体 provider。
- Agent 产出的每条论断都要带 `evidence_ids`; 用 `cite()` 辅助。
- 涉及"用户替身"必须**假设盲**: 不得把研究假设/期望答案写进 persona 的 system prompt (反谄媚) 。

## 编排 (StateGraph)
```

```
- 节点签名：`(state: PipelineState) -> PipelineState`（返回增量合并）。
- 条件边返回下一个节点名或 `END`。
- `decision_officer` 前设 `interrupt_before`（HITL 闸口），并写入 `DecisionRecord`。
- 全程 `Tracer` 记录每个节点的输入摘要、耗时、token、引用命中率。

## 方法论锚点（不可省略）
- 机会量化：ODI（重要性 × 满意度差）+ Impact = Reach×(Severity+Value+Strategic)。
- 发现结构：Opportunity Solution Tree（结果 → 机会 → 方案 → 实验）。
- 产品定义产物：Working Backwards PR/FAQ。
- 决策指标：NPS 预测。
```

scripts/pre_pr_check.py (79 行)

```python
#!/usr/bin/env python3
"""Pre-PR 自查（对标美团 Pre-PR 机制）。

提交前强制运行：在不依赖外部服务的前提下做基础规范校验，过滤掉 AI 编码常见的
低级问题，让人工/跨模型 CR 只聚焦业务语义。

校验项（确定性、零依赖）：
  1. 后端模块是否落在四层架构目录内（layering）。
  2. Python 文件是否使用了 `print(` / 标准库 `logging`（应使用 loguru）。
  3. Python 模块是否声明 `from __future__ import annotations`。
  4. 是否存在裸 `except:`（吞异常）。

退出码非 0 表示未通过，应阻止提交。
"""
from __future__ import annotations

import re
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
BACKEND = ROOT / "backend" / "anker_studio"
ALLOWED_LAYERS = {"common", "infrastructure", "application", "starter"}


def _py_files(base: Path):
    return [p for p in base.rglob("*.py") if "__pycache__" not in p.parts]


def check_layering(violations: list[str]) -> None:
    if not BACKEND.exists():
        return
    for p in _py_files(BACKEND):
        rel = p.relative_to(BACKEND)
        if rel.name == "__init__.py" and len(rel.parts) == 1:
            continue
        top = rel.parts[0]
        if top not in ALLOWED_LAYERS:
            violations.append(f"[layering] {rel} 不在四层架构目录内 {sorted(ALLOWED_LAYERS)}")


def check_style(violations: list[str]) -> None:
    if not BACKEND.exists():
```

```
        return
    print_re = re.compile(r"(^|\s)print\(")
    logging_re = re.compile(r"^\s*import logging|^\s*from logging\b")
    bare_except_re = re.compile(r"^\s*except\s*:\s*$")
    for p in _py_files(BACKEND):
        text = p.read_text(encoding="utf-8")
        rel = p.relative_to(ROOT)
        lines = text.splitlines()
        if "from __future__ import annotations" not in text:
            violations.append(f"[future] {rel} 缺少 `from __future__ import annotations`")
        # starter 层是程序入口，stdout 即产品输出，允许 print（不算违规）
        is_starter = "starter" in p.relative_to(BACKEND).parts
        for i, line in enumerate(lines, 1):
            if print_re.search(line) and "noqa" not in line and not is_starter:
                violations.append(f"[no-print] {rel}:{i} 使用了 print()（应用 loguru）")
            if logging_re.search(line):
                violations.append(f"[no-logging] {rel}:{i} 使用了标准库 logging（应用 loguru）")
            if bare_except_re.match(line):
                violations.append(f"[bare-except] {rel}:{i} 裸 except（应捕获具体异常）")


def main() -> int:
    violations: list[str] = []
    check_layering(violations)
    check_style(violations)
    if violations:
        print("Pre-PR 未通过，请修复以下问题：")
        for v in violations:
            print("  - " + v)
        return 1
    print("Pre-PR 通过：四层架构 / 日志 / 类型注解 / 异常处理 规范校验 OK。")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/cross_model_review.py (78 行)

```
#!/usr/bin/env python3
"""跨厂商模型对抗 Code Review（对标美团"高阶模型审低阶 + 不同厂商互审"）。

把一段 diff/文件交给一个"审查模型"（默认 MiniMax-M3），要求它按本仓 AI Rule
（四层架构 / Pydantic / loguru / 引用纪律 / 两步确认）输出结构化评审意见。

用法：
    python scripts/cross_model_review.py <file_or_dir> [--reviewer MiniMax-M3]

若未配置 MINIMAX_API_KEY，则退化为本地规则审查（提示用户配置后可得到语义级审查）。
"""
from __future__ import annotations

import argparse
import sys
from pathlib import Path

```

```
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "backend"))

REVIEW_RUBRIC = """你是一名资深架构师，按以下规范审查代码（仅输出问题清单，按严重度排序）：
1. 四层架构：starter→application→infrastructure→common 单向依赖，无反向/跨层依赖。
2. 数据模型用 Pydantic v2；完整类型注解；模块含 `from __future__ import annotations`。
3. 日志用 loguru，不用 print / logging。
4. 工具区分 READ/WRITE，WRITE 必须两步确认。
5. 引用纪律：洞察/结论必须带 evidence_ids，并可逐字命中校验。
6. 异常有 fallback，不静默失败。
输出格式：每行 `[严重度 高/中/低][规则编号] 文件:行 — 问题与修复建议`。无问题则输出 `PASS`。
"""


def collect_code(target: Path) -> str:
    files = [target] if target.is_file() else [
        p for p in target.rglob("*.py") if "__pycache__" not in p.parts
    ]
    chunks = []
    for p in files[:40]:
        chunks.append(f"\n===== FILE: {p.relative_to(ROOT)} =====\n" + p.read_text(encoding="utf-8"))
    return "\n".join(chunks)


def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("target", help="要审查的文件或目录")
    ap.add_argument("--reviewer", default="MiniMax-M3")
    args = ap.parse_args()

    target = Path(args.target).resolve()
    if not target.exists():
        print(f"目标不存在：{target}")
        return 2

    code = collect_code(target)
    try:
        from anker_studio.infrastructure.llm.gateway import LLMGateway
        from anker_studio.common.config import settings

        gw = LLMGateway.from_settings(settings())
        if not gw.has_remote:
            print("[降级] 未配置 MINIMAX_API_KEY，无法做跨厂商语义级审查。")
            print("       请 `export MINIMAX_API_KEY=...` 后重试，或先用 scripts/pre_pr_check.py 做规范校验。")
            return 0
        result = gw.complete(
            system=REVIEW_RUBRIC,
            prompt=f"请审查以下代码：\n{code}",
            model=args.reviewer,
            max_tokens=2048,
            task="code_review",
        )
        print(result.text)
        return 0
    except Exception as exc:  # noqa: BLE001 - 审查脚本对任何失败都应给出可读提示
        print(f"[降级] 跨模型审查不可用：{exc}")
```

```
        return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/deploy.py (188 行)

```
#!/usr/bin/env python3
"""一键部署到服务器（项目二：anker，端口 8766），不影响已有项目（小念/xiaonian）。

凭据从环境变量读取（绝不写入仓库）：
    SERVER_HOST, SERVER_USER, SERVER_PASS, [MINIMAX_API_KEY]

用法：
    SERVER_HOST=47.119.112.225 SERVER_USER=root SERVER_PASS=*** \
    MINIMAX_API_KEY=*** python3 scripts/deploy.py

依赖：paramiko（pip install paramiko）。
"""
from __future__ import annotations

import os
import subprocess
import sys
import tempfile
from pathlib import Path

REPO_URL = "https://github.com/bcefghj/anker-ai-product-studio.git"
APP_DIR = "/opt/projects/anker"
WWW_DIR = "/var/www/anker"
PORT = 8766
NGINX_CONF = "/etc/nginx/sites-available/projects"
# 国内服务器对 PyPI 官方源不稳定，默认走清华镜像（可用 PIP_INDEX 覆盖）
PIP_INDEX = os.environ.get("PIP_INDEX", "https://pypi.tuna.tsinghua.edu.cn/simple")
PROJECT_ROOT = Path(__file__).resolve().parents[1]

NGINX_BLOCK = """    # ---- 项目二：anker（AI 原生产品定义系统）----
    location = /anker/app { return 301 /anker/app/; }
    location /anker/app/ {
        proxy_pass http://127.0.0.1:8766/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_buffering off;
        proxy_read_timeout 300s;
    }
    location = /anker { return 301 /anker/; }
    location /anker/ {
        alias /var/www/anker/;
        index index.html;
        try_files $uri $uri/ /var/www/anker/index.html;
    }

"""
```

```

SYSTEMD_UNIT = """[Unit]
Description=Anker AI Native Product Definition Studio
After=network.target

[Service]
WorkingDirectory=/opt/projects/anker
EnvironmentFile=-/opt/projects/anker/.env
ExecStart=/opt/projects/anker/.venv/bin/python run.py
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
"""


def main() -> int:
    try:
        import paramiko
    except ImportError:
        print("缺少 paramiko：pip install paramiko")
        return 1

    host = os.environ["SERVER_HOST"]
    user = os.environ.get("SERVER_USER", "root")
    password = os.environ["SERVER_PASS"]
    minimax_key = os.environ.get("MINIMAX_API_KEY", "")

    cli = paramiko.SSHClient()
    cli.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    cli.connect(host, username=user, password=password, timeout=30)

    def run(cmd: str, quiet: bool = False) -> str:
        stdin, stdout, stderr = cli.exec_command(cmd, timeout=600)
        out = stdout.read().decode("utf-8", "ignore")
        err = stderr.read().decode("utf-8", "ignore")
        rc = stdout.channel.recv_exit_status()
        if not quiet:
            tag = "OK" if rc == 0 else f"RC={rc}"
            print(f"[{tag}] $ {cmd}")
            if out.strip():
                print("  " + out.strip().replace("\n", "\n  ")[:1500])
            if err.strip() and rc != 0:
                print("  ERR: " + err.strip().replace("\n", "\n  ")[:800])
        return out

    def put_file(remote: str, content: str) -> None:
        sftp = cli.open_sftp()
        with sftp.open(remote, "w") as f:
            f.write(content)
        sftp.close()
        print(f"[OK] 写入 {remote} ({len(content)} 字节)")

    print("\n===== 0. 预检（确认不碰已有项目）=====")
    run("systemctl is-active xiaonian.service nginx || true")
    run("ss -ltnp | grep ':8766' || echo '8766 端口空闲'")
```

```
106
107     print("\n===== 1. 打包并上传代码（绕过服务器 GitHub 访问不稳定）=====")
108     tar_path = Path(tempfile.gettempdir()) / "anker_deploy.tgz"
109     # 注意：不要排除名为 assets 的目录，否则会误删源码包 infrastructure/assets/
110     # COPYFILE_DISABLE=1：禁止 macOS bsdtar 生成 ._* AppleDouble 伴随文件
111     env = {**os.environ, "COPYFILE_DISABLE": "1"}
112     subprocess.run(
113         ["tar", "czf", str(tar_path),
114          "--exclude=.git", "--exclude=.venv", "--exclude=runs",
115          "--exclude=__pycache__", "--exclude=.env", "--exclude=node_modules",
116          "--exclude=data/processed", "-C", str(PROJECT_ROOT), "."],
117         check=True, env=env,
118     )
119     print(f"[OK] 本地打包 {tar_path}（{tar_path.stat().st_size} 字节）")
120     sftp = cli.open_sftp()
121     sftp.put(str(tar_path), "/tmp/anker_deploy.tgz")
122     sftp.close()
123     print("[OK] 上传到 /tmp/anker_deploy.tgz")
124     run(f"mkdir -p {APP_DIR} && tar xzf /tmp/anker_deploy.tgz -C {APP_DIR} && "
125         f"find {APP_DIR} -name '._*' -delete && ls {APP_DIR} | head")
126
127     print("\n===== 2. venv + 依赖（清华镜像）=====")
128     run("which python3 && python3 --version")
129     run(f"cd {APP_DIR} && (python3 -m venv .venv || (apt-get update && apt-get install -y python3-venv &&
        python3 -m venv .venv))")
130     run(f"cd {APP_DIR} && .venv/bin/pip install -q --upgrade pip -i {PIP_INDEX} && "
131         f".venv/bin/pip install -q -r requirements.txt -i {PIP_INDEX} && echo deps-ok")
132
133     print("\n===== 3. 样本数据 =====")
134     run(f"cd {APP_DIR} && .venv/bin/python data/make_sample.py | tail -2")
135
136     print("\n===== 4. 写 .env（不入 git）=====")
137     env_content = (
138         f"ANKER_LLM_PROVIDER=offline\n"
139         f"PORT={PORT}\n"
140         f"MINIMAX_API_KEY={minimax_key}\n"
141         f"MINIMAX_BASE_URL=https://api.minimax.io/v1\n"
142         f"MINIMAX_MODEL=MiniMax-M3\n"
143     )
144     put_file(f"{APP_DIR}/.env", env_content)
145
146     print("\n===== 5. systemd 服务 =====")
147     put_file("/etc/systemd/system/anker.service", SYSTEMD_UNIT)
148     run("systemctl daemon-reload && systemctl enable anker.service && systemctl restart anker.service")
149     run("sleep 2 && systemctl is-active anker.service")
150
151     print("\n===== 6. 落地静态宣传页 =====")
152     run(f"mkdir -p {WWW_DIR}")
153     landing = (Path(__file__).resolve().parents[1] / "web" / "landing" / "index.html").read_text(encoding=
        "utf-8")
154     put_file(f"{WWW_DIR}/index.html", landing)
155
156     print("\n===== 7. nginx 路由（幂等插入）=====")
157     conf = run(f"cat {NGINX_CONF}", quiet=True)
158     if "/anker/app/" in conf:
159         print("  anker 路由已存在，跳过插入。")
160     else:
```

```
        marker = None
        for cand in ["    # ---- 根导航 hub", "    # ---- 根路径", "    location / {"]:
            if cand in conf:
                marker = cand
                break
        if marker:
            conf = conf.replace(marker, NGINX_BLOCK + marker, 1)
            put_file(NGINX_CONF, conf)
            print(f"  已在 '{marker.strip()}' 之前插入 anker 路由。")
        else:
            print("  !! 未找到根路径标记，未自动插入，请手动检查 nginx 配置。")
    run("nginx -t && systemctl reload nginx")

    print("\n===== 8. 健康检查 =====")
    run("curl -s -o /dev/null -w 'local-app %{http_code}\\n' http://127.0.0.1:8766/api/health")
    run("curl -s -o /dev/null -w 'hub %{http_code}\\n' http://127.0.0.1/")
    run("curl -s -o /dev/null -w 'xiaonian %{http_code}\\n' http://127.0.0.1/xiaonian/")
    run("curl -s -o /dev/null -w 'anker-landing %{http_code}\\n' http://127.0.0.1/anker/")
    run("curl -s -o /dev/null -w 'anker-demo %{http_code}\\n' http://127.0.0.1/anker/app/")
    run("curl -s http://127.0.0.1/anker/app/api/health")

    cli.close()
    print("\n部署完成。")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

scripts/server_exec.py (35 行)

```
#!/usr/bin/env python3
"""在服务器上执行一条命令（运维/诊断用）。凭据从环境变量读取，不写入仓库。

    SERVER_HOST=.. SERVER_USER=root SERVER_PASS=.. python3 scripts/server_exec.py "命令"
"""
from __future__ import annotations

import os
import sys


def main() -> int:
    import paramiko

    cmd = " ".join(sys.argv[1:])
    if not cmd:
        print("用法: server_exec.py '<command>'")
        return 2
    cli = paramiko.SSHClient()
    cli.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    cli.connect(os.environ["SERVER_HOST"], username=os.environ.get("SERVER_USER", "root"),
                password=os.environ["SERVER_PASS"], timeout=30)
    stdin, stdout, stderr = cli.exec_command(cmd, timeout=600)
    out = stdout.read().decode("utf-8", "ignore")
    err = stderr.read().decode("utf-8", "ignore")
```

```
    rc = stdout.channel.recv_exit_status()
    sys.stdout.write(out)
    if err.strip():
        sys.stderr.write("\n[stderr]\n" + err)
    cli.close()
    return rc


if __name__ == "__main__":
    sys.exit(main())
```

scripts/update_hub.py　(60 行)

```
#!/usr/bin/env python3
"""把服务器导航 hub 里的"项目二"占位卡片替换为 Anker 项目的"运行中"卡片。

幂等：已替换则跳过。凭据从环境变量读取（SERVER_HOST/SERVER_USER/SERVER_PASS）。
"""
from __future__ import annotations

import os
import sys

HUB = "/var/www/hub/index.html"

NEW_CARD = """    <div class="proj live">
      <div class="face">  </div>
      <span class="badge on">  运行中</span>
      <h3>Anker AI 原生产品定义系统</h3>
      <div class="desc">让 AI 做超级智囊 · 用户替身 · 行业专家，从真实评论端到端跑出产品提案(PR/FAQ)，并量化对比"
     AI 驱动 vs 经验拍脑袋"。2026 AI 先锋未来人才大赛 · 安克命题参赛作品。</div>
      <a class="enter" href="/anker/">查看项目 →</a>
      <div class="links">
        <a href="/anker/app/" target="_blank">在线 Demo</a>
        <a href="https://github.com/bcefghj/anker-ai-product-studio" target="_blank">GitHub</a>
      </div>
    </div>"""


def main() -> int:
    import paramiko

    cli = paramiko.SSHClient()
    cli.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    cli.connect(os.environ["SERVER_HOST"], username=os.environ.get("SERVER_USER", "root"),
                password=os.environ["SERVER_PASS"], timeout=30)
    sftp = cli.open_sftp()
    with sftp.open(HUB, "r") as f:
        html = f.read().decode("utf-8")

    if "/anker/" in html and "Anker AI 原生" in html:
        print("hub 已包含 anker 卡片，跳过。")
        return 0

    anchor = "<h3>项目二</h3>"
    if anchor not in html:
```

```
        print("未找到 '项目二' 占位卡片，跳过（请手动检查 hub）。")
        return 1
    pos = html.index(anchor)
    start = html.rindex('<div class="proj soon">', 0, pos)
    end = html.index("</div>", html.index("敬请期待", pos)) + len("</div>")
    end = html.index("</div>", end) + len("</div>")  # 卡片外层闭合
    new_html = html[:start] + NEW_CARD + html[end:]

    with sftp.open(HUB, "w") as f:
        f.write(new_html)
    sftp.close()
    cli.close()
    print(f"hub 已更新：注入 anker 运行中卡片（{len(new_html)} 字节）。")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

data/make_sample.py (133 行)

```
#!/usr/bin/env python3
"""生成可复现的样本评论数据集（soundcore + 竞品）。

样本是"合成但贴近真实"的英文评论，用于在没有下载 Amazon Reviews 2023 时也能
端到端跑通、并让 ABSA 产出有意义的差异。真实运行请用
`python -m anker_studio.infrastructure.data.amazon_reviews` 下载真实数据。
"""
from __future__ import annotations

import json
import random
from pathlib import Path

OUT = Path(__file__).resolve().parent / "sample"
OUT.mkdir(parents=True, exist_ok=True)

YEARS = ["2021", "2022", "2023", "2024", "2025"]

# 每个 aspect 的正/负面句子模板（英文，贴合真实 Amazon 评论与词典）
PHRASES = {
    "anc": {
        "pos": ["The noise cancelling is excellent and blocks out the subway completely.",
                "ANC is amazing, great for flights."],
        "neg": ["The noise cancellation is weak and barely blocks any noise.",
                "ANC is disappointing compared to what they advertise."],
    },
    "sound": {
        "pos": ["Sound quality is great, the bass is clear and punchy.",
                "Amazing audio quality, the soundstage is impressive."],
        "neg": ["The sound is muffled and the bass is weak.",
                "Audio quality is poor and treble sounds harsh."],
    },
    "fit": {
        "pos": ["Very comfortable fit, I can wear them for hours.",
                "The ear tips are comfortable and they never fall out."],
```

```
        "neg": ["Uncomfortable fit, they hurt my ears after an hour.",
                "They keep falling out of my ears during workouts."],
    },
    "call": {
        "pos": ["Call quality is clear and the mic picks up my voice well.",
                "People say my voice is crisp on calls."],
        "neg": ["Call quality is terrible, callers say I sound muffled in wind.",
                "The microphone is poor on calls, lots of background noise."],
    },
    "battery": {
        "pos": ["Battery life is great, easily lasts all day.",
                "Impressive playtime, I charge them once a week."],
        "neg": ["Battery life is disappointing and drains fast.",
                "The battery degraded quickly after a few months."],
    },
    "app": {
        "pos": ["The app is good and the EQ is easy to use.",
                "Firmware updates through the app are smooth."],
        "neg": ["The app is buggy and the EQ settings keep resetting.",
                "Firmware update bricked the connection, the app is awful."],
    },
    "connect": {
        "pos": ["Bluetooth connection is stable and pairing is fast.",
                "Multipoint works great switching between phone and laptop."],
        "neg": ["The connection keeps dropping and multipoint switching fails.",
                "Bluetooth pairing is unreliable and there is annoying latency."],
    },
    "price": {
        "pos": ["Great value for the money, worth every penny.",
                "Cheap compared to the big brands and works well."],
        "neg": ["Way too expensive for what you get.",
                "Not worth the price, overpriced for the features."],
    },
    "durable": {
        "pos": ["Solid build quality, feels durable.",
                "Well built, no issues after a year."],
        "neg": ["One earbud stopped working after two months, poor quality control.",
                "The hinge broke quickly, bad build quality."],
    },
}

# 各品牌在各 aspect 上的"负面概率"(刻画差异，贴近公开认知，仅用于样本)
BRAND_NEG = {
    "soundcore": {"anc": 0.35, "sound": 0.25, "fit": 0.3, "call": 0.6, "battery": 0.3,
                  "app": 0.55, "connect": 0.6, "price": 0.15, "durable": 0.4},
    "Bose":      {"anc": 0.1, "sound": 0.2, "fit": 0.25, "call": 0.4, "battery": 0.45,
                  "app": 0.4, "connect": 0.45, "price": 0.7, "durable": 0.35},
    "Sony":      {"anc": 0.15, "sound": 0.15, "fit": 0.35, "call": 0.5, "battery": 0.3,
                  "app": 0.55, "connect": 0.4, "price": 0.55, "durable": 0.3},
    "Samsung":   {"anc": 0.5, "sound": 0.35, "fit": 0.3, "call": 0.45, "battery": 0.35,
                  "app": 0.4, "connect": 0.35, "price": 0.4, "durable": 0.4},
    "Apple":     {"anc": 0.25, "sound": 0.25, "fit": 0.55, "call": 0.3, "battery": 0.4,
                  "app": 0.2, "connect": 0.2, "price": 0.75, "durable": 0.35},
}
PRODUCTS = {
    "soundcore": "soundcore Liberty 4 NC", "Bose": "Bose QuietComfort Ultra Earbuds",
    "Sony": "Sony WF-1000XM5", "Samsung": "Samsung Galaxy Buds3 Pro", "Apple": "Apple AirPods Pro 2",
```

```
}


def gen_brand(brand: str, n: int, rng: random.Random):
    rows = []
    aspects = list(PHRASES.keys())
    for i in range(n):
        k = rng.choice([1, 2, 2, 3])
        chosen = rng.sample(aspects, k)
        texts, neg_any = [], False
        for asp in chosen:
            is_neg = rng.random() < BRAND_NEG[brand][asp]
            neg_any = neg_any or is_neg
            texts.append(rng.choice(PHRASES[asp]["neg" if is_neg else "pos"]))
        rating = rng.choice([1, 2, 3]) if neg_any else rng.choice([4, 5, 5])
        rows.append({
            "source_id": f"{brand}-{i:04d}",
            "brand": brand,
            "product": PRODUCTS[brand],
            "rating": rating,
            "text": " ".join(texts),
            "date": rng.choice(YEARS),
            "helpful_votes": rng.randint(0, 40),
        })
    return rows


def main() -> None:
    rng = random.Random(42)
    counts = {"soundcore": 120, "Bose": 70, "Sony": 70, "Samsung": 60, "Apple": 70}
    for brand, n in counts.items():
        rows = gen_brand(brand, n, rng)
        path = OUT / f"{brand}.jsonl"
        with path.open("w", encoding="utf-8") as fp:
            for r in rows:
                fp.write(json.dumps(r, ensure_ascii=False) + "\n")
        print(f"wrote {len(rows)} -> {path}")


if __name__ == "__main__":
    main()
```

backend/tests/test_smoke.py (45 行)

```
"""冒烟测试: 可用 pytest 运行, 也可 `python backend/tests/test_smoke.py` 直接运行。

验证 AI 原生工作流端到端跑通并满足关键不变量 (offline 确定性模式) 。
"""
from __future__ import annotations

import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))


```

```
def _run():
    from anker_studio.application.runner import run_studio
    return run_studio(thread_id="test")


def test_pipeline_end_to_end():
    art = _run()
    assert not art.awaiting_human
    assert art.state.voc_report is not None
    assert art.state.voc_report.review_count > 0
    assert art.state.chosen_concept is not None
    assert art.state.decision is not None
    assert len(art.state.personas) >= 1


def test_grounding_and_comparison():
    art = _run()
    rub = art.rubric
    assert rub is not None
    # 所有论断都应带引用
    assert rub.groundedness >= 0.9
    assert rub.citation_hit_rate >= 0.9
    # AI 原生应优于经验驱动
    cmp = art.comparison
    assert cmp is not None
    assert cmp.arm_b.evidence_citations > cmp.arm_a.evidence_citations
    assert cmp.arm_b.real_pain_hit_rate >= cmp.arm_a.real_pain_hit_rate


if __name__ == "__main__":
    test_pipeline_end_to_end()
    test_grounding_and_comparison()
    print("OK: smoke tests passed")
```